



RTL Design Sherpa

DDR2 / LPDDR2 Family Controller Hardware Architecture Specification 0.9

June 13, 2026

Table of Contents

1 Pumice Has Index.....	19
2 FUB Breakdown.....	20
2.1 Component Layout.....	20
2.2 Layer Hierarchy.....	20
2.3 FUB Inventory.....	21
2.3.1 pumice_axi4_layer.....	21
2.3.2 pumice_scheduler_layer.....	23
2.3.3 pumice_training_layer.....	24
2.3.4 pumice_dfi_layer.....	25
2.4 Data Width and Gearing.....	26
2.5 What Changed vs the Earlier Architecture.....	26
2.6 Integration.....	27
3 Training: LPDDR2 ZQ and DQ Calibration.....	29
3.1 The family rule: interfaces in hardware, searches in firmware.....	29
3.2 pumice_training_layer: arbitration and CDC.....	29
3.3 pumice_zq_ctrl: periodic ZQ calibration as maintenance traffic.....	30
3.4 pumice_lp_cal: one-shot DQ calibration over MRR.....	31
3.5 The MRR path: one new bit, not a new opcode.....	32
3.6 Arbitration policy.....	33
3.7 CSR summary.....	33
3.8 Verification and integration notes.....	35
4 Document Information.....	36

4.1 DDR2 / LPDDR2 Family Controller Hardware Architecture Specification.....	36
4.2 Document Purpose.....	36
4.3 Revision History.....	36
4.4 Scope of This Document.....	41
4.5 Out of Scope.....	41
5 Purpose and Scope.....	43
5.1 Document Purpose.....	43
5.2 Scope.....	43
5.3 Design Philosophy.....	44
5.4 Audience.....	45
6 Document Conventions.....	46
6.1 Typographic Conventions.....	46
6.2 Numbering Conventions.....	46
6.3 Signal Direction Conventions.....	46
6.4 Module Diagram Conventions.....	46
6.5 Cross-Reference Conventions.....	47
7 Definitions and Acronyms.....	48
7.1 Acronyms.....	48
7.2 Defined Terms.....	50
8 Scope and Goals.....	52
8.1 High-Level Goals.....	52
8.2 In Scope.....	52
8.3 Out of Scope.....	53

8.4 Target Performance Envelope.....	54
8.5 Differentiating Features.....	54
8.5.1 Features Adopted from Standard Memory-Controller Patterns.....	56
9 Block Diagram.....	58
9.1 Top-Level Block Diagram.....	58
9.2 Data Flow Summary.....	59
10 Module Hierarchy.....	61
10.1 Hierarchy Tree.....	61
10.2 Layered Organization.....	61
10.3 Layer Groupings.....	62
10.4 Memtype-Conditional Logic.....	62
10.5 Clock-Domain Crossings.....	63
10.6 What Changed vs the Earlier Architecture.....	63
11 Top-Level Interfaces.....	64
11.1 Parameter-Dependent Widths.....	64
11.2 1. AXI4 Slave (Host-side).....	65
11.2.1 Write Address Channel (AW).....	65
11.2.2 Write Data Channel (W).....	65
11.2.3 Write Response Channel (B).....	66
11.2.4 Read Address Channel (AR).....	66
11.2.5 Read Data Channel (R).....	66
11.3 2. DFI v2.1 Master (PHY-side).....	67
11.3.1 DFI Command Bus.....	67

11.3.2 DFI Write-Data Bus.....	68
11.3.3 DFI Read-Data Bus.....	68
11.3.4 DFI Init Handshake.....	68
11.4 3. CSR Register Interface (Configuration / Observation).....	68
11.5 4. Clocks, Resets, and Status.....	69
11.6 Hierarchical Top-Level View.....	69
12 AXI4 Frontend.....	71
12.1 Burst Splitters.....	71
12.2 pumice_wr_intake / pumice_rd_intake.....	71
12.2.1 Purpose.....	71
12.2.2 Write intake.....	71
12.2.3 Read intake.....	72
12.2.4 Per-ID Ordering.....	72
12.2.5 Backpressure.....	72
12.3 addr_mapper.....	72
12.3.1 Purpose.....	72
12.3.2 Interfaces.....	72
12.3.3 Mapping Function (single knob: bank_lsb).....	73
12.3.4 Classic Schemes as bank_lsb Settings.....	73
12.3.5 Optional Bank XOR-Hash.....	74
12.3.6 Relationship to the DFI BFM.....	74
13 Command Scheduler.....	75
13.1 Composition.....	75

13.2	pumice_cmd_arbiter.....	75
13.2.1	Purpose.....	75
13.2.2	Inputs.....	76
13.2.3	Outputs.....	76
13.2.4	Priority Function.....	76
13.2.5	ACT/PRE Re-Issue Guard.....	77
13.2.6	Page Policy (Auto-Precharge).....	77
13.2.7	Issue Rate.....	78
13.3	Mode Register Shadow.....	78
13.4	v1 Scope Notes.....	78
14	Bank Timers and Cross-Bank Timers.....	79
14.1	bank_timer.....	79
14.1.1	Purpose.....	79
14.1.2	Instantiation.....	79
14.1.3	Timers.....	79
14.1.4	Row State (minimal — not an FSM).....	80
14.1.5	Combinational Safe Outputs.....	80
14.1.6	Observability State.....	80
14.1.7	Refresh Interaction.....	80
14.2	global_timers.....	81
14.2.1	Purpose.....	81
14.2.2	Tracked Constraints.....	81
14.2.3	Interface.....	81

15 Refresh Controller.....	82
15.1 refresh_ctrl.....	82
15.1.1 Purpose.....	82
15.1.2 Interface.....	82
15.1.3 tREFI Counter and Pending Accumulator.....	82
15.1.4 Drain Quota.....	83
15.1.5 REFpb Bank Rotor.....	83
15.2 Arbiter-Side Refresh Sequence.....	83
15.3 Notes and Scope.....	84
16 Init Sequencer and Power-Down.....	85
16.1 init_sequencer.....	85
16.1.1 Purpose.....	85
16.1.2 Command Path.....	85
16.1.3 CSR-Backed Waits.....	85
16.1.4 DDR2 Sequence.....	85
16.1.5 LPDDR2 Sequence (fully functional).....	86
16.1.6 Memtype Selection.....	86
16.1.7 Status Outputs.....	87
16.2 powerdown_ctrl.....	87
16.2.1 Purpose.....	87
16.2.2 Modes.....	87
16.2.3 Selection.....	87
16.2.4 Scope / TODO.....	87

17 Command Formatter and Signal Pack.....	89
17.1 dfi_cmd_formatter.....	89
17.1.1 Purpose.....	89
17.1.2 Input.....	89
17.1.3 Multi-Phase Output.....	89
17.1.4 DDR2 Encoding.....	89
17.1.5 LPDDR2 Encoding (bit-exact CA bus).....	90
17.1.6 Idle Defaults.....	91
17.2 dfi_signal_pack.....	91
17.2.1 Purpose.....	91
17.2.2 Behavior.....	91
17.2.3 Multi-Cycle Commands.....	91
17.2.4 Verification.....	91
18 Write and Read Data Paths.....	93
18.1 Controller-Side CAMs.....	93
18.1.1 pumice_wr_data_cam.....	93
18.1.2 pumice_rd_cmd_cam.....	93
18.2 DFI-Side Data Movers.....	94
18.2.1 pumice_dfi_wr_serializer.....	94
18.2.2 pumice_dfi_rd_aligner.....	94
18.3 Response Generation.....	94
18.4 Runtime PHY Timing.....	94
19 DFI Layer and CSR.....	96

19.1 pumice_dfi_layer.....	96
19.1.1 Purpose.....	96
19.1.2 The Single CDC.....	96
19.1.3 DFI-Domain Datapath.....	96
19.1.4 DFI Sub-Interfaces.....	97
19.1.5 Runtime PHY Knobs.....	97
19.2 pumice_csr.....	97
19.2.1 Purpose.....	97
19.2.2 Interface.....	97
19.2.3 Access by Name.....	98
19.2.4 Address Mapping Register (ADDR_MAP @ 0x04C).....	98
19.2.5 Key Configuration Registers.....	98
19.2.6 Narrow-Device (x16) Width Granularities.....	99
20 AXI4 Slave Interface.....	100
20.1 Data Width Is Fixed at DW.....	100
20.1.1 Hard width rule (compile-enforced).....	100
20.2 Channels.....	100
20.2.1 AW (Write Address) — SoC master → controller slave.....	100
20.2.2 W (Write Data).....	101
20.2.3 B (Write Response).....	101
20.2.4 AR (Read Address).....	102
20.2.5 R (Read Data).....	102
20.3 Ordering Rules.....	103

20.4 Burst Type Support.....	103
20.5 Burst Length and DRAM-Burst Geometry.....	103
20.6 Narrow Transfers.....	103
20.7 Write Response Timing.....	103
20.8 Frontend and Backpressure.....	104
20.9 Host-Width Gearing.....	104
20.10 AXI4 Reset.....	104
21 DFI v2.1 Master Interface.....	105
21.1 Gearing and Per-Phase Packing.....	105
21.2 Command Interface (MC → PHY).....	105
21.2.1 DDR2 Command Encoding (JESD79-2).....	106
21.2.2 LPDDR2 Command Encoding (JESD209-2F).....	106
21.3 Write Data Interface (MC → PHY).....	107
21.4 Read Data Interface (PHY ↔ MC).....	107
21.5 Status / Init Interface.....	107
21.6 Timing Parameters Honored.....	108
21.7 Clock Domains and CDC.....	108
22 CSR Register Interface (PeakRDL cpuif).....	109
22.1 cpuif Signals.....	109
22.2 Access Semantics.....	109
22.3 Clock Domain.....	110
22.4 Register Map.....	110
22.5 Configuration Sequencing.....	110

23 Build-Time vs. Runtime Configuration.....	112
23.1 Principle.....	112
23.2 Why This Distinction Matters.....	112
23.3 The Hybrid Pattern: “Build-Time MAX + Runtime Active”	113
23.4 The “Build-Time + Runtime Mux” Pattern.....	113
23.5 The “Runtime Placement” Pattern.....	114
23.6 The “Runtime Only” Pattern.....	115
23.7 Configuration Quiet Points.....	115
23.8 Summary.....	115
24 Top-Level Parameter Table.....	117
24.1 Parameter Table.....	117
24.2 Memtype-Dependent Constraints.....	121
24.3 Timing-Configuration Sanity Checks.....	121
24.4 Parameter Categories.....	121
25 Characterization Knobs.....	123
25.1 Characterization Parameters.....	123
25.2 Sweep Plan.....	124
25.2.1 Benchmark Suite.....	125
25.2.2 Metrics Collected.....	125
25.2.3 Default Selection.....	125
25.3 Runtime Overrides via CSR.....	126
25.4 Observability for Sweep.....	126
26 Family-Wide Config Bits.....	128

26.1 Why a Family-Wide Registry.....	128
26.2 Applicability Notation.....	128
26.3 SCHED_TUNING (Scheduling Family Bits).....	129
26.4 REFRESH_TUNING (Refresh Family Bits).....	129
26.5 PAGE_POLICY_CFG / PAGE_TIMEOUT_CFG (Axis 2).....	130
26.6 ADDR_MAP (Address-Mapping Family Bits).....	134
26.7 INIT_TUNING (Init Family Bits).....	134
26.8 POWER_TUNING (Power-State Family Bits).....	135
26.9 ECC_TUNING (Future — Inline ECC).....	135
26.10 RANK_TUNING (Multi-Rank Family Bits).....	136
26.11 Quiet-Point Behavior.....	136
26.12 Bit Discovery via the ID Register.....	136
26.13 Per-Flavor Applicability Matrix.....	137
27 Clocks and Reset.....	139
27.1 Clock Domains.....	139
27.1.1 aclk.....	139
27.1.2 dfi_clk.....	139
27.2 Reset Signals.....	139
27.2.1 aresetn (active low, controller / AXI domain).....	139
27.2.2 dfi_rstn (active low, DFI / PHY domain).....	139
27.3 Reset Sequencing.....	140
27.3.1 Cold Boot.....	140
27.3.2 Warm Reset (Clock-Gated).....	140

27.4 Clock Domain Crossing Summary.....	140
27.5 Reset and Power Considerations.....	141
28 Init Sequences.....	142
28.1 DDR2 Cold Init.....	142
28.2 LPDDR2 Cold Init.....	143
28.3 Inter-Command Waits (CSR-Backed).....	144
28.4 Power-State Transitions.....	144
29 CSR Register Map.....	146
29.1 Control and Status (0x000 – 0x00F).....	146
29.1.1 CTRL (0x000, R/W).....	146
29.1.2 STATUS (0x004, R only).....	146
29.1.3 STATUS_HISTORY (0x008, R only).....	147
29.2 Timing Parameters (0x010 – 0x01F).....	147
29.2.1 TIMINGS_RC_RCD_RP_RAS (0x010, R/W).....	147
29.2.2 TIMINGS_RFC_REFI (0x014, R/W).....	147
29.2.3 TIMINGS_RRD_FAW_WTR_CCD (0x018, R/W).....	147
29.2.4 TIMINGS_CL_CWL_WR (0x01C, R/W).....	148
29.3 Mode Register Values (0x020 – 0x02F).....	148
29.3.1 MR0, MR1, MR2, MR3 (0x020, 0x024, 0x028, 0x02C, R/W).....	148
29.4 LPDDR2-Specific (0x030 – 0x03F).....	148
29.4.1 PASR_BANK_MASK_RANK0 (0x030, R/W).....	148
29.4.2 PASR_SEG_MASK_RANK0 (0x034, R/W).....	149
29.4.3 TEMP_DERATE_RANK0 (0x038, R only).....	149

29.5 Scheduler / Refresh / Page / Addr / Init Tuning (0x040 – 0x05F).....	149
29.5.1 SCHED_TUNING (0x040, R/W).....	149
29.5.2 PAGE_PRED_TUNING (0x044, R/W).....	150
29.5.3 REFRESH_TUNING (0x048, R/W).....	150
29.5.4 ADDR_MAP (0x04C, R/W).....	150
29.5.5 INIT_TUNING (0x050, R/W).....	151
29.5.6 INIT_TIMING0 (0x058, R/W).....	151
29.5.7 INIT_TIMING1 (0x05C, R/W).....	151
29.5.8 TIMINGS_RTP_RTW (0x054, R/W).....	151
29.6 DFI / PHY Timing (0x060 – 0x067).....	152
29.6.1 DFI_PHASE (0x060, R/W).....	152
29.6.2 PHY_TIMING (0x064, R/W).....	152
29.7 Per-Bank Observation (0x080 – 0x09F).....	152
29.7.1 OBS_ROW_HIT[0..7] (0x080 + N×4, R only).....	153
29.8 Calibration / Training (0x0A0 – 0x0FF).....	153
29.8.1 CAL_CTRL (0x0A0, R/W).....	153
29.8.2 CAL_ZQ_INTERVAL (0x0A4, R/W).....	153
29.8.3 CAL_ZQ_TIMING (0x0A8, R/W).....	154
29.8.4 CAL_TRAIN_TIMING (0x0AC, R/W).....	154
29.8.5 CAL_MRR32_DATA[0..3] (0x0B0 + N×4, R only).....	154
29.8.6 CAL_MRR40_DATA[0..3] (0x0E0 + N×4, R only).....	154
29.8.7 CAL_STATUS (0x0F0, R only).....	154
29.9 Retired observation registers (0x0C0 – 0x1E0).....	155

29.10 Module Identification (0xFF0 – 0xFFC).....	156
29.10.1 ID (0xFF0, R only, reset = 0xD2020001).....	156
29.10.2 BUILD (0xFF4, R only).....	156
29.11 Notes on This Revision.....	156
30 Verification Strategy.....	157
30.1 Layered Approach.....	157
30.1.1 Layer 1: Module-Level Unit Tests.....	157
30.1.2 Layer 2: Subsystem Tests.....	158
30.1.3 Layer 3: End-to-End with DFI BFM Slave.....	158
30.1.4 Layer 4: External-Reference Cross-Validation.....	158
30.2 Characterization Sweep.....	159
30.2.1 Sweep Matrix.....	159
30.2.2 Sweep Outputs.....	159
30.2.3 Out-of-Order Scheduler Trade-Offs.....	160
30.3 Verification Hooks.....	161
30.4 Coverage Targets.....	161
31 Open Issues and TODOs.....	163
31.1 Open Issues.....	163
31.1.1 1. AXI4 Narrow-Burst Handling Confirmation.....	163
31.1.2 2. AXI4 Read-Modify-Write Strobe Handling.....	163
31.1.3 3. CSR Address Space Allocation.....	163
31.1.4 4. Per-Bank Refresh Book-Keeping Cost.....	163
31.1.5 5. HAPPY Predictor Hash Function.....	163

31.1.6 6. Self-Refresh Exit Latency.....	163
31.1.7 7. Init Sequencer Retry on Error.....	164
31.1.8 8. CSR Write Atomicity.....	164
31.1.9 9. Burst Splitting at Row Boundaries.....	164
31.1.10 10. Multi-Rank.....	164
31.1.11 11. ECC.....	164
31.1.12 12. DFI Training Sub-Interface.....	164
31.1.13 13. AXI Quality of Service (QoS).....	164
31.1.14 14. Multi-Master AXI.....	165
31.1.15 15. Observation Counter Overflow.....	165
31.1.16 16. Retired-Architecture Modules Still on Disk (OLD/).....	165
31.1.17 17. Open-Page Read-Fetches-Zero Under Gapped Reads.....	165
31.2 TODOs Before v0.2.....	165
31.3 Feature Roadmap — planned advanced modes (ordered).....	166

List of Figures

No figures in this document.

List of Tables

Table 1: Table 3.0: pumice_training_layer responsibilities.....	29
Table 2: Table 3.1: Training-layer CSR summary.....	33

1 Pumice Has Index

Generated: 2026-10-08

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

2 FUB Breakdown

This section documents the **actual** Functional Unit Block decomposition as implemented in the rearchitected RTL: four integration layers under `pumice_core`, each built from leaf FUBs, driven by a PeakRDL CSR block in `pumice_top`.

2.1 Component Layout

Per the standard component layout:

```
pumice/
├── docs/                # this HAS, MAS, generated PDFs
├── dv/                  # cocotb tests, BFM glue, tbclasses
├── rtl/
│   ├── top/            # pumice_top, pumice_core,
├── pumice_top_geared
│   ├── macro/          # the four layer macros +
├── pumice_csr.rdl + generated/
│   ├── fub/            # leaf FUBs (this section)
│   ├── includes/       # shared `defines, package files
├── (pumice_pkg)
│   └── filelists/
│       ├── top/        # one .f per top
│       ├── macro/      # one .f per layer
│       └── fub/        # one .f per FUB
```

Each FUB has one top SystemVerilog file named `<fub>.sv` with module `<fub>`, a filelist under `filelists/fub/`, and a cocotb testbench under `dv/tests/`.

2.2 Layer Hierarchy

`pumice_top` instantiates the PeakRDL `pumice_csr` block (config by name) and `pumice_core`, which wires the four layers:

```
pumice_top                                (CSR block + core)
├── pumice_core
│   ├── pumice_axi4_layer                  ("host AXI4 -> CAM-
buffered requests")
│   │   ├── pumice_wr_intake              AXI4 slave wr + AW-meta
│   │   └── pumice_rd_intake              AXI4 slave rd + snarf
├── FIFO + wr-data FIFO
│   ├── pumice_wr_data_cam                WR CAM + wr-data SRAM
├── probe
│   ├── addr_mapper                      flat AXI addr -> (rank,
bank, row, col)
│   ├── pumice_wr_data_cam
│   └── pumice_rd_cmd_cam                 RD CAM + rd SRAM
```

(return-fill/drain)	
— pumice_scheduler_layer	("what command to issue
this cycle")	
— pumice_cmd_arbiter	single pick core (open-
page inline)	
— pumice_bank_timers (bank_timer)	per-(rank,bank) FSM-
free JEDEC "safe" timers	
— global_timers	tFAW / tRRD / tWTR /
tRTW / tCCD turnaround	
— refresh_ctrl	tREFI postponer,
REFab/REFpb dispatch	
— init_sequencer	DDR2 + LPDDR2 JEDEC MR
init	
— mode_register	MR shadow -> live
CL/CWL/BL/AL	
— pumice_training_layer	("LPDDR2 ZQ + DQ
calibration; inert for DDR2")	
— pumice_zq_ctrl	periodic ZQCS/ZQCL as
MRW(MR10) maintenance	
— pumice_lp_cal	one-shot MRR MR32/MR40
DQ calibration	
— pumice_dfi_layer	("single datapath CDC +
DFI v2.1 datapath")	
— pumice_dfi_cdc	the ONE datapath
crossing (async gaxi FIFOs)	
— pumice_dfi_cmd_path	unpack cmd, drive DFI
via dfi_cmd_formatter	
— (uses dfi_cmd_formatter + dfi_signal_pack)	
— pumice_dfi_wr_serializer	commit-drain WR CAM ->
dfi_wrdata + mask	
— pumice_dfi_rd_aligner	dfi_rddata capture ->
return-fill RD CAM	

Also present but not in the default top build: `page_predictor` and `powerdown_ctrl` (referenced / optional; verify against the filelists).

2.3 FUB Inventory

2.3.1 pumice_axi4_layer

2.3.1.1 pumice_wr_intake

- **Purpose:** AXI4 slave write engine. AW/W/B handshakes with an AW-meta FIFO and a wr-data FIFO; splits each host burst at DRAM-burst byte boundaries so each command maps to one DRAM burst.

- **Key params:** AXI_DATA_WIDTH, AXI_ID_WIDTH, AXI_ADDR_WIDTH, BL, NUM_ENTRIES.
- **Downstream:** addr_mapper, pumice_wr_data_cam.

2.3.1.2 *pumice_rd_intake*

- **Purpose:** AXI4 slave read engine; splits the read burst; probes the write-data CAM for a read-your-write snarf hit (unscheduled, same-id, same-BL) before committing the read command.
- **Key params:** AXI_DATA_WIDTH, AXI_ID_WIDTH, AXI_ADDR_WIDTH, BL, NUM_ENTRIES.

2.3.1.3 *addr_mapper*

- **Purpose:** Decode a flat AXI address into (rank, bank, row, col) using the single bank_lsb knob: the bank field slides within the byte-offset-stripped word address and the column auto-splits below (col_lo) and above (col_hi) it, with the row/rank positions invariant. An optional bank XOR-hash folds row bits (+ a seed) into the bank index. Combinational, single stage.
- **Key params:** AXI_ADDR_WIDTH, NUM_RANKS, NUM_BANKS, ROW_WIDTH, COL_WIDTH.
- **Runtime inputs:** bank_lsb_i, hash_en_i, hash_seed_i (ADDR_MAP.bank_lsb / hash_en / hash_seed).
- **Notable:** Mirrors the Python address-mapping class in the DV repo; the same decode drives RTL and BFM. There is no scheme selector.

2.3.1.4 *pumice_wr_data_cam*

- **Purpose:** Write-data CAM. Fills each write burst into an SRAM and records the command; provides scheduler lookup / oldest / commit ports; commit-drains the burst to the DFI write serializer; and is the snarf source for read forwarding. An r_fdone fill-complete flag gates schedulability and snarf. Streaming read engine is FIFO-fed / oldest-pick — no active-slot state latch.
- **Key params:** NUM_ENTRIES, N_SRAM_SLOTS, NUM_RANKS, NUM_BANKS, ROW_WIDTH, BL.

2.3.1.5 *pumice_rd_cmd_cam*

- **Purpose:** Read-command CAM / reorder buffer. Records read commands; return-fills DRAM read beats into an SRAM; oldest-first drain engine (gated

on data-ready) streams beats back to pumice_rd_intake. No active-slot latch.

- **Key params:** NUM_ENTRIES, N_SRAM_SLOTS, NUM_RANKS, NUM_BANKS, ROW_WIDTH, BL.

2.3.2 pumice_scheduler_layer

2.3.2.1 pumice_cmd_arbiter

- **Purpose:** The single command-pick core. Picks one abstract command per cycle from {ACT, RD/RDA, WR/WRA, PRE, REF, MRS, NOP} from the CAM lookups, gated by the per-(rank,bank) safe_* from pumice_bank_timers and the turnaround windows from global_timers. The open-page decision is inline.
- **Page policy:** OPEN (row-hit reuse + explicit PRE on miss), CLOSE (auto-pre on every column op via RDA/WRA), or an adaptive runtime mode (pumice_page_policy: fixed_open; the adaptive modes retired 2026-09-27). Programmed via REFRESH_TUNING.page_policy_or.
- **Key params:** NUM_ENTRIES, NUM_RANKS, NUM_BANKS, AGE_WIDTH.
- **Notable:** A combinational picker with registered feedback — always macro-test it, since registered-feedback latency created double-issue hazards caught only at the layer level.

2.3.2.2 pumice_bank_timers (instantiates bank_timer)

- **Purpose:** Stamps one bank_timer per (rank, bank) and fans the arbiter's command-event strobes to the addressed instance. bank_timer is FSM-free: preset/decrement countdown timers (tRCD, tRAS, tRC, tRP, precharge-block) + a row-open register + a single auto-precharge bit. The per-command safe_* outputs are combinational off the timers (one register stage), so the arbiter sees the just-issued command's effect one cycle later with no multi-stage lag.
- **Key params:** NUM_RANKS, NUM_BANKS, ROW_WIDTH.
- **Runtime inputs:** t_rcd_i, t_rp_i, t_ras_i, t_rc_i, t_wr_i, t_rtp_i.

2.3.2.3 global_timers

- **Purpose:** Cross-bank / cross-rank turnaround windows: tFAW (4-ACT window), tRRD, tWTR, tRTW, tCCD. ANDed by the arbiter.
- **Runtime inputs:** t_faw_i, t_rrd_i, t_wtr_i, t_rtw_i, t_ccd_i.

2.3.2.4 *refresh_ctrl*

- **Purpose:** tREFI down-counter; postponed-refresh accumulator (JEDEC max 8); refresh-pending signal to the arbiter; REFab / REFpb dispatch (REFRESH_TUNING.refpb_policy_or).
- **Runtime inputs:** t_refi_i, refresh_burst_i.

2.3.2.5 *init_sequencer*

- **Purpose:** Cold-boot engine that runs the memtype-specific JEDEC MR/init sequence (DDR2 and LPDDR2), driving CKE / RESET_N and MR-write strobes into mode_register, and holds off traffic until init completes.
- **Runtime inputs:** t_init_wait_i, t_dll_wait_i, t_mrd_wait_i, t_rp_wait_i, t_rfc_wait_i, memtype_i.

2.3.2.6 *mode_register*

- **Purpose:** Per-rank Mode Register shadow + live decode to CL / CWL / BL / AL (memtype-dependent), plus drive-strength and ODT-rule outputs consumed by the DFI layer.
- **Key params:** NUM_RANKS, MAX_MR_IDX (17; covers DDR2 MR0..3 and LPDDR2 MR0..16).

2.3.3 *pumice_training_layer*

2.3.3.1 *pumice_zq_ctrl*

- **Purpose:** Periodic ZQ calibration for LPDDR2 as maintenance traffic. LPDDR2 has no ZQCS/ZQCL command — both are MRW to MR10 (OP 0x56/0xAB) — so the FUB only raises requests and holds the post-grant bus-quiet window (t_zqcs/t_zqcl); the arbiter emits the MRW verbatim. Interval countdown, Mode-C defer-under-demand with overdue escalation to ZQCL, zqcs_total / zq_overdue telemetry.
- **Run gating:** zq_en && init_done && memtype == MEMTYPE_LPDDR2 && interval != 0. Inert for DDR2 (no ZQ pin).
- **Runtime inputs:** zq_interval_i, t_zqcs_i, t_zqcl_i, zq_defer_en_i, zq_overdue_max_i (the CAL_* CSRs).

2.3.3.2 *pumice_lp_cal*

- **Purpose:** One-shot DQ-calibration sequencer. On cal_start: wait for the maintenance grant, issue MRR to MR32 (pattern A), capture the first dfi_rddata_valid beat via the read aligner's cal sideband, wait t_mrr,

repeat for MR40 (pattern B), set sticky cal_done. Sticky cal_err on t_readout timeout; cal_abort cancels. The DQ-vs-tap sweep itself lives in firmware — this is the interface, not the search.

- **Run gating:** init_done && memtype == MEMTYPE_LPDDR2. Inert for DDR2 (no MRR).
- **Runtime inputs:** t_mrr_i, t_readout_i (CAL_TRAIN_TIMING).
- **Notable:** the layer ties the ZQ FUB's demand_i to 1'b0 pending a scheduler demand sideband, so Mode-C deferral is implemented and FUB-tested but dormant in the current integration.

2.3.3.3 pumice_training_layer (the macro itself)

- **Purpose:** Holds the two FUBs, arbitrates them one-active onto the single trn_cmd_* maintenance channel into the scheduler's arbiter (ZQ > lp_cal), and owns the mc_clk ↔ dfi_clk CDC for the calibration sideband (sync_pulse / cdc_synchronizer).
- **See:** Ch 3.9 for the full behavioral description, the MRR command path, and the arbitration policy.

2.3.4 pumice_dfi_layer

2.3.4.1 pumice_dfi_cdc

- **Purpose:** The single controller-to-PHY clock crossing, built from asynchronous gaxi FIFOs (command, write-data, read-data). One FIFO word is one DFI cycle, so the datapaths are bubble-free.
- **Key params:** CMD_FIFO_DEPTH, WD_FIFO_DEPTH, RD_FIFO_DEPTH, N_FLOP_CROSS.

2.3.4.2 pumice_dfi_cmd_path (uses dfi_cmd_formatter + dfi_signal_pack)

- **Purpose:** DFI-domain command path. Pops the abstract command stream from the CDC FIFO, unpacks {op, rank, bank, row, col, ap}, and drives the multi-phase DFI command bus via dfi_cmd_formatter; emits wr_fire / rd_fire strobes so the serializer / aligner can schedule their data phases.
- **dfi_cmd_formatter:** JEDEC command encoding into DFI wires — DDR2 drives cs_n / ras_n / cas_n / we_n / address / bank (including the A10 auto-precharge bit and ODT rule); LPDDR2 packs the 10-bit CA-bus command (two edges) onto dfi_address, per a memtype branch. Swap this when moving DFI generations.

- **dfi_signal_pack**: multi-phase aggregation — widens each DFI control bus to per-phase $\times$ DFI_RATE.

2.3.4.3 pumice_dfi_wr_serializer

- **Purpose**: On wr_fire, commit-drains the write burst from pumice_wr_data_cam's SRAM and drives dfi_wrdata / dfi_wrdata_en / dfi_wrdata_mask with PHY alignment.
- **AXI  DFI mask polarity**: AXI wstrb=1 means write; DFI mask=1 means do-not-write $\rightarrow$ dfi_wrdata_mask = $\sim$ wstrb.

2.3.4.4 pumice_dfi_rd_aligner

- **Purpose**: Drives dfi_rddata_en t_rddata_en cycles after a READ command; captures dfi_rddata beats; return-fills them into pumice_rd_cmd_cam. Also hosts the calibration capture sideband (cal_expect_i arm, first-valid beat captured to cal_data_o, cal_valid_o pulse) used by the training layer's MRR DQ calibration.
- **Runtime inputs**: t_rddata_en_i, t_phy_wrlat_i, rd_phase_i, wr_phase_i.

2.4 Data Width and Gearing

Two distinct concepts:

1. **Internal DFI geometry**. $DW = DRAM_BEAT_WIDTH * DFI_RATE$ (128 default). One AXI beat is one DFI word is DFI_RATE DRAM beats; one AXI burst is one DRAM burst. The $DW \rightarrow$ per-phase split happens in dfi_signal_pack / the DFI layer. The host AXI data width equals DW at pumice_top.
2. **Host-width freedom**. The optional pumice_top_geared wrapper adds a free HOST_AXI_DATA_WIDTH and inserts the repository's formally-verified axi4_dwidth_converter_wr / _rd between a host-width slave and the fixed-DW core. $HOST_AXI_DATA_WIDTH == DW$ is a bit-identical generate bypass. Verified host $\in \{64, 128, 256\}$.

2.5 What Changed vs the Earlier Architecture

The controller was rearchitected from an earlier FSM-based decomposition. The following blocks were retired; their behavior now lives in the FUBs shown:

Retired block	Replaced by
txn_queue	the two CAMs: pumice_wr_data_cam + pumice_rd_cmd_cam

Retired block	Replaced by
bank_machine	FSM-free bank_timer, stamped by pumice_bank_timers
xbank_timers	global_timers (turnaround) + pumice_bank_timers (per-bank)
cmd_encoder	dfi_cmd_formatter (+ dfi_signal_pack)
odt_ctrl	ODT inside dfi_cmd_formatter / mode_register (no standalone block)
page_predictor (standalone)	open-page decision inline in pumice_cmd_arbiter (page_predictor.sv optional)
wr_cmd_cam	pumice_wr_data_cam
scheduler	pumice_scheduler_layer (pumice_cmd_arbiter + timers + refresh + init)
gear_dfi	pumice_dfi_layer
axi_frontend / axi_intake / *_macro-as- architecture	pumice_axi4_layer, generated pumice_csr, pumice_core

The old names may still appear in retired sentinel tests; they are not part of the current architecture.

2.6 Integration

The four layer macros are structural wiring; behavioral logic lives in the leaf FUBs. The principal wiring concerns:

1. **Arbiter ↔ timing fan-out:** pumice_bank_timers exposes per-(rank,bank) safe_* off single-stage timers; global_timers exposes turnaround-OK signals. pumice_cmd_arbiter reduces these into one command per cycle.
2. **CAM ↔ DFI-datapath coupling:** pumice_wr_data_cam's commit-drain feeds pumice_dfi_wr_serializer; pumice_dfi_rd_aligner's return-fill advances pumice_rd_cmd_cam.
3. **The single datapath CDC:** everything up to pumice_dfi_cdc is on aclk; the command path, serializer, and aligner are on dfi_clk. The training layer's calibration sideband crosses the same boundary separately (cal_expect down, captured MRR data up) — narrow, FIFO-less, no datapath traffic.

3 Training: LPDDR2 ZQ and DQ Calibration

The training layer is `pumice_training_layer` (`rtl/macro/pumice_training_layer.sv`), the fourth layer of the core, instantiated in `pumice_core` between the scheduler layer and the DFI layer (“Layer 2b” in the RTL). It holds the two calibration FUBs — `pumice_zq_ctrl` (periodic ZQ calibration) and `pumice_lp_cal` (one-shot DQ calibration via mode-register reads) — muxes both onto a single maintenance command channel into the scheduler’s arbiter, and owns the `mc_clk`  `dfi_clk` clock crossing for the read-aligner calibration sideband.

Both FUBs run only when `init_done` is set and `memtype == MEMTYPE_LPDDR2`; DDR2 has no ZQ pin and no MRR mechanism, so the entire layer is inert for DDR2 builds. That is an elaboration-free inertness — same RTL, gates low — so the existing DDR2 test suites run unchanged beside it.

3.1 The family rule: interfaces in hardware, searches in firmware

Scoria settled this for DDR3 write leveling (decision D2), andesite carried it to read leveling and LPDDR4 CA training, and pumice makes the same decision for the same reasons: the controller provides the interface — the MRR engine, the timing windows, the capture sideband, the telemetry — and the search runs in firmware. A DQ-vs-read-delay-tap sweep’s step size, tap count and temperature behaviour are PHY properties, not controller properties; a calibration bug should be a script edit, not a respin. Concretely, the DQ-cal sweep (walk the board’s FPGA IDELAY taps, capture, compare) lives in board firmware driving the PHY primitives directly. The core exposes `cal_start` and the captured MRR data; it deliberately contains **no** PHY-primitive logic and gained **no** new top-level ports for training.

LPDDR2’s JEDEC menu is also smaller than DDR3/LPDDR4’s, and the layer does not invent the missing entries. Write leveling, read leveling, DLL calibration and CA training do not exist in LPDDR2 — verified against the JESD209-2F mode-register map (MR41–47 are “Do Not Use”, MR48–62 are reserved; CA training is an LPDDR4 MPC mechanism owned by andesite’s training layer). What LPDDR2 actually offers is two mechanisms, both implemented here: ZQ calibration through MR10 writes, and DQ calibration through MRR reads of MR32/MR40.

3.2 `pumice_training_layer`: arbitration and CDC

The layer is a passive wiring and arbitration shell; run gating lives inside the two FUBs.

Table 3.0: `pumice_training_layer` responsibilities

Responsibility	Detail
FUB holder	<code>u_zq</code> (<code>pumice_zq_ctrl</code>) + <code>u_lp_cal</code> (<code>pumice_lp_cal</code>)
One-active command mux	Both FUBs share one <code>trn_cmd_*</code> channel; priority ZQ > <code>lp_cal</code> , because ZQ is periodic maintenance while

Responsibility	Detail
Calibration sideband CDC	lp_cal is a firmware-initiated one-shot. trn_cmd_mrr_o is 0 for ZQ (MRW) and 1 for lp_cal (MRR) cal_expect crosses mc_clk → dfi_clk as a sync_pulse (one mc_clk assertion produces exactly one dfi_clk pulse); the captured beat crosses back through a cdc_synchronizer (data held stable by the aligner until the next arm) with a sync_pulse for the valid strobe
Status aggregation	cal_busy_o = zq_busy lp_busy; cal_done / cal_err come from lp_cal; zq_busy / zq_overdue / zqcs_total from ZQ

pumice_core wires the layer exactly as andesite wires its training layer: the trn_cmd_* channel nets into the scheduler layer's new maintenance client ports, and cal_expect_o / cal_data_i / cal_valid_i net into the DFI layer's read aligner. pumice_top connects the CSR side (CAL_CTRL fields, timing registers) and harvests status back into CAL_STATUS and the CAL_MRR32/40_DATA slices.

3.3 pumice_zq_ctrl: periodic ZQ calibration as maintenance traffic

LPDDR2 has no dedicated ZQCS/ZQCL command (unlike DDR3); both are **MRW writes to MR10**, OP = 0x56 (short, ZQCS) or 0xAB (long, ZQCL), per JESD209-2F §5.13.3. ZQ Init (0xFF, tZQINIT) already exists in init_sequencer state S_L_ZQ; this FUB owns everything after init: the periodic calibration that keeps the output impedance inside its ±15% window as temperature and voltage drift.

The FUB **never generates the command itself**. It raises a request and, on grant, the arbiter issues the FUB's payload verbatim — OP_MRS with the row packed as {MR10_index[5:0], OP[7:0]} in the row field (bank 0; the MR index rides the row field, the same packing init MRW uses). The four-state FSM:

1. **Interval countdown** — CAL_ZQ_INTERVAL MC cycles between calibrations; 0 disables the engine entirely. zq_en gates the whole FUB.

2. **Request** — on expiry, raise `zq_req` and hold it. While waiting ungranted under demand, the FUB marks itself `zq_overdue`.
3. **Mode-C deferral** (`zq_defer_en`) — at expiry, if the scheduler reports sustained demand, the request can be held in `ZQ_DEFER` instead, up to `zq_overdue_max` MC cycles (13-bit, 0 = no cap), after which the request fires regardless. A deferral that overruns its cap — or any nonzero deferral that exits on demand loss — escalates the request to **ZQCL**: after deferred calibration the long calibration re-establishes the $\pm 15\%$ RON budget that the short one only trims.
4. **Post-grant hold** — on grant the FUB loads `t_zqcs` or `t_zqcl` (`CAL_ZQ_TIMING`) and asserts `cal_busy` for that window: JEDEC forbids other data-bus activity during calibration, so the arbiter must keep demand quiet until the window expires. Only then does the interval reload.

Two integration facts worth recording. First, the FUB's `demand_i` port exists for the Mode-C policy but is **tied to 1'b0 in the committed layer** — the scheduler's demand sideband is not wired through the training layer yet, so in the current integration expiry requests immediately (ZQCS, never deferred), and `zq_overdue` can never assert — both the deferral entry condition and the request-state overdue set read `demand_i`. The deferral policy is implemented and FUB-tested; the hookup is the remaining wiring. Second, the MR10 writes are **transient commands, never shadowed**: the ZQ MRW flows through the arbiter's maintenance path straight to the formatter, and `mode_register`'s shadow is written only by `init_sequencer`, so a periodic ZQ calibration can never disturb the decoded CL/CWL/BL. Multi-rank ZQ (a shared ZQ resistor across ranks) is left as a documented generate hook; the board targets are single-rank, and the arbiter's pick is single-rank in v1 anyway.

ZQ bus placement follows the all-banks-idle grant contract rather than the JEDEC minimum: LPDDR2-S4 requires all banks precharged for ZQ calibration while LPDDR2-N permits Idle-or-Active; the conservative contract satisfies both.

3.4 `pumice_lp_cal`: one-shot DQ calibration over MRR

JESD209-2F §5.12.2 defines DQ calibration: MRR reads of MR32 (pattern A, 1010) and MR40 (pattern B, 0011) make the DRAM drive a known, training-only pattern onto DQ, so firmware can observe the read-capture timing without any memory array in the loop. `pumice_lp_cal` is the one-shot sequencer that issues those two reads and captures the answers; what firmware does with the answers — the tap sweep — is deliberately outside the core.

The eight-state FSM, started by a `cal_start` pulse (a self-clearing CSR write):

1. Wait for the maintenance grant (the arbiter's all-banks-idle contract plus the `w_out_safe` demand gate leave no read or write data in flight — which

also satisfies the JEDEC MRR spacings: $RD \rightarrow MRR \geq BL/2$, $WR \rightarrow MRR \geq WL+1+BL/2+t_{WTR}$).

2. Issue MRR to **MR32** — `OP_MRS` with the MRR flag set, row `{MR32_index[5:0], OP=8'h00}` — and pulse `cal_expect` for one cycle, arming the DFI-layer read aligner.
3. Capture the first `dfi_rddata_valid` beat (MRR data lands $RL \cdot t_{CK} + t_{DQSQ}$ + t_{DQSQ} after the command; $BL=4$, only beat 0 carries the MR data on `DQ[7:0]`, with x16 devices mirroring the pattern on `DQ[8]`). A `t_readout` timeout without a beat sets sticky `cal_err` and ends the run.
4. Wait `t_mrr` (JEDEC §5.12: 2 clocks, the CSR reset default), then repeat for **MR40**.
5. On the MR40 capture set sticky `cal_done`. Both captured beats are exposed to firmware through `CAL_MRR32_DATA` / `CAL_MRR40_DATA` (full DFI read width, four 32-bit slices each at the default 128-bit geometry).

`cal_abort` (or losing the run condition mid-sequence) returns the FSM to idle; abort additionally drops the captured-data valids. The sticky `cal_done` / `cal_err` survive both — they are cleared only by soft reset — so a host can always distinguish “never ran”, “ran and converged”, “ran and timed out”, and “ran, aborted” after the fact.

3.5 The MRR path: one new bit, not a new opcode

`dram_op_e` stays exactly as wide as it was — all 16 opcodes are consumed and MRR never enters the demand path. Instead MRR rides the existing `OP_MRS` as a new 1-bit flag, end to end:

- The arbiter passes the granted maintenance op **verbatim** (no re-encoding) and latches the requester’s `trn_cmd_mrr_i` into the command word.
- The scheduler’s output `cmd` FIFO word grows by one bit to `{mrr, ap, col, row, bank, rank, op}` (pack site `pumice_scheduler_layer.sv`, unpack site `pumice_dfi_cmd_path.sv`). Demand commands simply carry `mrr = 0`.
- In `dfi_cmd_formatter`’s `OP_MRS` branch, `cmd_mrr_i` drives **CA3r**: MRW and MRR share the same MA/OP field packing — `MA[7:0]` on `{CA1f, CA0f, CA9r..CA4r}` — and MRR is MRW with CA3r inverted, bit-exact to JESD209-2F §5.12 (the `OP` field is don’t-care-zero for MRR; the `lp_cal` row packs it 0). The `lpddr2_mrr` formatter test encodes the same table.
- No formatter change was needed for ZQ: LPDDR2 has no ZQCS/ZQCL opcodes, so those encodings remain NOP at the formatter’s default branch — LPDDR2 ZQ is pure MRW.

On the return side, `pumice_dfi_rd_aligner` gains a calibration capture sideband in the `dfi_clk` domain: a single-cycle `cal_expect_i` arms a capture register; the first `dfi_rddata_valid` beat is latched into `cal_data_o` (full DFI read width, held until the next arm) with a one-cycle `cal_valid_o` pulse, and the register disarms. The sideband cannot collide with the normal read FIFO push path because it only fires while armed, and the arbiter's grant contract guarantees no read is outstanding when calibration runs.

3.6 Arbitration policy

The scheduler's arbiter gains exactly one maintenance client (`trn_cmd_*`), in the family's standing shape — the same contract scoria and andesite use for ZQ and training. Priority, descending: **init** > **refresh** > **trn** > **demand** (column, then ACT, then PRE). The training branch fires only under `w_trn_safe`: every bank idle in the registered view, no row-affecting command in flight or inside its 2-cycle guard window, no tRFC recovery running, and no grant in flight. The granted op passes through verbatim; the grant is a single-cycle pulse on the fire. Refresh outranks training, so a pending refresh drains banks and wins the bus first; training outranks every demand class.

The post-grant hold is where BUG-003 discipline matters. Both FUBs drive `cal_busy` during their bus-quiet windows (ZQ hold, `lp_cal` until done), and the arbiter folds `cal_busy_i` into `w_out_safe` for **every demand class** — ACT, RD/WR and PRE all carry a `!cal_busy_i` term, while the training class itself is unconditional. The gate lives in the fire stage, never the pick cone: `cmd_valid_o = r_pick_valid && w_out_safe` and `w_fire_out = r_pick_valid && cmd_ready_i && w_out_safe`, so a pick the safety gate rejects is never pushed to the cmd FIFO and the fire is the push by construction. That is precisely the invariant BUG-003 restored for the bank timers; the training gate had to land there, not in the pick pipeline, or it would have reopened the same divergence.

3.7 CSR summary

The layer is controlled and observed through seven registers in the 0x0A0 block of `pumice_csr` (field definitions and resets: `rtl/macro/pumice_csr.rdl`):

Table 3.1: Training-layer CSR summary

Address	Register	Access	Contents
0x0A0	CAL_CTRL	rw	zq_en[0], zq_defer_en[1] (Mode-C), cal_start[4] (wo, self-clearing), cal_abort[5] (wo),

Address	Register	Access	Contents
			zq_overdue_max[28:16] (max deferral, 0 = no cap)
0x0A4	CAL_ZQ_INTERVAL	rw	ZQCS interval in MC cycles, 0 = disabled
0x0A8	CAL_ZQ_TIMING	rw	t_zqcs[15:0], t_zqcl[31:16] post-grant hold windows
0x0AC	CAL_TRAIN_TIMING	rw	t_mrr[15:0] (reset 2), t_readout[31:16] MRR issue→data timeout
0x0B0 +4×	CAL_MRR32_DATA[0..3]	ro	Captured MR32 beat 0, four 32-bit slices of the DFI read width
0x0E0 +4×	CAL_MRR40_DATA[0..3]	ro	Captured MR40 beat 0, four 32-bit slices
0x0F0	CAL_STATUS	ro	zq_busy[0], cal_busy[1], cal_done[2] (sticky), cal_err[3] (sticky), zq_overdue[4], zqcs_total[31:16]

The data arrays start at 0x0B0 and 0x0E0 because the retired 0x0C0–0x0DC hole is never reused (pumice doctrine) — 0x0E0 clears it. cal_done / cal_err are sticky and clear only on soft reset.

3.8 Verification and integration notes

- **FUB:** `test_pumice_zq_ctrl.py` (interval expiry, Mode-C deferral, overdue → ZQCL select, hold window, DDR2-inert) and `test_pumice_lp_cal.py` (one-shot sequence, MRR32 → capture → tMRR → MRR40 → capture, timeout error, abort).
- **FUB:** `test_dfi_cmd_formatter.py::lpddr2_mrr` checks the MRR CA encoding against JESD209-2F §5.12; `test_pumice_dfi_rd_aligner.py::test_pumice_dfi_rd_aligner_cal_capture` checks the sideband capture with zero reads outstanding and no interference with normal returns.
- **Macro:** `test_pumice_scheduler_layer.py` (training priority vs refresh / demand, all-banks-idle gating, demand blocked during `cal_busy`, the `fire==valid` invariant). The command-stream checker allows MRR and MRW-MR10 in its stream rules.
- **Top:** the CSR/top tests drive an end-to-end `cal_start` on the LPDDR2 model and observe both captured patterns; ZQCS MRW at interval with demand blocked.
- **TB tie-off:** `pumice_core_tb_top.sv` ties every training input to 0 (`zq_en`, `zq_defer_en`, `zq_interval`, `t_zqcs/t_zqcl`, `zq_overdue_max`, `cal_start`, `cal_abort`, `t_mrr`, `t_readout`) and leaves the status outputs open — that TB scores the datapath and paging sweep, not calibration. Enables at 0 make both FUBs inert by construction.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

4 Document Information

4.1 DDR2 / LPDDR2 Family Controller Hardware Architecture Specification

Document Number: DDR2-LPDDR2-HAS-001 **Version:** 0.9 **Status:** Draft - reconciled with rearchitected RTL **Classification:** Open Source - MIT License

4.2 Document Purpose

This Hardware Architecture Specification (HAS) provides a high-level architectural overview of the DDR2 / LPDDR2 unified memory controller family. It describes the system-level design, external interfaces, characterization parameters, and integration requirements without detailing internal implementation specifics.

Target Audience:

- System architects evaluating the controller for SoC integration
- Hardware engineers planning RTL implementation
- Verification engineers planning system-level testing
- Software engineers developing low-level memory configuration

Companion Documents:

- DDR2/LPDDR2 Family Controller Pre-Architecture Spec (`../pre-aspec.md`) — bullet-form architecture rationale
 - DDR2/LPDDR2 Paging and Refresh Notes (`../paging-refresh-notes.md`) — research-derived parameter sources
-

4.3 Revision History

Version	Date	Author	Notes
0.1	2026-06-13	RTL Design Sherpa	First sketch — module decomposition
0.2	2026-06-14	RTL Design	Top-level interface port list (§2.4), FUB SWAG (§2.5), family-wide config-bit registry (§5.4), multi-

Version	Date	Author	Notes
0.3	2026-07-07	Sherpa RTL Design Sherpa	rank support across §2 / §3.3 / §3.4 / §3.6 / §5.2 / §6.3 Narrow-device (x16) support: distinguish DRAM beat vs physical device word (DRAM_DEVICE_WIDTH); device-word column granularity + burst-length scaling; DFI_PHASE (rd/wr phase) CSR. Fixes on-silicon DDR2 read failure (Nexys A7 x16 bring-up).
0.4	2026-07-12	RTL Design Sherpa	Full reconciliation with the rearchitected RTL: three-layer core (pumice_axi4_layer / pumice_scheduler_layer / pumice_dfi_layer); FSM-free bank_timer; de-FSM'd CAMs; single ADDR_MAP.bank_lsb address knob (scheme selector retired); fully-functional LPDDR2 (bit-exact JESD209-2F CA + JEDEC MR init); host-width pumice_top_geared gearing; block/hierarchy diagrams regenerated.
0.5	2026-09-xx	RTL Design Sherpa	(No revision-history row was recorded when v0.5 was cut; its content is whatever the 2026-07-22 build captured. Noted rather than reconstructed.)
0.6	2026-09-26	RTL Design Sherpa	Paging modes 6/7 (rbl_static/rbl_dyn) RETIRED and the pumice_rbl_table FUB removed — measured on silicon on a workload built to suit them, the mechanism worked (thrash 100% -> 57.8%) and still lost: mode 6 -26% bandwidth, mode 7 bit-identical to plain open page. PAGE_RBL_CFG @ 0x07C left a documented HOLE, not reused. Newly documented: the bank timers' advisory lookahead (safe*_la_o), the arbiter's in-flight shadow and FINAL-STAGE timing authority, and the TASK-006 stall counters + REF_STATS_REF_BUSY, none of which appeared in any prior revision.
0.7	2026-09-26	RTL Design Sherpa	Page-policy characterization; the default change is RECOMMENDED but BLOCKED (BUG-003). Mode 3 (fixed_open) with tr_init = 2 measured strictly dominant over open page on the board at

Version	Date	Author	Notes
			txn_scale=1000: +41.2% col_major_interleaved_bl4, +8.6..11.6% col_major, exactly flat on incremental/row_major, no scenario regressing, zero integrity failures. The RDL resets were changed to mode 3 / TR=2 and REVERTED the same day: the component gate caught a read-return-ring assertion (“DFI return beat with NO ticket in flight”) at rd_gap >= 8, proven by controlled A/B to be caused by the new reset – a short timeout fires inside a read’s DFI return latency and precharges under an in-flight read (BUG-003, P1). The mechanism finding stands regardless: auto-precharge costs 4.9x the activations and 2x the read latency on identical traffic (160,006 vs 32,400 ACT) because it is uncancellable, whereas a background precharge gated on bank idle is implicitly cancellable. Mode 4 (adapt_time) documented as SUBSUMED by mode 3 – its TR decays to tr_min and it equals fixed_open(tr_min) exactly at three different floors – and policy_scope=0’s “per-bank TR” documented as a fiction (r_mc is one global counter). Mode 5 (adapt_access) documented as unproven and mis-plumbed. Two CSR traps now documented: tr_init = 0 DISABLES the timeout, and PAGE_ADAPT_CFG.check_interval = 0 gates the entire mode-4 adjustment off – which is why mode 4 measured inert on every campaign before this one.
0.8	2026-09-27	RTL Design Sherpa	BUG-003 FIXED and the adaptive paging modes REMOVED. BUG-003 root-caused in pumice_cmd_arbiter.sv: cmd_valid_o was r_pick_valid with no w_out_safe term, so a pick the arbiter’s own final safety gate had REJECTED was still pushed to the cmd FIFO and executed by

Version	Date	Author	Notes
			the DRAM – while <code>w_fire_out</code> (which does carry the term) withheld <code>evt_*</code>, so the bank timers, the guard chains and the CAM retire all recorded a command the DRAM had already run. That produced BOTH an illegal column op to a just-closed bank AND, because the CAM entry was never retired, a second issue of the same read – the “extra read” chased for weeks as a DV read-return modelling defect. One line; <code>w_fire_out == cmd_valid_o && cmd_ready_i</code> now holds by construction. Measured at the scheduler layer on the board operating point with the fix reverted: 470 pushes, 10 with <code>w_out_safe</code> low, 9 illegal column ops, plus a leaked PRE in <code>static_close</code>; with the fix, 0 and 0. Paging modes 4 (<code>adapt_time</code>) and 5 (<code>adapt_access</code>) RETIRED with <code>pumice_row_pred_table.sv</code> and the <code>PAGE_ADAPT_CFG</code> register (TASK-014): mode 4 was measured to BE <code>fixed_open(tr_min)</code>, mode 5 drove auto-precharge at 4.9x the activations and was removed as a decision rather than a measurement. <code>policy_scope</code>, <code>tr_min/tr_max/tr_step</code> and the hybrid counter fields are RESERVED at their original bit positions; 0x078 is left a hole, and the generated register maps were diffed to prove it – 81 -> 80 registers, ZERO moved. Modes 4..7 fall through to the build default, a contract now regressed by <code>test_page_predictor</code> and the scheduler matrix. New verification: 12 named DRAM operating points shared by the scheduler and top TBs; a JEDEC command-stream checker with armed-rule counts; a 12-config x 7-mode x 8-pattern scheduler matrix (672 arms, 1.38M requests, 3.15M commands checked); and the FIRST check of the DDR2 init

Version	Date	Author	Notes
0.9	2026-10-08	RTL Design Sherpa	sequence's JEDEC legality at any level (TASK-016, P1) – command order plus a differential proof that the sequencer honours its own wait registers. LPDDR2 training layer landed and documented (Ch 3.9). The core gains its fourth layer, <code>pumice_training_layer</code> (<code>rtl/macro/pumice_training_layer.sv</code>), instantiated in <code>pumice_core</code> between the scheduler and DFI layers: <code>pumice_zq_ctrl</code> raises periodic ZQ calibration as maintenance traffic (LPDDR2 has no ZQCS/ZQCL command – both are MRW to MR10, OP = 0x56 / 0xAB, with Mode-C defer-under-demand and overdue-escalation-to-ZQCL in the FUB), and <code>pumice_lp_cal</code> runs the one-shot DQ-calibration sequence (digital drift compensation: MRR to MR32 pattern A, capture, tMRR, MRR to MR40 pattern B, capture, with readout timeout and sticky done/err). MRR rides the existing OP_MRS opcode as a new 1-bit <code>cmd_mrr</code> flag – <code>dram_op_e</code> is not widened – through a +1-bit scheduler cmd FIFO word {<code>mrr</code>, <code>ap</code>, <code>col</code>, <code>row</code>, <code>bank</code>, <code>rank</code>, <code>op</code>} to the formatter's MRR CA word (JESD209-2F §5.12, CA3r=H), and the DFI read aligner gains a cal-capture sideband (<code>cal_expect/cal_data/cal_valid</code>). The arbiter gains one maintenance client on a <code>trn_cmd_*</code> channel at priority <code>init > refresh > trn > demand</code>, granted only with all banks idle, nothing row-affecting in flight, and tRFC elapsed; the post-grant hold (<code>cal_busy</code>) is folded into <code>w_out_safe</code>, never the pick cone, per the BUG-003 <code>fire==valid</code> discipline (<code>w_fire_out == cmd_valid_o && cmd_ready_i</code> by construction). New CSRs at 0x0A0-0x0F0 (<code>CAL_CTRL</code>, <code>CAL_ZQ_INTERVAL</code>, <code>CAL_ZQ_TIMING</code>, <code>CAL_TRAIN_TIMING</code>, <code>CAL_MRR32_DATA</code>, <code>CAL_MRR40_DATA</code>, <code>CAL_STATUS</code>); DDR2 builds inert

Versi on	Date	Author	Notes
			(no ZQ pin, no MRR). Both FUBs plus the formatter/aligner/scheduler paths are FUB- and macro-tested; pumice_core_tb_top ties the training inputs off (datapath/paging TB, not calibration).

4.4 Scope of This Document

This HAS describes a single parameterized memory controller covering DDR2 and LPDDR2. It defines:

- External interfaces (AXI4 slave, DFI v2.1 master, APB CSR slave)
- Module-level decomposition and behavioral specification
- Build-time and run-time configuration parameters
- Initialization, power-state, and refresh policies
- Verification strategy and characterization plan

It does not define:

- Bit-level pin assignments (the SystemVerilog port list in rtl/top/pumice_top.sv is the canonical wire-level source)
- Detailed timing diagrams (deferred to v0.3)
- Verilog package skeletons or RTL stubs
- Floorplan or layout guidance

4.5 Out of Scope

The following features are explicitly out of scope for this version of the controller and will be addressed in higher-generation family controllers or future revisions:

- Inline ECC
- DFI training, frequency-change, and low-power sub-interfaces
- AXI exclusive access semantics
- Bank groups (not present in DDR2 / LPDDR2)
- Command/Address parity, CRC, Data Bus Inversion (not present in DDR2 / LPDDR2)

5 Purpose and Scope

5.1 Document Purpose

This Hardware Architecture Specification (HAS) defines the high-level architecture of the DDR2 / LPDDR2 Family Memory Controller. It serves as the primary reference for:

- **System Integration** — understanding the controller’s external interfaces and system-level requirements
- **RTL Implementation Planning** — module decomposition, interface contracts, and parameter sweeps
- **Verification Planning** — system-level test scenarios and the characterization sweep matrix
- **Driver Development** — programming model and APB register interface

This document complements the upcoming Micro-Architecture Specification (MAS), which will provide detailed block-level implementation specifics, signal-level pinouts, and timing diagrams.

5.2 Scope

This HAS covers a single unified controller that supports both DDR2 and LPDDR2 memory. The memory type is a runtime CSR selection (`PHY_TIMING.memtype`: 0 = DDR2, 1 = LPDDR2). The controller is built as three layers under `pumice_core`:

- **pumice_axi4_layer** — AXI4 slave host interface: dumb write/read intakes, an address mapper, a write-data CAM (write buffer + read-your-write snarf source), and a read-command CAM (read reorder buffer).
- **pumice_scheduler_layer** — the command-scheduling layer: a single command arbiter (`pumice_cmd_arbiter`, open-page decision inline), per-(rank,bank) FSM-free JEDEC “safe” timers (`pumice_bank_timers / bank_timer`), global turnaround timers (`tFAW / tRRD / tWTR / tRTW / tCCD`), a refresh controller, an init sequencer (DDR2 and LPDDR2 JEDEC MR init), and a mode-register shadow (CL / CWL / BL / AL decode).
- **pumice_dfi_layer** — a single controller-to-PHY clock crossing (`pumice_dfi_cdc`, async FIFOs) plus the DFI-domain command path (`dfi_cmd_formatter / dfi_signal_pack`), write serializer, and read aligner, presenting the DFI v2.1 pin bus.

Configuration (timings, phases, page policy, address map, memtype) is delivered by name from a PeakRDL-generated CSR register block (`pumice_csr`) instantiated in `pumice_top`. An optional outer wrapper, `pumice_top_geared`, gives the host a free AXI data width by inserting the repository's formally-verified `axi4_dwidth_converter_wr / _rd` between a host-width slave and the fixed-width core.

The differences between DDR2 and LPDDR2 are the command encoding (DDR2 `ras/cas/we` vs the LPDDR2 10-bit CA bus, both in `dfi_cmd_formatter`), the init MR sequence in `init_sequencer`, and a small number of mode-register decode differences. Both memtypes pass the full simulation suite; everything else is shared.

5.3 Design Philosophy

This controller is intended as a **characterization-first** design. Algorithmic choices that have meaningful research-backed alternatives — page policy, refresh policy, refresh deferral depth, address mapping — are exposed as CSR fields rather than hardcoded. The verification plan includes a benchmark sweep over representative AXI traffic patterns to pick defaults from data, not assumption.

The principal characterization knobs, all programmed by name through the CSR block, are:

- **Page policy** — static OPEN / CLOSE via `REFRESH_TUNING.page_policy_or`, plus the runtime page-policy engine (`pumice_page_policy`, selected by `PAGE_POLICY_CFG.policy_mode`): `fixed_open` `idle-timeout` `close`. The adaptive modes are retired: `adapt_time` (the Ghasempour-2015 adapting-timeout scheme) and `adapt_access` (a per-row 2-bit close predictor) were removed 2026-09-27 — `adapt_time` measured as `fixed_open(tr_min)` and `adapt_access` drove auto-precharge at 4.9x the activations (TASK-014). The earlier HAPPY_HYBRID address-hash predictor had already been retired; it was never wired into the rearchitected core. The open-page decision itself is inline in `pumice_cmd_arbiter`.
- **Refresh policy** (`REFRESH_TUNING.refpb_policy_or`: `ROUND_ROBIN` / `OLDEST_FIRST` / `DARP`) — where DARP is the dynamic access refresh parallelization scheme from Chang et al. (HPCA 2014).
- **Address map** (`ADDR_MAP.bank_lsb`, `hash_en`, `hash_seed`) — a single knob that slides the bank field within the word address, subsuming the classic `ROW_MAJOR` / `BANK_INTERLEAVE` schemes, with an optional bank XOR-hash.

These are described in detail in Chapter 5.

5.4 Audience

This document assumes the reader is familiar with:

- AXI4 protocol semantics (handshakes, burst types, ID-based ordering)
- DFI v2.1 specification (control / write-data / read-data sub-interfaces)
- JEDEC DDR2 (JESD79-2F) and LPDDR2 (JESD209-2F) command tables
- Basic memory-controller concepts (open vs closed page policy, refresh deadlines, bank state machines)

Readers new to these topics should first consult the companion `paging-refresh-notes.md` in the family working area for an entry-point bibliography.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

6 Document Conventions

6.1 Typographic Conventions

Style	Meaning
monospace	Module names, parameter names, signal names, register names
bold	Defined terms on first introduction; important warnings
<i>italic</i>	Emphasis; placeholder values to be replaced
NUM_RANKS	Top-level build-time parameter (all caps)
bank_timer	RTL module name (lowercase with underscores)
tRC, tRCD	JEDEC timing parameters (as in JESD79-2 / JESD209-2)

6.2 Numbering Conventions

- **Decimal** numbers have no prefix: 16, 256.
- **Hexadecimal** numbers are prefixed with 0x: 0x800, 0xFF.
- **Binary** literals use Verilog notation when needed: 8'b1011_0010.
- Bit ranges use the convention [MSB:LSB], e.g., dfi_address[19:0].

6.3 Signal Direction Conventions

For DFI signals, direction is from the **memory controller's** perspective:

- **MC → PHY** — driven by the controller
- **PHY → MC** — driven by the PHY (sampled by controller)

For AXI signals, direction follows the standard AXI4 convention with the controller as the **slave**.

6.4 Module Diagram Conventions

ASCII block diagrams in this document use the following conventions:

All diagrams in this document are rendered from Mermaid source files in assets/mermaid/. The Mermaid source files (.mmd) are linked alongside each rendered image so the diagrams can be edited and re-rendered.

- **Rectangular boxes with sharp corners** denote RTL modules.
- **Subgraph rectangles** group modules by functional layer.

- **Solid arrows** show primary data or control flow.
- **Dashed arrows** show configuration or observation paths (typically from the CSR slave to other modules).
- **Trapezoidal nodes** denote external interface ports (AXI, DFI, APB).

6.5 Cross-Reference Conventions

- References to other chapters: “see Chapter 4”
- References to specific sections: “see §4.1” or “see Section 4.1”
- References to other documents: “see `pre-aspec.md`” (relative path)
- References to external standards: “per JESD79-2F §x.y”

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

7 Definitions and Acronyms

7.1 Acronyms

Acronym	Expansion
AHB	Advanced High-performance Bus (ARM AMBA)
APB	Advanced Peripheral Bus (ARM AMBA)
AP	Auto-Precharge (DRAM column command modifier)
APD	Active Power Down
AXI	Advanced eXtensible Interface (ARM AMBA)
AxLEN	AXI burst length — the AWLEN / ARLEN signals (beats = AxLEN+1). NOT the DRAM BL: one AXI burst at core width is (AxLEN+1)/DFI_RATE DFI words and splits into fixed-BL DRAM commands. Use “AxLEN” whenever burst length refers to the AXI side, to avoid confusion with the JEDEC BL below.
BL	Burst Length — JEDEC/DRAM burst (MR0 device beats: BL4/BL8). Distinct from AxLEN.
CA	Command/Address (LPDDR2/3/4 multiplexed bus)
CDC	Clock Domain Crossing
CL	CAS Latency
CMD	Command
CSR	Control/Status Register
CWL	CAS Write Latency
DARP	Dynamic Access Refresh Parallelization
DBI	Data Bus Inversion
DDR	Double Data Rate
DDR2	Double Data Rate 2 (JESD79-2)
DFI	DDR PHY Interface
DPD	Deep Power Down (LPDDR2)
FSM	Finite State Machine
FR-FCFS	First-Ready, First-Come-First-Served
HAPPY	Hybrid Address-based Page Policy in DRAMs (retired predictor; see

Acronym	Expansion
	pumice_page_policy)
HAS	Hardware Architecture Specification
LPDDR2	Low Power DDR 2 (JESD209-2)
MAS	Micro-Architecture Specification
MC	Memory Controller
MR / MRS	Mode Register / Mode Register Set command
EMR / EMRS	Extended Mode Register / Extended MRS
NOP	No Operation
ODT	On-Die Termination
OOO	Out-of-Order
PASR	Partial Array Self-Refresh (LPDDR2 feature)
PHY	Physical Layer (DRAM electrical interface)
PRE	Precharge command
PREA	Precharge All Banks
RD	Read column command
RDA	Read with Auto-Precharge
REF	Refresh command
REFab	All-Bank Refresh (LPDDR2)
REFpb	Per-Bank Refresh (LPDDR2)
SDRAM	Synchronous Dynamic RAM
SoC	System on Chip
SR	Self-Refresh
TCSR	Temperature-Compensated Self-Refresh (LPDDR2)
WR	Write column command
WRA	Write with Auto-Precharge
ZQ	Impedance Calibration (ZQ Calibration)

7.2 Defined Terms

Bank Timer — A per-bank JEDEC “safe” timing tracker (`bank_timer`). It is *not* a finite state machine: it is a set of preset/decrement countdown timers (`tRCD`, `tRAS`, `tRC`, `tRP`, precharge-block) plus a small open-row register and a single auto-precharge bit. The per-command `safe_*` outputs are combinational off the timers (one register stage). `pumice_bank_timers` stamps one instance per (rank, bank).

Characterization Knob — A build-time or runtime CSR field intentionally exposed for performance sweeping during system characterization. Page policy, refresh policy, and address map are the principal knobs in this controller.

Command Formatter — `dfi_cmd_formatter`: the module that translates the controller’s abstract command record into DFI wire signals. A single module with a memtype branch — DDR2 drives `ras/cas/we`; LPDDR2 packs the 10-bit CA bus (two edges) onto `dfi_address`. `dfi_signal_pack` performs the final per-phase wire packing.

Command Arbiter — `pumice_cmd_arbiter`: the single command-pick core. It scans CAM readiness against the bank and global timers, applies the page policy (the open-page decision is inline here), and emits one abstract DRAM command per cycle.

FR-FCFS — First-Ready, First-Come-First-Served. The baseline DRAM scheduling policy: ready commands beat unready, row-hits beat row-misses, older entries beat younger on ties. Established by Rixner et al. (ISCA 2000).

Gearing (two senses) — (1) The internal DFI-rate split: one AXI/DFI word is spread across DFI_RATE DRAM beats and packed to per-phase buses in `dfi_signal_pack` / the DFI layer. (2) Host-width gearing: `pumice_top_geared` inserts formally-verified AXI data-width converters so the host AXI data width can differ from the core width.

Init Sequencer — `init_sequencer`: the cold-boot engine that issues the memtype-specific JEDEC MR/init sequence, driving MR loads and commands and holding off traffic until init completes.

Page Hit / Row Hit — An access whose target row matches the currently open row in the target bank. Avoids the `tRP` + `tRCD` penalty.

Page Conflict / Row Miss — An access whose target row differs from the currently open row in the target bank. Pays `tRP` + `tRCD` before the access can begin.

Page Policy — The strategy for when to close an open row. Static OPEN / CLOSE via `REFRESH_TUNING.page_policy_or`; the runtime idle-timeout mode (`fixed_open`) via `PAGE_POLICY_CFG.policy_mode` in `pumice_page_policy`. The per-command decision is inline in `pumice_cmd_arbiter`.

Command CAMs — The two content-addressable buffers that hold in-flight requests between the AXI interface and the scheduler: `pumice_wr_data_cam` (write data buffer + snarf source) and `pumice_rd_cmd_cam` (read reorder buffer). Both store burst data in an SRAM with FIFO-fed / oldest-pick streaming read engines and no active-slot state latch.

8 Scope and Goals

8.1 High-Level Goals

This controller provides a single parameterized RTL implementation supporting both DDR2 and LPDDR2 memory devices via DFI v2.1. Primary design goals:

1. **Unified engine, one command formatter with a memtype branch.** DDR2 and LPDDR2 differ only at the wire-encoding layer (ras/cas/we vs the 10-bit CA bus, both in `dfi_cmd_formatter`) and in the init MR sequence; the command arbiter, bank/global timers, refresh policy, init sequencer, address mapper, and AXI interface are shared between both targets. Memtype is a runtime CSR selection (`PHY_TIMING.memtype`).
2. **Embedded / mobile / mid-range SoC targets.** Not optimized for high-end server DRAM (DDR5, RDIMM, multi-rank). Optimized for power efficiency and area economy.
3. **Characterization-first parameterization.** Algorithmic choices with research-backed alternatives are exposed as build-time parameters so the design can be swept on real workloads before defaults are locked.
4. **Verification-friendly.** Mirrors the data structures and conventions of the DDR2 / LPDDR2 DFI BFM in the DV repository so the same address mapping, timing parameters, and command encodings can drive both RTL and simulation.

8.2 In Scope

The following features are explicitly within scope for this controller:

- **AXI4 slave** at the fixed core data width $DW = \text{DRAM_BEAT_WIDTH} * \text{DFI_RATE}$; an optional pumice_top_geared wrapper gives the host a free AXI data width (verified host $\in \{64, 128, 256\}$) via formally-verified data-width converters
- **DFI v2.1 master** driving the command, write-data, and read-data sub-interfaces
- **CSR register block** (PeakRDL-generated `pumice_csr`, passthrough `cpuif`) for configuration and observation, programmed by name

- **DDR2 and LPDDR2 device support** selected at runtime by the `PHY_TIMING.memtype` CSR field
 - **DFI frequency ratio** of 1, 2, or 4 phases via the `DFI_RATE` parameter
 - **Multi-rank operation** — 1, 2, or 4 ranks via `NUM_RANKS` parameter; per-rank `CS_n`, `CKE`, `ODT`, and refresh tracking; rank-aware `ODT` cross-termination rules per JEDEC
 - **Page policies:** `OPEN`, `CLOSE`, plus the runtime page-policy engine (fixed `_open` idle-timeout; the adaptive modes retired 2026-09-27, TASK-014; CSR-selected via `PAGE_POLICY_CFG.policy_mode`; open-page decision inline in `pumice_cmd_arbiter`)
 - **Per-bank refresh** for LPDDR2 with DARP-style scheduling (parameterized)
 - **All-bank refresh** for DDR2 (and as LPDDR2 fallback)
 - **Elastic refresh deferral** up to JEDEC ceiling of $8 \times t_{REFI}$ (parameterized)
 - **Partial-array self-refresh (PASR)** for LPDDR2 with CSR-configurable mask
 - **Temperature-compensated refresh** (LPDDR2 MR4 derating)
 - **Cold-boot initialization** via microprogram step tables
 - **Power state management** (Active, Active Power-Down, Self-Refresh, Deep Power Down)
 - **Bring-up observability** via CSR-exposed counters (row-hit rate, refresh latency, queue depth)
-

8.3 Out of Scope

Explicitly **not** addressed in this controller version:

Feature	Reason
Inline ECC	Out-of-controller responsibility (SoC sideband)
DFI training sub-interface	Not required for v2.1 DDR2 / LPDDR2 operation
DFI frequency-change protocol	No use case for this generation
DFI low-power sub-interface	Power-state FSM uses <code>CKE</code> only
AXI exclusive accesses	Embedded targets don't require

Feature	Reason
Bank groups	Not present in DDR2 / LPDDR2 (introduced in DDR4 / LPDDR4)
Command/Address parity	Not present in DDR2 / LPDDR2 (introduced in DDR4 / LPDDR4)
Write CRC	Not present in DDR2 / LPDDR2
Data Bus Inversion (DBI)	Not present in DDR2 / LPDDR2
DDR3 write leveling	Not present in DDR2 / LPDDR2 (LPDDR3 introduces)
DDR3 ZQCS / ZQCL on column	DDR2 uses OCD calibration; LPDDR2 uses MR10 ZQ
Higher DDR / LPDDR generations	Will be addressed in DDR3-LPDDR3 and DDR4-LPDDR4 controllers

8.4 Target Performance Envelope

Metric	Target
DRAM data rate	Up to DDR2-800 / LPDDR2-1066
AXI clock	Up to 200 MHz (typical embedded SoC)
Sustained read bandwidth	$\geq 70\%$ peak on stream workloads
99th-percentile read latency	≤ 200 ns (typical)
Refresh blocking overhead	$< 5\%$ (LPDDR2 with DARP); $< 8\%$ (DDR2)
Area	$< 50K$ gates

These are target envelopes for v1; not guarantees. They will be validated and refined during the characterization sweep.

8.5 Differentiating Features

This controller is informed by published memory-controller literature and the open-source memory-controller ecosystem (LiteDRAM in particular — used as the cross-validation reference; see §6.4), but it is not a re-implementation of any existing controller. The design's distinguishing features:

1. **Centralized FR-FCFS scheduler with full out-of-order issue.** A single scheduler scans the full transaction queue every cycle, considering all banks' readiness and row-hit state

in one decision. This is closer to the commercial-vendor pattern (Synopsys, Cadence, Xilinx MIG) than to most open-source RTL controllers — open-source controllers typically expose OoO only at the cross-bank level (a per-bank FIFO model where bank parallelism is the only source of reordering). True per-port within-stream reordering — where a younger row-hit request bypasses an older row-conflict request — is uncommon outside vendor IP. AXI4 per-ID ordering is honored at the response side regardless. **For real-time / safety-critical / formally-verifiable systems**, OoO reordering can be disabled at runtime via the `SCHED_TUNING.force_inorder` CSR field (§3.2); the scheduler degrades to a first-ready FIFO that preserves arrival order at the cost of 10–25% sustained bandwidth on streaming workloads.

2. **Multi-rank support with JEDEC-compliant cross-rank ODT.** Most open-source DDR2/3 controllers are single-rank only — LiteDRAM’s default config, UberDDR3, and most academic prototypes assume one rank per channel. This controller supports 1, 2, or 4 ranks via the `NUM_RANKS` build parameter. Per-rank `CS_n`, `CKE`, and `ODT` signals are driven; the rank-aware refresh manager tracks `last_ref_age` per (rank, bank); and the cross-rank ODT termination rules (read from rank $N \rightarrow$ ODT-high on rank $\neq N$ during the read window; write to rank $N \rightarrow$ ODT-high on rank N during the write window) are handled inside `dfi_cmd_formatter / mode_register` (no standalone ODT block) per JESD79-2 §x.y. Multi-rank is what makes the controller usable on real DIMM-class platforms rather than just on-board point-to-point DRAM.
3. **Unified DDR2 + LPDDR2 engine with one command formatter.** A single controller core supports both memtypes via the runtime `PHY_TIMING.memtype` CSR field. The LPDDR2 path includes the 10-bit CA bus encoding (JESD209-2F Table 60; packed two edges onto `dfi_address` in `dfi_cmd_formatter`), PASR mask handling (MR16/17), per-bank refresh (REFpb), temperature-compensated refresh (MR4 derating), and Deep Power Down — none of which are present in the open-source DDR2/3/4 reference controllers. LPDDR2 is fully functional (reads and writes) and passes the full simulation suite.
4. **DARP-style per-bank refresh scheduling** — picks the *idle* bank with the longest time since last refresh, falling back to oldest-first when no banks are idle. Based on Chang et al. (HPCA 2014); not present in available open-source controllers.
5. **Two-stage page management** — exact lookahead first, history-based fallback second.
 - Stage 1: a lookahead window (`SCHED_TUNING.lookahead_active`, 0 disables) provides exact auto-precharge decisions when the next same-bank request is already visible to the arbiter.
 - Stage 2: when lookahead is inconclusive (shallow queue or bursty traffic), the adaptive-timeout page policy (Ghasempour et al. 2015) uses address-bit-hashed saturating counters to predict the next access. Open-source controllers typically have either lookahead OR a

fixed page policy. The two-stage combination is original to this design.

6. **Characterization-first parameterization.** Page policy, lookahead depth, refresh policy, refresh deferral, and address mapping are all runtime CSR fields with a defined sweep matrix (see §5 and §6.4). Most published controllers expose only a subset of these; few expose them all.
7. **Closed-page policy as an explicit option.** `REFRESH_TUNING.page_policy_or` $\in$ {OPEN, CLOSE} (the HYBRID encoding is retired; adaptive policies are `PAGE_POLICY_CFG.policy_mode`). Most open-source controllers expose only on/off auto-precharge (effectively OPEN with optional lookahead). The CLOSE option is useful for adversarial / security-sensitive workloads where row-state side channels matter.
8. **Single-knob address mapping.** `ADDR_MAP.bank_lsb` slides the bank field within the byte-offset-stripped word address; the column auto-splits below (`col_lo`) and above (`col_hi`) it while the row/rank positions stay invariant. The classic schemes are just settings of this one knob (`bank_lsb == COL_WIDTH = ROW_MAJOR`; `bank_lsb == log2(DRAM_BL) = maximum bank interleave`), and an optional bank XOR-hash (`ADDR_MAP.hash_en / hash_seed`) folds row bits into the bank index to defeat pathological row-conflict strides. There is no separate scheme selector.
9. **AXI4-first interface** with ID-aware out-of-order completion. The AXI integration is a first-class design constraint rather than a wrapper on a native protocol.
10. **Mirrored verification address-mapping object.** The `addr_mapper` mirrors a Python address-mapping class used by the DFI BFM, so the same mapping function drives both RTL and simulation. This eliminates a common source of address-decode bugs between RTL and verification.

8.5.1 Features Adopted from Standard Memory-Controller Patterns

These design patterns are widespread in the memory-controller literature and the open-source controllers we examined. Adopting them is the right call; they are noted here so the design rationale is transparent:

- **Per-bank FSM-free JEDEC “safe” timers** (`bank_timer`, stamped per (rank, bank) by `pumice_bank_timers`) + a central cross-bank/rank timer pool (`global_timers: tRRD / tFAW / tWTR / tRTW / tCCD`) — the standard bank/global split, here without a per-bank state machine.
- **FR-FCFS scheduler baseline** (Rixner et al. ISCA 2000) — the academic baseline; realized in `pumice_cmd_arbiter`.
- **Elastic Refresh deferral** (Stuecheli et al. MICRO 2010) — postponer with deterministic back-to-back drain, in `refresh_ctrl`.

- **Refresh arbitrated against bank readiness** — the arbiter waits for the addressed banks to be safe before issuing REF; established pattern.
- **Periodic ZQCS piggybacked on the refresh window** — established pattern for systems with thermal drift.
- **Init sequencer** issuing the memtype-specific JEDEC MR/init sequence (init_sequencer) — established pattern.

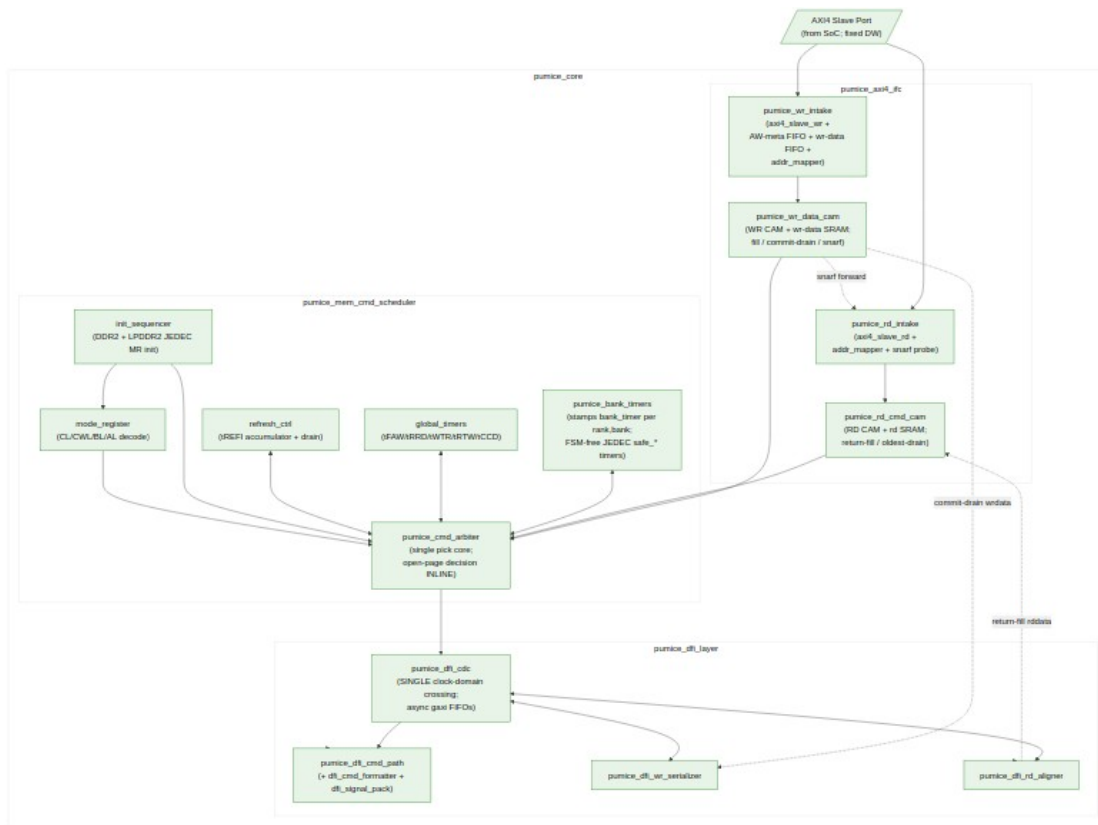
The design rationale for adopting versus extending each is documented in the relevant section.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

9 Block Diagram

9.1 Top-Level Block Diagram

The controller's top-level data and control flow is shown below. AXI4 traffic enters at top-left into `pumice_axi4_layer`, which maps addresses and pushes per-direction records into its two CAMs (`pumice_wr_data_cam`, `pumice_rd_cmd_cam`). The `pumice_scheduler_layer` queries the CAMs each cycle and picks the next abstract command (`pumice_cmd_arbiter` against the bank and global timers). The `pumice_training_layer` (not drawn) raises LPDDR2 ZQ / DQ-calibration maintenance commands on a dedicated `trn_cmd_*` channel into the arbiter and owns the calibration read-capture sideband into the DFI layer (Ch 3.9). The `pumice_dfi_layer` crosses the single controller-to-PHY clock boundary (`pumice_dfi_cdc`) and formats the chosen command into DFI v2.1 wires (`dfi_cmd_formatter / dfi_signal_pack`), while its write serializer and read aligner move write beats out and return read beats in alignment with the scheduled commands. `pumice_top` instantiates `pumice_core` (these four layers) plus the PeakRDL `pumice_csr` block that drives all configuration by name.



Top-Level Block Diagram

Source: [01_block_diagram.mmd](#)

9.2 Data Flow Summary

Write path: 1. AXI master issues an AW/W transaction; `pumice_wr_intake` (an AXI4 slave write engine + AW-meta FIFO + wr-data FIFO) accepts it and splits the host burst at DRAM-burst boundaries. 2. `addr_mapper` translates the flat AXI address into (rank, bank, row, col) using `bank_lsb` (+ optional hash). 3. `pumice_wr_data_cam` fills the write burst into its SRAM and records the command; the `r_fdone` fill-complete flag gates schedulability and `snarf`. 4. `pumice_cmd_arbiter` queries the wr/rd CAMs against the per-(rank,bank) `safe_*` outputs from `pumice_bank_timers` and the turnaround windows from `global_timers`; it picks the next abstract command (ACT, WR/WRA, RD/RDA, PRE, REF, MRS, NOP) and applies the page policy (open-page decision inline). 5. In `pumice_dfi_layer`, the command crosses the CDC and `dfi_cmd_formatter` encodes it into DFI `cs_n / ras_n / cas_n / we_n / address / bank` wires (DDR2) or the packed CA bus (LPDDR2); closed-page uses WRA (auto-precharge). 6. `dfi_signal_pack` aggregates the per-phase DFI control bus. 7. On the write-fire strobe, the DFI write serializer commit-drains the burst from `pumice_wr_data_cam`'s SRAM and drives `dfi_wrdata / dfi_wrdata_en / dfi_wrdata_mask` with PHY alignment. 8. On retire, the write CAM slot is freed and `pumice_wr_intake` returns the B-response to the AXI master.

Read path: 1. AXI master issues an AR transaction; `pumice_rd_intake` accepts and splits the burst. 2. Address mapping identical to the write path; `pumice_rd_cmd_cam` records the read command. `pumice_rd_intake` probes the write-data CAM: on a `snarf` hit (unscheduled, same-id, same-BL) the read is served directly from the write CAM SRAM without going to DRAM (read-your-write forwarding). 3. `pumice_cmd_arbiter` issues ACT, then RD / RDA via the DFI layer. 4. The DFI read aligner drives `dfi_rddata_en t_rddata_en` cycles after the RD command and captures `dfi_rddata` beats for the burst. 5. Returned beats fill `pumice_rd_cmd_cam`'s SRAM; its oldest-first drain engine (gated on data-ready) streams them back out, tagged with the original AXI ID, on the R channel.

Refresh path: 1. `refresh_ctrl`'s tREFI counter elapses, incrementing the `refresh_pending` accumulator (JEDEC max 8 postponed). 2. When `refresh_pending` exceeds the soft threshold, refresh becomes highest-priority in the arbiter. 3. For REFab: the arbiter waits for the addressed banks to be safe (via `pumice_bank_timers`), then issues REF; the affected timers reload. 4. LPDDR2 per-bank refresh (REFpb) selection is driven by `REFRESH_TUNING.refpb_policy_or`.

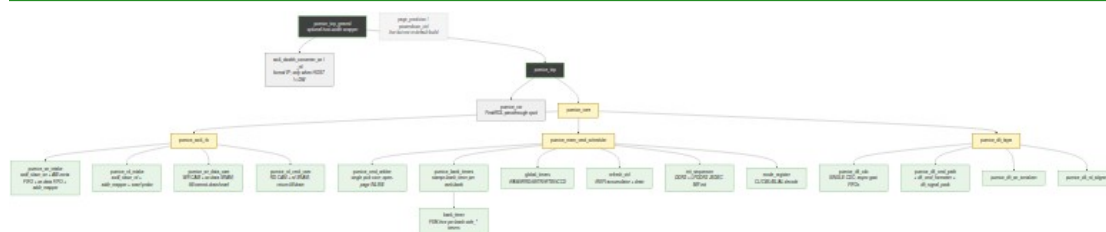
Init / power path: 1. On cold reset, `init_sequencer` holds off AXI traffic. 2. `init_sequencer` executes the memtype-specific JEDEC MR/init sequence, driving CKE / RESET_N and issuing MR-write strobes into `mode_register`. `mode_register` propagates live CL / CWL / BL / AL to the DFI layer. 3. On completion, `init_done_o` asserts and the arbiter begins servicing AXI traffic. 4. Power-down / self-refresh entry (Active / APD / SR / DPD) is managed by `powerdown_ctrl` (present but optional; not in the default top build), with SR-entry coordinated with `refresh_ctrl`.

Detailed per-module behavior follows in Chapter 3.

10 Module Hierarchy

The controller is decomposed into **four layers under `pumice_core`**, each built from leaf FUBs. Each FUB is independently verified at the unit level; each layer is verified as an integration (“macro”) unit. See [FUB Breakdown](#) for the per-FUB role descriptions and [Chapter 3](#) for the behavioral details.

10.1 Hierarchy Tree



Module Hierarchy

Source: 02_module_hierarchy.mmd

10.2 Layered Organization

```

SoC level
├── pumice_top_geared          ← OPTIONAL host-width wrapper
│                               (axi4_dwidth_converter_wr/_rd when
HOST != DW)
├── pumice_top                ← instantiated by the SoC
│   ├── pumice_csr            ← PeakRDL passthrough cpuif register
block
│   └── pumice_core            ← the controller proper
│       ├── pumice_axi4_layer  ← host AXI + wr/rd CAMs
│       └── pumice_scheduler_layer ← "what command to issue this
cycle"
│           ├── pumice_training_layer ← LPDDR2 ZQ + DQ calibration
(inert for DDR2)
│           └── pumice_dfi_layer      ← single datapath CDC + DFI
v2.1 datapath

```

The single controller-to-PHY clock crossing for the datapaths lives inside `pumice_dfi_layer` (`pumice_dfi_cdc`, async FIFOs only). The training layer owns one small sideband crossing of its own (`cal_expect` down, captured MRR data up) for the read aligner’s calibration capture — see [Training](#). The host AXI interface, CAMs, and command scheduler all run on `aclk`; the DFI command path, write serializer, and read aligner run on `dfi_clk`.

10.3 Layer Groupings

Layer / macro	FUBs
pumice_axi4_layer	pumice_wr_intake, pumice_rd_intake, addr_mapper, pumice_wr_data_cam, pumice_rd_cmd_cam
pumice_scheduler_layer	pumice_cmd_arbiter, pumice_bank_timers (bank_timer), global_timers, refresh_ctrl, init_sequencer, mode_register
pumice_training_layer	pumice_zq_ctrl (periodic ZQ calibration), pumice_lp_cal (one-shot MRR DQ calibration)
pumice_dfi_layer	pumice_dfi_cdc, pumice_dfi_cmd_path (dfi_cmd_formatter, dfi_signal_pack), pumice_dfi_wr_serializer, pumice_dfi_rd_aligner
pumice_core	(wraps the four layers above)

Also present in the tree but not in the default top build: `page_predictor` and `powerdown_ctrl` (referenced / optional; verify against the filelists in `rtl/filelists/`).

Chapter 3 follows the layer ordering (one section per architectural layer, with module names hyperlinked to their MAS chapters).

10.4 Memtype-Conditional Logic

Memtype (DDR2 vs LPDDR2) is a runtime CSR selection (`PHY_TIMING.memtype`), not an elaboration parameter. The memtype-dependent logic lives in:

- **dfi_cmd_formatter** — a memtype branch: DDR2 drives ras/cas/we; LPDDR2 packs the 10-bit JESD209-2F CA-bus command (two edges) onto `dfi_address`.
- **init_sequencer** — runs the DDR2 or LPDDR2 JEDEC MR/init sequence.
- **mode_register** — CL / CWL / BL / AL decode differs by memtype.
- **pumice_training_layer** — both FUBs gate on `memtype == MEMTYPE_LPDDR2`; DDR2 has no ZQ pin and no MRR, so the layer is inert for DDR2 builds.

Both memtypes pass the full simulation suite.

10.5 Clock-Domain Crossings

The datapath clock-domain crossing is exactly one: `pumice_dfi_cdc` inside `pumice_dfi_layer`, built from asynchronous `gaxi` FIFOs. One FIFO word is one DFI cycle, so the command / write-data / read-data datapaths are bubble-free. Everything up to the CDC is on `aclk`; everything past it is on `dfi_clk`.

The training layer adds one narrow sideband crossing (`sync_pulse` / `cdc_synchronizer`, not FIFOs): `cal_expect` down to `dfi_clk` to arm the read aligner's calibration capture, and the captured MRR beat back up to `aclk`. It carries no datapath traffic — see Chapter 3.9.

10.6 What Changed vs the Earlier Architecture

The controller was rearchitected from an earlier FSM-based, elaboration-time decomposition into the current three-layer, CSR-driven design. Notably:

- **Retired:** `txn_queue`, `bank_machine`, `xbank_timers`, `cmd_encoder`, `odt_ctrl`, `scheme_selector`, the `*_macro`-as-architecture names, and `axi_intake/axi_frontend`.
- **Replaced by:** the two CAMs (`pumice_wr_data_cam` + `pumice_rd_cmd_cam`), FSM-free `bank_timer` (stamped by `pumice_bank_timers`), `global_timers`, `dfi_cmd_formatter` (+ `dfi_signal_pack`), and the single `ADDR_MAP.bank_lsb` knob.
- **Introduced:** the single-CDC DFI layer, the PeakRDL `pumice_csr` block (configuration by name), and the optional `pumice_top_geared` host-width wrapper.

RTL Design Sherpa · Learning Hardware Design Through Practice · [GitHub](#) · [Documentation Index](#) · [MIT License](#)

11 Top-Level Interfaces

This section is the integrator's quick-reference for the controller's top-level module port list. Detailed protocol semantics for each interface are in Chapter 4 (Interfaces); this is the **wire-level** view — names, directions, widths, parameter dependencies.

The top module is `pumice_top` and has four external interfaces:

1. AXI4 slave (host request path), signals prefixed `s_axi_*`
2. DFI v2.1 master (DRAM PHY path), signals suffixed `_o / _i`
3. CSR register `cpuif` (configuration and observation) — a PeakRDL passthrough bus, **not** APB
4. Clocks and resets (two domains) plus `init_done_o`

All four are exposed at the top level as flat per-signal ports. SystemVerilog interfaces are **not** used at the top boundary because the framework's interface convention (see `CocoTBFramework` BFM components) drives flat-port pin-level handshakes. Internal sub-blocks use interface objects, but the top-of-design is flat for synthesis and BFM compatibility.

An optional outer wrapper, `pumice_top_geared`, adds a `HOST_AXI_DATA_WIDTH` parameter and inserts formally-verified AXI data-width converters ahead of `pumice_top`; when `HOST_AXI_DATA_WIDTH == DW` the converters are bypassed and the build is bit-identical. This section describes `pumice_top` itself.

11.1 Parameter-Dependent Widths

The core data width is fixed by the DFI geometry: $DW = \text{DRAM_BEAT_WIDTH} * \text{DFI_RATE}$. There is no separate `AXI_DATA_WIDTH` parameter on `pumice_top` — the host AXI data width equals `DW`. Throughout this section:

- $AW = \text{AXI_ADDR_WIDTH}$ (default 32)
- $DFI_RATE = \text{DFI phase count} / \text{frequency ratio}$ (default 2)
- $DRAM_BEAT_WIDTH = \text{per-beat DRAM data width}$ (default 64)
- $DW = \text{DRAM_BEAT_WIDTH} * \text{DFI_RATE}$ (host AXI + DFI word width; default 128)
- $SW = DW/8$ (byte-strobe width)
- $IW = \text{AXI_ID_WIDTH}$ (default 8)
- $NR = \text{NUM_RANKS}$ (default 1)
- $NB = \text{NUM_BANKS}$ (default 8)
- $RW = \text{ROW_WIDTH}$ (default 14)
- $CW = \text{COL_WIDTH}$ (default 10)
- $BKW = \lceil \log_2(\text{NUM_BANKS}) \rceil$

- CSR_ADDR_W = CSR address width (default 12)

11.2 1. AXI4 Slave (Host-side)

All host AXI ports carry the s_axi_ prefix.

11.2.1 Write Address Channel (AW)

Signal	Direction	Width	Description
s_axi_awid	input	IW	Write transaction ID
s_axi_awaddr	input	AW	Write start address
s_axi_awlen	input	8	Burst length – 1
s_axi_awsz	input	3	Beat size encoding
s_axi_awburst	input	2	Burst type (INCR mandatory; FIXED / WRAP optional)
s_axi_awlock	input	1	Lock (ignored — exclusives not supported)
s_axi_awcache	input	4	Cache hint (observed; no behavior)
s_axi_awprot	input	3	Protection bits (observed)
s_axi_awqos	input	4	QoS hint (boosts scheduler priority — see §3.2)
s_axi_awregion	input	4	Region (observed)
s_axi_awuser	input	1	User sideband (observed)
s_axi_awvalid	input	1	Address-valid
s_axi_awready	output	1	Address-ready

11.2.2 Write Data Channel (W)

Signal	Direction	Width	Description
s_axi_wdata	input	DW	Write data
s_axi_wstrb	input	SW	Byte enables
s_axi_wlast	input	1	Last beat of burst
s_axi_wuser	input	1	User sideband
s_axi_wvalid	input	1	Data-valid
s_axi_wready	output	1	Data-ready

11.2.3 Write Response Channel (B)

Signal	Direction	Width	Description
s_axi_bid	output	IW	Response ID
s_axi_bresp	output	2	Response code (OKAY / SLVERR)
s_axi_buser	output	1	User sideband
s_axi_bvalid	output	1	Response-valid
s_axi_bready	input	1	Response-ready

11.2.4 Read Address Channel (AR)

Signal	Direction	Width	Description
s_axi_arid	input	IW	Read transaction ID
s_axi_araddr	input	AW	Read start address
s_axi_arlen	input	8	Burst length – 1
s_axi_arsize	input	3	Beat size encoding
s_axi_arburst	input	2	Burst type
s_axi_arlock	input	1	Lock (ignored)
s_axi_arcache	input	4	Cache hint
s_axi_arprot	input	3	Protection
s_axi_arqos	input	4	QoS hint
s_axi_arregion	input	4	Region
s_axi_aruser	input	1	User sideband
s_axi_arvalid	input	1	Address-valid
s_axi_arready	output	1	Address-ready

11.2.5 Read Data Channel (R)

Signal	Direction	Width	Description
s_axi_rid	output	IW	Read data ID
s_axi_rdata	output	DW	Read data
s_axi_rresp	output	2	Response code
s_axi_rlast	output	1	Last beat
s_axi_ruser	output	1	User sideband
s_axi_rvalid	output	1	Data-valid

Signal	Direction	Width	Description
s_axi_rready	input	1	Data-ready

11.3 2. DFI v2.1 Master (PHY-side)

The DFI v2.1 sub-interfaces present on this controller are limited to **command**, **write-data**, **read-data**, and the **init handshake**. Training, frequency-change, low-power, update, and CRC sub-interfaces are not driven — see §2.1 Out of Scope.

The DFI ports are flat, per-phase-packed buses (the DFI_RATE phases are concatenated into one vector) rather than SystemVerilog arrays. All DFI ports carry the `_o` (output) or `_i` (input) suffix. Command formatting runs on `dfi_clk`, past the single CDC in `pumice_dfi_layer`.

11.3.1 DFI Command Bus

Signal	Direction	Width	Description
dfi_addresses_o	output	RW * DFI_RATE	Address operand (row / column); LPDDR2 CA-bus command is packed here
dfi_bank_o	output	BKW * DFI_RATE	Bank operand
dfi_cs_n_o	output	NR * DFI_RATE	Active-low chip-select, per rank per phase
dfi_ras_n_o	output	DFI_RATE	RAS strobe per phase (DDR2; idle for LPDDR2)
dfi_cas_n_o	output	DFI_RATE	CAS strobe per phase (DDR2; idle for LPDDR2)
dfi_we_n_o	output	DFI_RATE	WE strobe per phase (DDR2; idle for LPDDR2)
dfi_odt_o	output	NR * DFI_RATE	ODT, per rank per phase

11.3.2 DFI Write-Data Bus

Signal	Direction	Width	Description
dfi_wrdata_o	output	DW	Write data (all phases packed; = one DFI word)
dfi_wrdata_en_o	output	DFI_RATE	Per-phase write-data enable

Signal	Direction	Width	Description
dfi_wrdata_mask_o	output	SW	Per-byte write mask (DFI polarity: 1 = do-not-write)

11.3.3 DFI Read-Data Bus

Signal	Direction	Width	Description
dfi_rddata_en_o	output	DFI_RATE	Per-phase read-data enable
dfi_rddata_i	input	DW	Read data (all phases packed)
dfi_rddata_valid_i	input	DFI_RATE	Per-phase read-data valid

11.3.4 DFI Init Handshake

Signal	Direction	Width	Description
dfi_init_start_o	output	1	Init handshake — start of DRAM init
dfi_init_complete_i	input	1	PHY signals init complete

11.4 3. CSR Register Interface (Configuration / Observation)

Configuration is driven by name from a PeakRDL-generated register block (pumice_csr) via a passthrough **cpuif** request bus (not APB). Address width is CSR_ADDR_W (default 12). Detailed register map in §6.3.

Signal	Direction	Width	Description
s_cpuif_req	input	1	Request valid
s_cpuif_req_is_wr	input	1	1 = write, 0 = read
s_cpuif_addr	input	CSR_ADDR_W	Register address
s_cpuif_wr_data	input	32	Write data
s_cpuif_wr_biten	input	32	Per-bit write enable
s_cpuif_req_stall_wr	output	1	Write back-pressure
s_cpuif_req_stall_rd	output	1	Read back-pressure
s_cpuif_rd_ack	output	1	Read data valid
s_cpuif_rd_err	output	1	Read error
s_cpuif_rd_data	output	32	Read data
s_cpuif_wr_ack	output	1	Write accepted

Signal	Direction	Width	Description
s_cpuif_wr_err	output	1	Write error

11.5 4. Clocks, Resets, and Status

The design has two clock domains — the controller/AXI domain (aclk) and the DFI/PHY domain (dfi_clk) — with the single crossing inside pumice_dfi_layer.

Signal	Direction	Width	Description
aclk	input	1	Controller clock (host AXI, CAMs, scheduler)
aresetn	input	1	Controller reset, active low
dfi_clk	input	1	DFI/PHY clock
dfi_rstn	input	1	DFI reset, active low
init_done_o	output	1	Asserted when DRAM init completes

11.6 Hierarchical Top-Level View

The top-level pinout in shorthand:

```
pumice_top
├── AXI4 slave          (host)          s_axi_awid,
s_axi_awaddr, ..., s_axi_rvalid, s_axi_rready, ...
├── DFI v2.1 master    (PHY)          dfi_address_o, dfi_bank_o,
dfi_cs_n_o, ..., dfi_rddata_i, ...
├── CSR cpuif          (config)       s_cpuif_req,
s_cpuif_addr, ..., s_cpuif_rd_data, ...
└── Clocks + status
dfi_rstn, init_done_o          aclk, aresetn, dfi_clk,
```

The canonical SystemVerilog port list is rtl/top/pumice_top.sv. This section is the **integrator's overview**, not the build-time canonical list.

12 AXI4 Frontend

The AXI frontend is `pumice_axi4_layer` (`rtl/macro/pumice_axi4_layer.sv`). It bolts the repository's common AXI burst splitters onto two “dumb” 1:1 intakes and holds the two CAMs that decouple AXI-side handshake from the DRAM-side command scheduler:

```
host AXI4 -> [axi_master_wr_splitter] -> pumice_wr_intake ->
pumice_wr_data_cam
          -> [axi_master_rd_splitter] -> pumice_rd_intake ->
pumice_rd_cmd_cam
```

The intakes each contain a common `axi4_slave` (write or read side) plus an `addr_mapper` that translates the flat AXI address into (rank, bank, row, col). The write intake also carries an AW-meta FIFO and a write-data FIFO; the read intake carries a snarf (read-your-write) probe into the write CAM. This chapter covers the splitters, the intakes, and `addr_mapper`. The two CAMs are covered in the data-paths chapter (`07_data_paths.md`).

12.1 Burst Splitters

`axi_master_wr_splitter` and `axi_master_rd_splitter` are formally-verified common modules. Each host burst is split at DRAM-burst-byte boundaries so every burst that reaches an intake maps to exactly one DRAM burst. The alignment mask is `DRAM_BURST_BYTES - 1`, where `DRAM_BURST_BYTES = BL * (DRAM_BEAT_WIDTH / 8)`. Splitting is transparent to the AXI master: a host burst that crosses a DRAM-burst boundary is decomposed into multiple aligned sub-bursts, each of which becomes one CAM entry / one DRAM command.

12.2 `pumice_wr_intake` / `pumice_rd_intake`

12.2.1 Purpose

Terminate the AXI channels (via the embedded `axi4_slave`), decode each transaction's address, and push a {rank, bank, row, col, id} insert request downstream into the corresponding CAM. The intakes are deliberately “dumb”: there is no reordering or scheduling here — that lives in the scheduler layer reading the CAMs.

12.2.2 Write intake

- Accepts AW transactions and issues an `aw_push` insert into `pumice_wr_data_cam`.
- Streams W beats into the write-data FIFO tagged for the CAM SRAM fill mover.
- Returns the B response when the CAM signals commit-done for the transaction ID (`wr_done_valid` / `wr_done_id`), matching AXI4 posted-write semantics.

12.2.3 Read intake

- Accepts AR transactions and issues an `ar_push` insert into `pumice_rd_cmd_cam`.
- Before committing a miss to the read CAM, it probes the write CAM with a `snarf` query (`snarf_probe_*`). On a `snarf` hit the read is serviced directly from the write CAM's SRAM (read-your-write forwarding) rather than being scheduled to DRAM.
- Drains the read CAM (in AR order) onto the AXI R channel with the correct ID and `rlast`.

12.2.4 Per-ID Ordering

AXI4 requires reads from the same ID to return in order and writes from the same ID to commit in order. Because each intake inserts in AR/AW arrival order and the read CAM drains in insert order, per-ID ordering is preserved. The CAMs allow row-hit scheduling to reorder DRAM commands across entries, but the AXI completion layer honors per-ID order.

12.2.5 Backpressure

- `AW/AR.ready` deassert when the corresponding CAM has no free entry (`ins_ready` low).
 - `W.ready` deasserts when the write-data FIFO/SRAM fill path is full.
 - `R` and `B.valid` follow standard AXI protocol; a stalled consumer stalls only the drain path and does not corrupt the controller core.
-

12.3 `addr_mapper`

12.3.1 Purpose

Translate a flat AXI address into DRAM coordinates (`rank`, `bank`, `row`, `col`). RTL: `rtl/fub/addr_mapper.sv`. It is combinational and single-stage, instantiated inside each intake.

12.3.2 Interfaces

Inputs:

- `axi_addr_i` — `AXI_ADDR_WIDTH` bits
- `bank_lsb_i[4:0]` — the `ADDR_MAP.bank_lsb` CSR field
- `hash_en_i` — the `ADDR_MAP.hash_en` CSR field
- `hash_seed_i[7:0]` — the `ADDR_MAP.hash_seed` CSR field

Outputs:

- `rank_o` — $\$clog2(\text{NUM_RANKS})$ bits
- `bank_o` — $\$clog2(\text{NUM_BANKS})$ bits
- `row_o` — `ROW_WIDTH` bits
- `col_o` — `COL_WIDTH` bits

12.3.3 Mapping Function (single knob: `bank_lsb`)

There is **no scheme selector** anymore. The mapping is driven by one runtime knob, `ADDR_MAP.bank_lsb`, which places the bank field within the byte-offset-stripped word address (`word = axi_addr >> BYTE_OFFSET_WIDTH`). The column auto-splits around the bank; row and rank stack above the column region and their positions are **invariant**:

```
word address, low -> high:
[ col_lo(bank_lsb) | bank(BW) | col_hi(CW - bank_lsb) | row(RW) |
rank(KW) ]
col = { col_hi, col_lo }
row LSB is always at CW + BW (invariant – only the bank slides)
```

The RTL clamps `bank_lsb` to `[0, COL_WIDTH]` so the `col_hi` width stays non-negative and the row/rank slices land where the geometry expects.

12.3.4 Classic Schemes as `bank_lsb` Settings

The former named schemes are just settings of the one knob:

Effect	Setting	Notes
<code>ROW_MAJOR</code>	<code>bank_lsb == COL_WIDTH</code>	Bank above the whole column; default (0x0A)
<code>max BANK_INTERLEAVE</code>	<code>bank_lsb == log2(DRAM_BL)</code>	Minimal <code>col_lo</code> keeps a DRAM burst inside one bank
<code>partial interleave</code>	any value in between	Column splits around the bank

Software keeps $\log2(\text{DRAM_BL}) \leq \text{bank_lsb} \leq \text{COL_WIDTH}$ so a DRAM burst's column walk stays inside one bank. (`DRAM_BL`, not `cols/burst` and not `BL/DFI_RATE` — the mapper indexes device words, so a burst spans `DRAM_BL` columns here; measured in `dv/tests/fub/test_addr_mapper.py::minimum_bank_lsb_is_measured`. pumice BUG-022.)

12.3.5 Optional Bank XOR-Hash

When `hash_en` is set, the bank index is XOR-folded with row bits and the seed to defeat power-of-two-stride hot-banking (the former `XOR_HASH` scheme, now an orthogonal overlay on any `bank_lsb` placement):

```
bank[i] ^= row[i] ^ row[i + BW] ^ hash_seed[i]
```

The mid-slice index is clamped so it never runs past `ROW_WIDTH`.

12.3.6 Relationship to the DFI BFM

This module mirrors the `AddressMapping` class in the DFI BFM (`CocoTBFramework/components/dfi/`). The same `bank_lsb` / `hash` configuration drives both RTL elaboration-time verification and BFM checking, so the address decode is consistent between simulation and silicon.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

13 Command Scheduler

The command-scheduling layer is `pumice_scheduler_layer` (`rtl/macro/pumice_scheduler_layer.sv`). It is a single controller-clock (`ac1k`) layer that wires together the pick core and all the timing / bring-up support blocks, and emits one abstract DRAM command stream `{op, rank, bank, row, col, ap, mrr}` into a command FIFO for the DFI layer to pack onto phases.

The scheduler does **not** hold the transaction queue — pending requests live in the two CAMs inside `pumice_axi4_layer`. The scheduler reads them through external lookup / oldest / commit / issue ports.

13.1 Composition

`pumice_scheduler_layer` instantiates:

- **`pumice_cmd_arbiter`** — the single pick core (open-page decision is inline).
- **`pumice_bank_timers`** (per-rank/bank `bank_timer` instances) — FSM-free per-bank JEDEC “safe” timers (see `03_bank_machines.md`).
- **`global_timers`** — cross-bank / bus turnaround (`tFAW`, `tRRD`, `tWTR`, `tRTW`, `tCCD`).
- **`refresh_ctrl`** — `tREFI` tracking + postpone accumulator (see `04_refresh.md`).
- **`init_sequencer`** — JEDEC MRS cold-boot init; gates traffic until done (see `05_init_power.md`).
- **`mode_register`** — CL / CWL / BL decode shadow, updated by init MRS writes.
- an output command FIFO (`gaxi_fifo_sync`).

13.2 `pumice_cmd_arbiter`

13.2.1 Purpose

Pick exactly one abstract DRAM command per cycle and push it into the scheduler-to-DFI command interface. It is PHY/nphases-agnostic and single-issue. JEDEC timing readiness is supplied by the per-bank (`pumice_bank_timers`) and global (`global_timers`) inputs; the arbiter never re-derives timing itself.

13.2.2 Inputs

- Per-bank readiness from `pumice_bank_timers`: `bank_act_ready`, `bank_rdwr_ready`, `bank_pre_ready`, `bank_row_active`, `bank_open_row`.

- Global readiness from `global_timers`: `tfaw_ok`, `trrd_ok` (per-rank), `twtr_ok`, `trtw_ok`, `tccd_ok`.
- Per-bank scheduler lookups into both CAMs (query each bank's open row), plus the CAMs' oldest ports.
- Refresh request / drain from `refresh_ctrl`; init command passthrough from `init_sequencer`.
- Training maintenance request `trn_cmd_*` from `pumice_training_layer`, plus its post-grant bus-quiet hold `cal_busy_i` (see `09_training.md`).

13.2.3 Outputs

- Command push `{op, rank, bank, row, col, ap, mrr}` with `cmd_valid / cmd_ready`.
- Event strobes to the timers (`evt_act`, `evt_rd`, `evt_wr`, `evt_pre`, `evt_ap`, `evt_rank`, `evt_bank`, `evt_row`) — fired only on an accepted issue.
- CAM side effects: write-CAM commit, read-CAM issue, refresh grant — all gated on accepted issue.

`op` is drawn from `dram_op_e` (`OP_NOP`, `OP_ACT`, `OP_RD`, `OP_RDA`, `OP_WR`, `OP_WRA`, `OP_PRE`, `OP_PREA`, `OP_REF`, `OP_REFPB`, `OP_MRS`, ...).

13.2.4 Priority Function

Evaluated combinationally each cycle (descending priority):

1. **Init in progress** (`!init_done`) — forward the `init_sequencer` command verbatim; block all normal traffic.
2. **Refresh** (`refresh_req` or drain active) — precharge active banks one per cycle, then — only under `w_ref_safe` (all rows closed in the registered view, nothing row-affecting in flight or guarded, prior REF's tRFC recovery elapsed) — issue REF and assert the refresh grant. Each fired REF loads the arbiter's tRFC down-counter (`TIMINGS_RFC_REFI.tRFC`); while non-zero, ACTs and further REFs are blocked.
3. **Training** (`trn_cmd_req` from the training layer) — only under `w_trn_safe` (every bank idle, nothing row-affecting in flight or guarded, tRFC elapsed, no grant in flight); the `op` passes through verbatim and `trn_cmd_grant` pulses on the fire. During the FUBs' post-grant hold (`cal_busy`), every demand class below is gated off in `w_out_safe`.
4. **Column row-hit** — a RD/WR to an already-open row whose bank is column-ready (`bank_rdwr_ready`) and whose bus turnaround permits it (`tccd_ok` +

twtr_ok for reads / trtw_ok for writes). **Reads have priority over writes**; within each, the **oldest** entry (max CAM relative age) wins the tie-break.

5. **Fallback** — no ready row-hit: ACT the oldest pending op's row on an idle bank (subject to tfaw_ok / trrd_ok and the ACT/PRE guard), or PRE a bank that is open on the wrong row.

The fallback target is read-priority: the read CAM's oldest port is consulted first, then the write CAM's.

13.2.5 ACT/PRE Re-Issue Guard

The bank timers register their readiness outputs (there is a two-cycle latency from an event to act_ready / pre_ready dropping). A stateless combinational arbiter would otherwise re-issue ACT/PRE to the same bank before the timers reflect it. The arbiter therefore keeps a two-cycle per-bank guard (r_guard0 / r_guard1) that blocks a bank for two cycles after an accepted ACT, PRE **or column** to it — columns are included because a just-fired column has not yet dropped the bank's registered pre_ready (tRTP/tWR load), so an ungarded PRE (normal or refresh-drain) could precharge on stale readiness. Columns still self-limit against each other (both CAMs exclude a just-committed / just-issued slot from the next cycle's lookup), so the guard never gates column-vs-column. The shared-DQ-bus burst-occupancy constraint (a BL burst owns the bus for BL/DFI_RATE DFI cycles) is enforced downstream in the DFI command path, not here; the CDC decouples aclk command issue from the dfi_clk DQ timing.

13.2.6 Page Policy (Auto-Precharge)

The column auto-precharge bit is set directly from page_policy_i:

- **OPEN** — ap = 0; rows stay open. Column ops stream to an open row at tCCD rate.
- **CLOSE** — ap = 1; every column op auto-precharges (issues RDA / WRA).
- **Adaptive policies** — the runtime page-policy engine (pumice_page_policy, PAGE_POLICY_CFG.policy_mode): fixed_open idle-timeout close (the adaptive modes adapt_time and adapt_access were RETIRED 2026-09-27, TASK-014). (The former HAPPY_HYBRID enum encoding and page_predictor.sv are retired; the encoding maps to build default.)

The open-page “keep the row open” decision therefore lives inline in the arbiter and the per-bank bank_timer (which keeps the row open on RD/WR), not in a separate predictor or lookahead unit.

13.2.7 Issue Rate

One command issue per controller clock cycle. Multi-phase / multi-cycle placement (including LPDDR2's 2-edge CA word) happens downstream in the DFI layer, so the arbiter always sees a single issue.

13.3 Mode Register Shadow

`mode_register` maintains the live CL / CWL / BL decode. Its shadow is written in lockstep by the `init_sequencer` MRS writes (`mr_seq_we` / `mr_seq_index` / `mr_seq_data`), so the controller's timing decode tracks exactly what was programmed into the DRAM during init. It exposes `cl_o` / `cwl_o` / `bl_o` to the DFI layer.

13.4 v1 Scope Notes

The arbiter picks from a single rank (rank 0) in v1; write-drain watermarking, multi-rank pick, and power-down coordination are documented TODOs. The `busy_o` output aggregates “init not done”, “refresh pending”, “command in the FIFO”, and “any bank row active”.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

14 Bank Timers and Cross-Bank Timers

Per-bank JEDEC timing is tracked by an **FSM-free** `bank_timer` per (rank, bank); cross-bank / bus-turnaround timing is tracked by `global_timers`. There is no multi-state bank machine in this design.

14.1 `bank_timer`

14.1.1 Purpose

Track one DRAM bank's JEDEC “safe” timing. RTL: `rtl/fub/bank_timer.sv`. It is **not a state machine** — it is a set of preset/decrement countdown timers plus a trivial row-open register and a single auto-precharge bit. The arbiter drives one `set_*` strobe per issued command; each timer loads its config value on its trigger, free-runs decrementing (saturating at 0), and reports its constraint as “safe” when the count is 0.

Because the per-command `safe_*` outputs are a purely combinational AND of the relevant timers behind a **single register stage** (the counter), a just-issued command is reflected one cycle later with no multi-stage lag. This was the point of retiring the old 3-state FSM, whose double-registered state was the root cause of the refresh-vs-ACT and column-vs-PRE hazards.

14.1.2 Instantiation

`pumice_bank_timers` (`rtl/fub/pumice_bank_timers.sv`) stamps one `bank_timer` per (rank, bank) pair — for the default (`NUM_RANKS=1`, `NUM_BANKS=8`) that is 8 instances. It routes the arbiter's `evt_*` event strobes to the addressed bank and fans the per-bank readiness back out to the arbiter.

14.1.3 Timers

Each timer loads on its event, counts down, and saturates at 0:

Timer	JEDEC	Loaded on	Gates
<code>r_rcd</code>	<code>tRCD</code>	ACT	<code>safe_rd/safe_wr</code>
<code>r_ras</code>	<code>tRAS</code>	ACT	<code>safe_pre</code>
<code>r_rc</code>	<code>tRC</code>	ACT	<code>safe_act</code>
<code>r_rp</code>	<code>tRP</code>	explicit PRE or auto-PRE fire	<code>safe_act</code>
<code>r_preblk</code>	<code>tRTP</code> (RD) / <code>tWR</code> (WR)	RD / WR	<code>safe_pre</code> only

The timer config inputs (`t_rcd_i`, `t_rp_i`, `t_ras_i`, `t_rc_i`, `t_wr_i`, `t_rtp_i`) are controller-cycle, command-relative values supplied by the scheduler layer from CSRs.

14.1.4 Row State (minimal — not an FSM)

Two elements track row occupancy:

- `r_row_valid + r_open_row` — set on ACT (captures `row_i`), cleared on an explicit PRE or when an auto-precharge completes.
- `r_ap_pending` — a single bit set from the `set_ap_i` qualifier on a RD/WR. When both `r_preblk` and `r_ras` reach 0, the auto-precharge “fires” (`w_ap_fire`): the row closes internally and tRP loads — with no scheduler PRE and no extra states.

14.1.5 Combinational Safe Outputs

The readiness outputs are combinational off the timers (single register stage behind):

- `safe_act_o = !r_row_valid && (r_rp == 0) && (r_rc == 0)` — bank precharged, tRP since PRE, tRC since last ACT.
- `safe_rd_o = r_row_valid && (r_rcd == 0) && !r_ap_pending` — row open, tRCD met, not auto-precharging.
- `safe_wr_o = safe_rd_o` — same condition.
- `safe_pre_o = r_row_valid && (r_ras == 0) && (r_preblk == 0) && !r_ap_pending` — row open, tRAS and tRTP/tWR met, not auto-precharging.

Open-page behavior falls out naturally: RD/WR only load `r_preblk` and (optionally) the auto-PRE bit, so column commands stream to an open row. The tCCD / tWTR / tRTW turnaround constraints are enforced globally in `global_timers` and ANDed by the arbiter, not here.

14.1.6 Observability State

For debug only, `state_o` is derived combinatorially from the timers into a `bank_state_e` value (BANK_IDLE, BANK_ACTIVATING, BANK_ACTIVE, BANK_PRECHARGING). No downstream logic depends on it — it is purely an observation output alongside the `obs*_nz` timer-nonzero flags.

14.1.7 Refresh Interaction

There is no per-bank refresh handshake. The arbiter serializes refresh: on a refresh request it precharges active banks one per cycle (using `safe_pre / bank_row_active`), then issues REF once no bank has an open row AND `w_ref_safe` holds (no in-flight/guarded row op; prior tRFC recovery elapsed — see the scheduler chapter). REF recovery (tRFC) is an arbiter-side down-counter; the `bank_timer` never sees REF events. The `bank_timer` participates only through its standard row-open and `safe_pre` outputs.

14.2 global_timers

14.2.1 Purpose

Enforce the cross-bank and shared-bus turnaround constraints that a single bank cannot see locally. RTL: `rtl/fub/global_timers.sv`. Implemented once (not per bank) because these constraints are inherently global.

14.2.2 Tracked Constraints

Constraint	Description	Scope
tFAW	At most 4 ACTs in any rolling tFAW window	per-rank
tRRD	Minimum gap between any two ACT commands	per-rank
tWTR	Write-to-read data-bus turnaround	global
tRTW	Read-to-write data-bus turnaround	global
tCCD	Column-to-column spacing	global

14.2.3 Interface

The timers consume the arbiter's `evt_act` (with `evt_act_rank`), `evt_rd`, and `evt_wr` strobes and produce the readiness window signals the arbiter ANDs into its pick:

- `tfaw_window_ok_o` / `trrd_window_ok_o` — per-rank vectors, gate ACT.
- `twtr_global_ok_o` / `trtw_window_ok_o` / `tccd_window_ok_o` — gate column commands.

tFAW / tRRD are kept per-rank because they are device-local activate-stagger limits; the data-bus turnaround windows (tWTR / tRTW / tCCD) are global because the DQ bus is shared across ranks.

All five are registered from the **next** state of their counters rather than the current one, so the arbiter may AND them into its pick directly and every JEDEC window holds. They were previously flopped from the current state, which left each gate open for one cycle past the command that should have closed it – see pumice ISSUE-018, and `formal/pumice/global_timers` for the proof that the published outputs are now sufficient on their own.

15 Refresh Controller

`refresh_ctrl` (`rtl/fub/refresh_ctrl.sv`) owns tREFI timing and refresh request accounting. It does not issue commands itself — it raises a request to the command arbiter, which serializes the precharge-then-REF sequence and pulses a grant once REF reaches the DFI bus.

15.1 `refresh_ctrl`

15.1.1 Purpose

Track the interval between mandatory refreshes and tell the scheduler when refresh is owed. JEDEC permits up to 8 postponed refreshes; `refresh_ctrl` uses an 8-deep accumulator plus a drain-quota mechanism so a batch of owed refreshes can be granted back-to-back.

15.1.2 Interface

Signal	Direction	Purpose
<code>t_refi_i[15:0]</code>	input	Refresh interval in MC cycles (CSR-backed)
<code>refresh_burst_i[3:0]</code>	input	1..8 REFs to drain per request cycle (CSR-backed)
<code>refpb_mode_i</code>	input	0 = REFab, 1 = REFpb (LPDDR2) bank-rotor select
<code>enable_i</code>	input	Gates tREFI counting; driven by <code>init_done</code>
<code>refresh_req_o</code>	output	High while pending refreshes remain
<code>refresh_grant_i</code>	input	Pulsed by the arbiter when REF issues on the bus
<code>refresh_drain_active_o</code>	output	High during a back-to-back drain burst
<code>refresh_kind_o</code>	output	Registered REFab/REFpb selector
<code>refresh_bank_o</code>	output	Bank rotor value (valid in REFpb mode)
<code>pending_refreshes_o</code>	output	Current accumulator value
<code>obs_*</code>	output	Internal state harvested for CSR readout

15.1.3 tREFI Counter and Pending Accumulator

`r_refi_cnt` counts down from `t_refi_i`. It only ticks while `enable_i` is high (i.e. after `init` completes); before that it is held reloaded at `t_refi_i`. On expiry it reloads and the pending accumulator `r_pending` increments by 1 (saturating at the JEDEC maximum of 8). Each accepted

grant decrements `r_pending`. A simultaneous expiry and grant is a net-zero change. `refresh_req_o` stays high whenever `r_pending > 0`.

Saturating at 8 is JEDEC-conformant: the controller is expected to keep at most 8 postponed refreshes outstanding before it must catch up. If the system stays bandwidth-starved past 8 x tREFI, the counter saturates (a DRAM retention violation looming) rather than growing unbounded.

15.1.4 Drain Quota

`r_burst_remaining` implements back-to-back draining. When the previous burst is fully drained and pending work exists, it (re)loads `min(refresh_burst_i, r_pending)`. Each grant decrements it. While `r_burst_remaining > 0` and `r_pending > 0`, `refresh_drain_active_o` is asserted, which the arbiter reads as “keep granting REF back-to-back without yielding to reads/writes”. Setting `refresh_burst_i = 1` disables batching (one REF per tREFI).

15.1.5 REFpb Bank Rotor

When `refpb_mode_i` is set (LPDDR2 per-bank refresh), `r_bank_rotor` advances `0..NUM_BANKS - 1` on each grant and is exposed on `refresh_bank_o`. In REFab mode it stays at 0. The current default build wires `refpb_mode_i = 0` (REFab) in the scheduler.

15.2 Arbiter-Side Refresh Sequence

`refresh_ctrl` only raises `refresh_req / refresh_drain`. The precharge-then-REF sequence is performed by `pumice_cmd_arbiter` at refresh priority (second only to init):

1. While any bank on the target rank has an open row, precharge the active banks one per cycle (lowest ready bank first, honoring the ACT/PRE guard).
2. Once no bank has an open row **and w_ref_safe holds** — nothing row-affecting in flight or inside its 2-cycle guard window (the registered bank view alone is 2-3 cycles stale, which once let a REFab collide with a just-issued ACT), and the previous REF’s tRFC recovery elapsed — issue `OP_REF` and assert `refresh_grant_o` to `refresh_ctrl`, decrementing the accumulator / drain quota. On each fired REF the arbiter loads a **tRFC down-counter** from `TIMINGS_RFC_REFI.tRFC(t_rfc_i)`; while non-zero, ACT picks and further REFs are blocked (mission-mode refresh recovery — init-time refreshes wait `INIT_TIMING1.t_rfc_wait` separately).
3. Repeat back-to-back while `refresh_drain_active` is high (each REF spaced by tRFC), until the accumulator is drained.

This “wait until all banks are precharged” behavior is implicit in the arbiter’s readiness gating rather than an explicit bank-grant handshake.

15.3 Notes and Scope

- **Self-refresh / power-down** are handled by the separate `powerdown_ctrl` block (see `05_init_power.md`), not by `refresh_ctrl`.
- **PASR, DARP idle-bank-first selection, temperature-compensated tREFI, and periodic ZQCS** are not part of the current `refresh_ctrl` RTL. REFpb selection is a simple bank rotor; the richer selection policies remain future work.
- **REFab vs REFpb** for LPDDR2 is selectable via `refpb_mode_i` and the bank rotor, but the default build uses REFab.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

16 Init Sequencer and Power-Down

Cold-boot bring-up is handled by `init_sequencer`; low-power entry/exit by `powerdown_ctrl`.

16.1 `init_sequencer`

16.1.1 Purpose

Sequence post-reset DRAM bring-up: the DFI init handshake, JEDEC mode-register loads, precharge, and refresh. RTL: `rtl/fub/init_sequencer.sv`. It is a compact hard-coded FSM (not a microprogram ROM). Its commands are **issued to the DRAM**, not merely shadowed — this was the fix for the on-board bring-up failure where an earlier shadow-only walk left the read DLL unlocked.

16.1.2 Command Path

While `init_busy_o` is high, the scheduler stays idle and forwards the sequencer's single-cycle command pulses (`init_cmd_valid_o` / `init_cmd_op_o` / `init_cmd_bank_o` / `init_cmd_row_o`) straight to `dfi_cmd_formatter`. Each command state is occupied for exactly one cycle, then the FSM parks in `S_WAIT` for the JEDEC inter-command delay before advancing. Because `dfi_cmd_formatter` is always `cmd_ready`, a single-cycle pulse issues exactly one command with no grant handshake.

The `mode_register` shadow is updated in lockstep (`mr_seq_we_o` / `mr_seq_index_o` / `mr_seq_data_o`) so the live CL/CWL/BL decode tracks what was programmed.

16.1.3 CSR-Backed Waits

The inter-command delays are CSR-backed (INIT_TIMING registers), zero-extended into a 16-bit countdown, rather than hardcoded:

Input	JEDEC	Default
<code>t_init_wait_i</code>	CKE / tINIT	512
<code>t_dll_wait_i</code>	DLL lock tDLLK	256
<code>t_mrd_wait_i</code>	tMRD	8
<code>t_rp_wait_i</code>	tRP	8
<code>t_rfc_wait_i</code>	tRFC	8

16.1.4 DDR2 Sequence

Mirrors LiteDRAM's DDR2 PHY init (the reference proven on the Nexys A7 board):

1. Assert `dfi_init_start_o`; wait `dfi_init_complete_i` (the PHY runs its own DLL-lock / IO training), then wait `tINIT`.
2. Precharge All.
3. EMRS in JEDEC order EMRS(2) then EMRS(3) then EMRS(1) — all defaults 0.
4. MRS(0) + DLL reset (`MR0.VAL | 0x100`; reset default `0x533` = `BL8/CL3/tWR3/DLL_RESET`); wait `tDLLK`.
5. Precharge All.
6. Auto Refresh x2 (each followed by `tRFC`).
7. MRS(0) without DLL reset (`MR0.VAL`, reset default `0x433`) — clears the reset bit.
8. EMRS(1) + OCD default (`0x380`) then EMRS(1) + OCD exit (`0x000`).
9. `init_done_o = 1`.

The DDR2 MR values are CSR-backed (`MR0..MR3.VAL`; `MR0` reset `0x433`, `MR1` `0x0000`, `MR2/MR3` `0x0000`) with the OCD default (`0x380`) ORed in by the sequencer; software may retune them before an `init_restart`.

16.1.5 LPDDR2 Sequence (fully functional)

Per JESD209-2F power-up and mode-register configuration:

1. Assert `dfi_init_start_o`; wait `dfi_init_complete_i`; `tINIT` settle.
2. `MRW(MR63)` = Reset (OP don't-care).
3. `MRW(MR10)` = `0xFF` ZQ Init Calibration.
4. `MRW(MR1)` = `0x23` (`BL8 / nWR3`), `MRW(MR2)` = `0x01` (`RL3 / WL1`), `MRW(MR3)` = `0x02` (`DS 40 ohm`).
5. `init_done_o = 1`.

Only `MR1/MR2/MR3` update the `CL/CWL/BL` decode shadow; `MR63/MR10` are issued to the DRAM but not shadowed (they exceed the 5-bit shadow index). The MR index (`MA`, up to `MR63`) and data (`OP`) are carried packed as `{MA[5:0], OP[7:0]}` in the ROW request field, so `dfi_cmd_formatter` can build the full LPDDR2 CA `MRW` word — a 3-bit bank port alone could not reach `MR10/MR63`.

16.1.6 Memtype Selection

`memtype_i` (from the `PHY_TIMING` CSR: 0 = DDR2, 1 = LPDDR2) chooses the sequence at runtime. After the shared DFI-init step, DDR2 branches to `S_PREA1` and LPDDR2 to `S_L_RESET`.

16.1.7 Status Outputs

Signal	Purpose
dfi_init_start_o	High after leaving reset
init_busy_o	High until the FSM reaches S_DONE
init_done_o	Asserted at S_DONE
zqcl_req_o	Tied off — DDR2 has no ZQCL

16.2 powerdown_ctrl

16.2.1 Purpose

Idle-detect into a low-power state. RTL: rtl/fub/powerdown_ctrl.sv. It is available but not in the default top build; it is documented here for completeness.

16.2.2 Modes

- **PDE (Precharge Power Down)** — CKE low, banks may stay active. Wakes on new activity within $\sim t_{XPDLL}$ cycles.
- **SR (Self Refresh)** — CKE low plus an SRE command issued by the scheduler; the DRAM self-refreshes internally and the controller stops issuing REFs. Exit requires SRX plus t_{XSDLL} (DDR2) / t_{XSR} (LPDDR2) before new commands. SR entry requires all banks precharged first (the scheduler enforces the precondition).

16.2.3 Selection

- enable_sref_i high: on the idle threshold, request SR (pdn_kind_o = 1).
- enable_sref_i low, enable_pde_i high: request PDE (pdn_kind_o = 0).
- neither: never request.

16.2.4 Scope / TODO

Deep Power Down (LPDDR2), per-rank power-down, and the dfi_dram_clk_disable cooperation with dfi_signal_pack are documented TODOs. The block currently powers down all ranks together and trusts the scheduler not to grant before init completes.

RTL Design Sherpa · Learning Hardware Design Through Practice · GitHub · Documentation Index · MIT License

17 Command Formatter and Signal Pack

Two modules translate the scheduler's abstract command into DFI wire signals:

`dfi_cmd_formatter` (memtype-specific encoding and phase placement) and `dfi_signal_pack` (the final registered pipeline stage). `dfi_cmd_formatter` is instantiated inside `pumice_dfi_cmd_path` in the DFI layer.

17.1 `dfi_cmd_formatter`

17.1.1 Purpose

Translate the scheduler's chosen `dram_op_e` plus (`rank`, `bank`, `row`, `col`) into the multi-phase DFI v2.1 control bus. RTL: `rtl/fub/dfi_cmd_formatter.sv`. It is the only place where DDR2 and LPDDR2 differ at the wire layer, and its outputs are strict-flopped.

17.1.2 Input

The command arrives on `cmd_valid_i` / `cmd_ready_o` (always ready) with `cmd_op_i` (`dram_op_e`), `cmd_rank_i`, `cmd_bank_i`, `cmd_row_i`, `cmd_col_i`, `cmd_len_i`, `cmd_mrr_i` (MRR vs MRW select on the LPDDR2 OP_MRS CA word), and the runtime phase-placement knobs `rd_phase_i` / `wr_phase_i`.

17.1.3 Multi-Phase Output

For `DFI_RATE = N`, every control signal is packed as `N` phases side by side on the bus (`dfi_address_o`, `dfi_bank_o`, `dfi_cas_n_o`, `dfi_ras_n_o`, `dfi_we_n_o`, `dfi_cs_n_o`, `dfi_odt_o`). The decoded command is placed on its target phase and the other phases emit selected/deselected NOP. The target phase is:

- `rd_phase_i` for OP_RD / OP_RDA,
- `wr_phase_i` for OP_WR / OP_WRA,
- phase 0 for everything else (ACT/PRE/REF/MRS have no data-phase contract).

The R/W phase knobs match the PHY's `rdphase/wrphase` contract (e.g. `a7ddrphy` DDR2 CL3 `nphases=2`). Defaults of 0/0 preserve the legacy "everything on phase 0" behavior. `cs_n` is per-rank: `cs_n[r]=0` selects rank `r`; all-ones is a deselected NOP.

17.1.4 DDR2 Encoding

Combinational truth table per JESD79-2 Table 49:

op	cs_n	ras_n	cas_n	we_n	bank	addr
NOP	sel	1	1	1	-	-

op	cs_n	ras_n	cas_n	we_n	bank	addr
ACT	sel	0	1	1	BA	row
RD	sel	1	0	1	BA	col (A10=0)
RDA	sel	1	0	1	BA	col + (1<<10)
WR	sel	1	0	0	BA	col
WRA	sel	1	0	0	BA	col + (1<<10)
PRE	sel	0	1	0	BA	A10=0
PREA	sel	0	1	0	-	(1<<10)
REF	sel	0	0	1	-	-
MRS	sel	0	0	0	MR	MR data

The auto-precharge / all-bank bit is A10. MRS data rides `cmd_row_i` (ROW_WIDTH), not `cmd_col_i` — MR0 = 0x533 needs bit 10, which a 10-bit column field would truncate.

17.1.5 LPDDR2 Encoding (bit-exact CA bus)

For LPDDR2 the command rides the multiplexed 10-bit CA bus over 2 edges, packed as a flat 20-bit word carried on `dfi_address` (low bits); `ras_n/cas_n/we_n` stay idle and `cs_n` asserts for the target rank. The two CA edges are already inside the word, so there is no per-DFI-phase command placement.

The CA word is built **bit-exact to JESD209-2F Table 60**, matching the DV BFM's `lpddr_ca` encoder. Layout: `w_lpddr2_ca[i] = CA{i}` rising edge ($i = 0..9$), `w_lpddr2_ca[10+i] = CA{i}` falling edge. Encoded commands include ACT, RD/RDA, WR/WRA, PRE, PREA, REF (all-bank), REFPB (per-bank), MRW, and MRR; NOP/Deselect drives CA0r..CA3r high. Column bit C0 is implied 0 and never transmitted; the auto-precharge flag lands on CA0f. The transcription reference is `docs/uarch/LPDDR2_CA_ENCODING.md`.

For MRW, MA0..MA5 map to CA4r..CA9r, MA6/MA7 to CA0f/CA1f, and OP0..OP7 to CA2f..CA9f. The init sequencer supplies the full MR index by packing `{MA[5:0], OP[7:0]}` into the ROW field, so MR10/MR63 are reachable.

MRR rides the same `OP_MRS` branch as a mode: `cmd_mrr_i` drives CA3r (1 for MRR, 0 for MRW), so MRR is MRW with CA3r inverted, bit-exact to JESD209-2F §5.12 — the MA select field packing is unchanged and the OP field is don't-care-zero for MRR. LPDDR2 has no ZQCS/ZQCL opcodes, so those stay on the default NOP branch: ZQ calibration is pure MRW (see `09_training.md`).

Note: the module's top-of-file header comment still carries a stale "LPDDR2 (TODO)" line from an earlier revision. The body implements the full bit-exact CA encoding above; LPDDR2 reads and writes are functional and pass the sim suite.

17.1.6 Idle Defaults

When not issuing, every phase is driven to its deselected idle: `cs_n=1`, `ras_n=1`, `cas_n=1`, `we_n=1`, `bank=0`, `address=0`, `odt=0` (DFI v2.1 Table 2 defaults). ODT is driven from this module — there is no standalone `odt_ctrl` block.

17.2 dfi_signal_pack

17.2.1 Purpose

Final pipeline-register stage on the DFI v2.1 bus. RTL: `rtl/fub/dfi_signal_pack.sv`. It latches every command / write-data / read-data-enable input and drives it out the next MC cycle, owning `dfi_dram_clk_disable` and reset-safe output values.

17.2.2 Behavior

- `v1` is a pure one-cycle registered pipeline. Bus widths are `DFI_*_WIDTH * DFI_RATE`, so the multi-phase content from `dfi_cmd_formatter` passes through unchanged. This is where the internal `DW = DRAM_BEAT_WIDTH * DFI_RATE` word is presented across the `DFI_RATE` phases.
- `dram_clk_disable` is held at 0 in `v1`; per-phase staggering and power-down assertion are documented TODOs.

17.2.3 Multi-Cycle Commands

LPDDR2's 2-edge CA command is not split here. The 20-bit CA word is already packed onto `dfi_address` by `dfi_cmd_formatter`; the PHY splits it into two DRAM cycles per DFI v2.1.

17.2.4 Verification

The RTL output round-trips against the DFI BFM: the DDR2 truth table against the JESD79-2 reference, and the LPDDR2 CA bus against the shared `lpddr_ca` encoder/decoder (the same layout the RTL encodes).

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

18 Write and Read Data Paths

The data path spans two clock domains. On the controller (ac_lck) side the two CAMs in `pumice_axi4_layer` buffer burst data; on the DFI (dfi_{cl}ck) side the write serializer and read aligner drive/capture the DFI data bus. All four are **de-FSM'd streaming readers** — they are FIFO-fed / beat-counter datapaths with no active/slot state latch.

18.1 Controller-Side CAMs

18.1.1 `pumice_wr_data_cam`

RTL: `rtl/fub/pumice_wr_data_cam.sv`. A write-command CAM plus a write-data SRAM. Entries are keyed on {bank, row, col} with a free-running age; the burst payload lives in an SRAM slot. Three data movers stream over the SRAM:

- **fill** — captures W beats into the slot on insert.
- **commit-drain** — streams the slot to the DFI write path when the scheduler commits the entry.
- **snarf** — forwards a slot to a matching read (read-your-write), limited to unscheduled entries with a matching id and burst length.

Age is compared wrap-safe as a relative age (`age_ctr - entry_age`). The snarf lookup returns the **youngest** match (latest data), the scheduler lookup returns the **oldest** match (in-order commit per row), and the oldest port returns the oldest valid entry for the scheduler fallback. A fill-complete flag gates schedulability and snarf so a partially-filled burst is never committed or forwarded.

There is no standalone `wr2rd_forward` block — write-to-read forwarding is exactly the snarf mover in this CAM.

18.1.2 `pumice_rd_cmd_cam`

RTL: `rtl/fub/pumice_rd_cmd_cam.sv`. The read miss path — an outstanding-read tracker acting as a **reorder buffer**. Entries are keyed {bank, row, col} with a free-running age and N scheduler lookups so reads row-hit-schedule like writes. Data flows the opposite direction from the write CAM: it comes **in** from the DFI read return and drains **out** to `pumice_rd_intake`. DRAM read data returns in issue order and is buffered per entry; it drains in AR (insert) order, which is what enforces per-ID read ordering.

The issue-side oldest/lookups pick the oldest not-yet-issued entry; the drain side releases the oldest valid entry gated on data-ready. There is no snarf port (that is a write-CAM concept).

18.2 DFI-Side Data Movers

Both movers live in the DFI layer on `dfi_clk`. Their internal unit is the **DFI word** ($\text{DRAM_BEAT_WIDTH} * \text{DFI_RATE}$ wide), which already carries all DFI_RATE phases; `dfi_signal_pack` splits it to the pins. One FIFO pop is one DFI cycle, so both are bubble-free.

18.2.1 pumice_dfi_wr_serializer

RTL: `rtl/fub/pumice_dfi_wr_serializer.sv`. On a `wr_fire` from the command path it waits `t_phy_wrlat_i` DFI cycles, then streams one DFI-word per cycle from the write-data FIFO onto `dfi_wrdata` (with `dfi_wrdata_en` and `dfi_wrdata_mask`) until the burst's last word. Because the write CAM pre-stages the burst into the FIFO when the command is scheduled, the burst is already waiting at `wr_fire` and the drive never stalls. The DFI byte mask is the inverted AXI strobe (`mask = ~strb`; AXI `wstrb=1` = write byte, DFI `mask=1` = mask).

18.2.2 pumice_dfi_rd_aligner

RTL: `rtl/fub/pumice_dfi_rd_aligner.sv`. The mirror of the write serializer. On a `rd_fire` it drives `dfi_rddata_en` for the read window starting `t_rddata_en_i` DFI cycles later, captures `dfi_rddata` whenever the PHY asserts `dfi_rddata_valid`, and pushes one DFI-word per valid cycle into the read FIFO as `{last, resp, data}`. `BL_WORDS = BL/DFI_RATE` words per burst; `last` marks the final word so the read intake can split words back into AXI R beats. v1 supports one outstanding read window (single-issue, `tRTW/tCCD` spaced).

18.3 Response Generation

- **B response** — the write intake returns B when the write CAM signals commit-done for the transaction id, matching AXI4 posted-write semantics and decoupling B latency from DRAM-side delays.
- **R response** — the read intake drains the read CAM onto the R channel in AR order with the correct id and `rlast`. In v1 all reads return OKAY (the DFI `resp` field is carried through but DDR2/LPDDR2 have no CA parity to surface).

18.4 Runtime PHY Timing

The write and read timing offsets are runtime CSR knobs rather than hard-coded PHY latencies, so one build ports across PHYs: `t_phy_wrlat` (WR command to `dfi_wrdata_en`) and `t_rddata_en` (RD command to `dfi_rddata_en`), plus the DFI_PHASE CSR (`rd_phase / wr_phase`) covered in the DFI/CSR chapter.

19 DFI Layer and CSR

The bottom of the controller is the DFI layer (`pumice_dfi_layer`), which owns the single controller-to-PHY clock crossing, and the generated register block (`pumice_csr`), which carries all runtime configuration.

19.1 `pumice_dfi_layer`

19.1.1 Purpose

Present the controller-clock command / write-data / read-data streams on one side and the DFI v2.1 pin bus on the other, with exactly one clock-domain crossing between them. RTL: `rtl/macro/pumice_dfi_layer.sv`. The controller runs on `ctl_clk`; the PHY-facing datapath runs on `dfi_clk`.

19.1.2 The Single CDC

`pumice_dfi_cdc` (`rtl/fub/pumice_dfi_cdc.sv`) is the **only** clock-domain crossing in the whole controller. It is built from asynchronous gaxi FIFOs — three streams cross it:

- **cmd** (`ctl_clk -> dfi_clk`): the abstract command stream from the scheduler.
- **wrdata** (`ctl_clk -> dfi_clk`): DFI-word-granular {last, strb, data} drained from the write CAM.
- **rddata** (`dfi_clk -> ctl_clk`): {last, resp, data} captured from the PHY.
- plus the init handshake (`init_start` across, `init_complete` back).

The internal datapath unit is the DFI word (`dfi_wrdata` width, all DFI_RATE phases), so one FIFO word equals one DFI cycle and the crossing is bubble-free. Keeping the CDC in one place is what makes the rest of the controller a single-clock design.

19.1.3 DFI-Domain Datapath

On the `dfi_clk` side the layer instantiates three mechanical stages:

- **pumice_dfi_cmd_path** — pops the command FIFO and drives the multi-phase DFI command bus via `dfi_cmd_formatter` (see `06_encoder_output.md`). It emits `wr_fire` / `rd_fire` strobes to time the data movers and enforces the shared-DQ-bus burst occupancy (`COL_BURST_CYC = BL/DFI_RATE`).
- **pumice_dfi_wr_serializer** — drives `dfi_wrdata` at `t_phy_wrlat` (see `07_data_paths.md`).

- **pumice_dfi_rd_aligner** — captures `dfi_rddata` at `t_rddata_en` (see `07_data_paths.md`).

19.1.4 DFI Sub-Interfaces

The layer drives the three core DFI v2.1 sub-interfaces needed for DDR2 / LPDDR2 operation: the command bus, the write-data interface (`wrdata` / `wrdata_en` / `wrdata_mask`), and the read-data interface (`rddata_en` / `rddata` / `rddata_valid`), plus the DFI init handshake. Update / training / low-power / status sub-interfaces beyond `init_complete` are not driven; the controller manages power via CKE directly.

19.1.5 Runtime PHY Knobs

Rather than hard-coded PHY latencies, the layer takes runtime inputs so one build ports across PHYs: `rd_phase_i` / `wr_phase_i` (per-command DFI sub-phase placement), `t_phy_wrlat_i`, and `t_rddata_en_i`, plus `memtype_i`. These are driven from the CSR (see below).

19.2 pumice_csr

19.2.1 Purpose

Hold all runtime configuration and observation registers. The register block is **generated by PeakRDL** from `rtl/macro/pumice_csr.rdl` (via `bin/peakrdl_generate.py`), with a **passthrough cpuif** — it is not a hand-written APB3 slave.

19.2.2 Interface

The generated block exposes a simple request/response passthrough `cpuif` rather than a full bus protocol:

Signal	Direction	Purpose
<code>s_cpuif_req</code>	input	Request valid
<code>s_cpuif_req_is_wr</code>	input	Write (1) vs read (0)
<code>s_cpuif_addr</code>	input	Register address
<code>s_cpuif_wr_data[31:0]</code>	input	Write data
<code>s_cpuif_wr_biten[31:0]</code>	input	Per-bit write enable
<code>s_cpuif_rd_data[31:0]</code>	output	Read data
<code>s_cpuif_*_ack / *_err</code>	output	Response / flow-control

Signal	Direction	Purpose
/*_stall_*		

Registers are regwidth = 32 / accesswidth = 32. pumice_top connects the cpuif and fans the decoded hwif_out struct fields to the core; the core's status feeds back through hwif_in. Any external bus (APB, AXI-Lite) is adapted to this cpuif outside the block.

19.2.3 Access by Name

DV never uses hard-coded offsets. Configuration is programmed by field name through the generated dv/tbclasses/pumice_regmap.py, which stays in sync with the RDL. This is split-proof — if a register moves, the by-name access still resolves.

19.2.4 Address Mapping Register (ADDR_MAP @ 0x04C)

This register replaced the retired ADDR_MAP_TUNING (the old scheme_or / synth_mask_obs scheme selector is gone). It drives addr_mapper (see 01_axi_frontend.md):

Field	Bits	Default	Purpose
bank_lsb	[4:0]	0x0A	Bank field LSB in the word address (= COL_WIDTH -> ROW_MAJOR)
hash_en	[8]	0	Enable bank XOR-hash bank ^= fold(row) ^ hash_seed
hash_seed	[23:16]	0x00	XOR-hash seed

There is no scheme mux; bank_lsb alone selects row-major vs interleaved placement, and hash_en is an orthogonal overlay.

19.2.5 Key Configuration Registers

The RDL is authoritative; notable runtime-config registers include:

Register	Offset	Notable fields
DFI_PHASE	0x06 0	rd_phase[2:0], wr_phase[6:4] — DFI sub-phase placement (default 0/0)
PHY_TIMING	0x06 4	t_phy_wrlat[7:0] (reset 0; Nexys A7 tuple programs 1), t_rddata_en[15:8] (reset 6), memtype[16] (0=DDR2, 1=LPDDR2), refresh_burst[23:20] (1..8)
ADDR_MAP	0x04 C	bank_lsb, hash_en, hash_seed (above)

Register	Offset	Notable fields
INIT_TUNING	0x050	zq_retries[3:0], init_timeout_ms[15:8]
INIT_TIMING0	0x054	t_init_wait / t_dll_wait (JEDEC init waits, formerly hardcoded)
	area	

memtype selects DDR2 vs LPDDR2 at runtime and is consumed by init_sequencer, dfi_cmd_formatter, and mode_register. Confirm every register and field against the RDL and the generated regmap.

19.2.6 Narrow-Device (x16) Width Granularities

The controller keeps two width concepts distinct: the **DRAM beat** (one DFI phase's data, DRAM_BEAT_WIDTH) and the **physical device word** (the DQ width, DRAM_DEVICE_WIDTH, e.g. 16 for a x16 device). When a beat is wider than the device, burst length (beats per command) and the column-address stride are device-word quantities. The RTL scales the MR0 burst length by $\text{DRAM_BEAT_WIDTH} / \text{DRAM_DEVICE_WIDTH}$ and sizes the column granularity to the device word; missing either produced the on-silicon DDR2 read failure on the Nexys A7 x16 bring-up (over-read plus column-overlap scramble). PHY read-path integration (data-vs-rddata_valid skew and per-command read/write phase) is handled by the runtime DFI_PHASE / PHY_TIMING knobs above rather than hard-coded PHY latencies, so one controller build ports across PHYs.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

20 AXI4 Slave Interface

The controller exposes a standard AXI4 slave (pumice_top / pumice_core) for SoC connectivity. All slave ports use the `s_axi_*` prefix with no `i_/o_` decoration (e.g. `s_axi_awid`, `s_axi_awaddr`, `s_axi_wdata`, `s_axi_rdata`).

The AXI side runs on `aclk` / `aresetn`.

20.1 Data Width Is Fixed at DW

The core's AXI data width is **not** a free parameter. It is fixed at

$$DW = \text{DRAM_BEAT_WIDTH} * \text{DFI_RATE}$$

i.e. one AXI beat carries exactly one DFI word (= `DFI_RATE` DRAM beats). There is no independent `AXI_DATA_WIDTH` parameter on `pumice_top`; `s_axi_wdata` / `s_axi_rdata` / `s_axi_wstrb` are always `DW` / `DW` / `DW/8` bits.

To attach a host AXI bus of a *different* width, use the optional wrapper `pumice_top_geared` (see [Host-Width Gearing](#) below).

20.1.1 Hard width rule (compile-enforced)

`pumice_top_geared`'s `HOST_AXI_DATA_WIDTH` and the DFI word `DW` **MUST** relate by an **exact power-of-two ratio** — `AXI:DFI = G:1` or `1:G`, $G \in \{1, 2, 4, 8, \dots\}$. An initial `assert ... $fatal` in `pumice_top_geared` **fails elaboration** / **Vivado synthesis** on any other pairing, so a bad width choice is a **compile error**, never a silent broken build. Rationale (identical to LiteDRAM, whose AXI frontend is 1:1 with its native port and routes all width change through a power-of-two stride converter): the DFI word is the atomic memory-side transfer, so an AXI beat must be a whole power-of-two number of DFI words. `DW` and `HOST_AXI_DATA_WIDTH` are the **only** compile-time width parameters — `gear` (active phases) and `burst length` are runtime CSRs (build-for-max).

20.2 Channels

Standard AXI4 — all five channels (AW, W, B, AR, R) are mandatory. No exclusive access support; `s_axi_awlock` / `s_axi_arlock` are present but ignored.

20.2.1 AW (Write Address) — SoC master → controller slave

Signal	Width	Notes
<code>s_axi_awid</code>	<code>AXI_ID_WIDTH</code>	Transaction ID
<code>s_axi_awaddr</code>	<code>AXI_ADDR_WIDTH</code>	Flat byte address
<code>s_axi_awlen</code>	8	Burst length minus 1

Signal	Width	Notes
s_axi_awsz	3	Beat size; up to bus width
s_axi_awburst	2	INCR mandatory; FIXED/WRAP optional
s_axi_awlock	1	Present; ignored (no exclusive access)
s_axi_awcache	4	Present; ignored
s_axi_awprot	3	Present; ignored
s_axi_awqos	4	Present; ignored
s_axi_awregion	4	Present; ignored
s_axi_awuser	1	Present; ignored
s_axi_awvalid	1	Master asserts
s_axi_awready	1	Controller asserts when the WR intake / CAM can accept

20.2.2 W (Write Data)

Signal	Width	Notes
s_axi_wdata	DW	Beat data (DW = DRAM_BEAT_WIDTH*DFI_RATE)
s_axi_wstrb	DW/8	Byte enables; inverted to dfi_wrdata_mask
s_axi_wlast	1	Final beat of burst
s_axi_wuser	1	Present; ignored
s_axi_wvalid	1	Master asserts
s_axi_wready	1	Controller asserts while the WR data path can accept

Note: AXI4 W channel has no wid; beats are presumed in the order their AW transactions were issued. The WR intake associates write data with its AW by arrival order.

20.2.3 B (Write Response)

Signal	Width	Notes
s_axi_bid	AXI_ID_WIDTH	Matches awid
s_axi_bresp	2	OKAY or SLVERR
s_axi_buser	1	Present; ignored
s_axi_bvalid	1	Controller asserts on response ready
s_axi_bready	1	Master asserts

20.2.4 AR (Read Address)

Signal	Width	Notes
s_axi_arid	AXI_ID_WIDTH	Transaction ID
s_axi_araddr	AXI_ADDR_WIDTH	Flat byte address
s_axi_arlen	8	Burst length minus 1
s_axi_arsize	3	Beat size
s_axi_arburst	2	INCR mandatory; others optional
s_axi_arlock	1	Present; ignored
s_axi_arcache	4	Present; ignored
s_axi_arprot	3	Present; ignored
s_axi_arqos	4	Present; ignored
s_axi_arregion	4	Present; ignored
s_axi_aruser	1	Present; ignored
s_axi_arvalid	1	Master asserts
s_axi_arready	1	Controller asserts when the RD intake / CAM can accept

20.2.5 R (Read Data)

Signal	Width	Notes
s_axi_rdata	DW	Beat data
s_axi_rid	AXI_ID_WIDTH	Matches arid
s_axi_rresp	2	OKAY (v1 always)
s_axi_rlast	1	Final beat of burst
s_axi_ruser	1	Present; ignored
s_axi_rvalid	1	Controller asserts when beat ready
s_axi_rready	1	Master asserts

20.3 Ordering Rules

- **Within an ID:** AXI4 ordering preserved. Reads from arid = X return in AR order; writes to awid = X commit in AW order.

- **Across IDs:** Out-of-order completion permitted by default (`AXI_000_ACROSS_IDS = true`). Disabling forces in-order across IDs at the cost of bandwidth.

20.4 Burst Type Support

Burst	Mandatory	Notes
FIXED	optional	Enable with <code>SUPPORT_FIXED_BURST</code> ; rarely useful for DRAM
INCR	mandatory	Standard incrementing burst
WRAP	optional	Enable with <code>SUPPORT_WRAP_BURST</code>

20.5 Burst Length and DRAM-Burst Geometry

AXI4 allows up to 256 beats. The frontend `pumice_axi4_layer` splits every host burst at DRAM-burst-byte boundaries (`axi_master_wr_splitter / axi_master_rd_splitter`) so each split request maps to exactly one DRAM burst.

At the DW side the required geometry is:

$(awlen + 1) * DFI_RATE == BL$

i.e. one AXI burst at DW equals one DRAM burst of BL beats. (The older “`AXI_DATA_WIDTH == DRAM_BEAT_WIDTH`” coupling no longer applies — the AXI beat is a full DFI word of `DFI_RATE` DRAM beats.)

20.6 Narrow Transfers

AXI4 supports `awsize < bus width`. The narrow case is handled via `wstrb` — the unused byte lanes are masked. `dfi_wrdata_mask = ~wstrb`, so the DRAM commits only the unmasked bytes.

20.7 Write Response Timing

The B response is **commit-driven**: it is returned once the write is accepted and committed by the controller (`pumice_wr_intake` raises B from the downstream commit-done handshake). A SLVERR B is self-generated by the intake for a malformed request; otherwise OKAY.

20.8 Frontend and Backpressure

The AXI frontend is `pumice_axi4_layer`: dumb 1:1 write and read intakes (`pumice_wr_intake`, `pumice_rd_intake`) feeding a write-data CAM (`pumice_wr_data_cam`, the write buffer / snarf

source) and a read reorder CAM (pumice_rd_cmd_cam), with in-order commit. There is no monolithic txn_queue.

- `AW / AR .ready` deassert when the corresponding intake or CAM cannot accept a new request (insertion ready low).
- `W .ready` deasserts when the write-data path (CAM buffer) is full.
- `R / B .valid` follow standard protocol; consumers may stall indefinitely without controller misbehavior.

20.9 Host-Width Gearing

To use a host AXI data width different from DW, instantiate the optional wrapper `pumice_top_geared` (parameter `HOST_AXI_DATA_WIDTH`, verified at {64, 128, 256}). It inserts the repo's formally-verified `axi4_dwidth_converter_wr / axi4_dwidth_converter_rd` between the host-width slave and the DW-width core. When `HOST_AXI_DATA_WIDTH == DW`, a generate branch bypasses the converters entirely, so the build is bit-identical to `pumice_top` (no converter, no added latency). See `docs/AXI_DRAM_GEARING_SCOPE.md`.

20.10 AXI4 Reset

`aresetn` is the AXI reset (active-low). When asserted, all AXI-side state is cleared. `acclk / aresetn` clock and reset the controller/AXI domain; the DFI side uses the independent `dfi_clk / dfi_rstn` (see the DFI chapter).

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

21 DFI v2.1 Master Interface

The controller drives DFI v2.1 master signals to the PHY. DFI v2.1 is the first DFI revision that supports LPDDR2; this controller targets that revision.

21.1 Gearing and Per-Phase Packing

The gearing / phase count is set by the parameter `DFI_RATE` (default 2), **not** `N_PHASES`. Rather than the `DFI_pN` suffix convention, this controller packs the `DFI_RATE` sub-phases **flat** into each bus. All DFI pins carry the `_o/_i` direction suffix (e.g. `dfi_address_o`, `dfi_rddata_i`).

Widths (top-level pumice_top, derived from geometry):

- `dfi_address_o` — $\text{DFI_ADDR_BUS_W} = \text{ROW_WIDTH} * \text{DFI_RATE}$
- `dfi_bank_o` — $\text{clog2}(\text{NUM_BANKS}) * \text{DFI_RATE}$
- `dfi_cas_n_o` / `dfi_ras_n_o` / `dfi_we_n_o` — $1 * \text{DFI_RATE}$
- `dfi_cs_n_o` / `dfi_odt_o` — $\text{NUM_RANKS} * \text{DFI_RATE}$
- `dfi_wrdata_o` / `dfi_rddata_i` — $\text{DFI_DATA_WIDTH} = \text{DRAM_BEAT_WIDTH} * \text{DFI_RATE}$
- `dfi_wrdata_mask_o` — $\text{DFI_DATA_WIDTH}/8$
- `dfi_wrdata_en_o` / `dfi_rddata_en_o` — DFI_RATE
- `dfi_rddata_valid_i` — DFI_RATE

Per-phase slice `p` of a bus occupies `[p*W_phase +: W_phase]`.

Command placement (see `dfi_cmd_formatter.sv`): the decoded DDR2 command is placed on a target sub-phase — READ on `rd_phase`, WRITE on `wr_phase` (both CSR-driven via the `DFI_PHASE` register), and all other commands (ACT / PRE / REF / MRS) on phase 0. Non-driven phases emit NOP. This aligns R/W commands with the PHY's `rdphase` / `wrphase` contract.

21.2 Command Interface (MC → PHY)

Signal	Width	Notes
<code>dfi_address_o</code>	$\text{ROW_WIDTH} * \text{DFI_RATE}$	DDR2: row / column per phase; LPDDR2: 20-bit packed CA word on the low bits
<code>dfi_bank_o</code>	$\text{clog2}(\text{NUM_BANKS}) * \text{DFI_RATE}$	DDR2: bank; LPDDR2: held idle
<code>dfi_cas_n_o</code>	DFI_RATE	DDR2: CAS encoding; LPDDR2: held idle
<code>dfi_ras_n_o</code>	DFI_RATE	DDR2: RAS encoding; LPDDR2: held idle
<code>dfi_we_n_o</code>	DFI_RATE	DDR2: WE encoding; LPDDR2: held idle

Signal	Width	Notes
dfi_cs_n_o	NUM_RANKS*DFI_RATE	Per-rank chip select; cs_n[r]=0 selects rank r
dfi_odt_o	NUM_RANKS*DFI_RATE	On-die termination

There is no dfi_cke or dfi_reset_n port on the controller top; CKE / reset sequencing is not part of this pin bus.

21.2.1 DDR2 Command Encoding (JESD79-2)

The DDR2 truth table implemented by dfi_cmd_formatter.sv:

op	cs_n	ras_n	cas_n	we_n	bank	addr[10]	addr[..0]
NOP	0	1	1	1	X	X	X
ACT	0	0	1	1	BA	row	row
RD	0	1	0	1	BA	0	col
RDA	0	1	0	1	BA	1 (auto-PRE)	col
WR	0	1	0	0	BA	0	col
WRA	0	1	0	0	BA	1 (auto-PRE)	col
PRE	0	0	1	0	BA	0 (single bank)	X
PRE	0	0	1	0	X	1 (all banks)	X
A							
REF	0	0	0	1	X	X	X
MRS	0	0	0	0	MR idx	MR data	MR data

cs_n is per-rank: cs_n[r]=0 selects rank r; all-ones deselects (NOP). A10 (addr[10]) is the auto-precharge bit for RD/WR and the all-bank bit for PRE. MRS carries its mode-register data on the row field (cmd_row_i, ROW_WIDTH), not the column field, so the full DDR2 MR bit range is preserved.

21.2.2 LPDDR2 Command Encoding (JESD209-2F)

LPDDR2 is fully functional (reads and writes). LPDDR2 has no RAS/CAS/WE strobes: the command and a scrambled address are multiplexed onto the 10-bit CA bus over two clock edges.

dfi_cmd_formatter.sv builds the bit-exact JESD209-2F Table 60 encoding as a flat **20-bit** word (10 bits rising + 10 bits falling) placed on the low 20 bits of dfi_address_o; dfi_ras_n_o / dfi_cas_n_o / dfi_we_n_o are held idle and dfi_cs_n_o is asserted for the target rank. The full pin layout is the locked spec in docs/uarch/LPDDR2_CA_ENCODING.md, which both the RTL and the DV BFM encode against.

21.3 Write Data Interface (MC → PHY)

Signal	Width	Notes
dfi_wrdata_o	DFI_DATA_WIDTH	Write data (one DFI word = all phases)
dfi_wrdata_en_o	DFI_RATE	High during valid write-data phases
dfi_wrdata_mask_o	DFI_DATA_WIDTH/8	mask = ~wstrb (1 = mask byte)

Write data is driven by `pumice_dfi_wr_serializer`: on a WR command it waits `t_phy_wrlat` DFI cycles, then streams one DFI word per cycle from the write-data FIFO until the burst's last word (bubble-free, back-to-back bursts).

21.4 Read Data Interface (PHY ↔ MC)

Signal	Width	Direction	Notes
dfi_rddata_i	DFI_DATA_WIDTH	PHY → MC	Read data (one DFI word)
dfi_rddata_valid_i	DFI_RATE	PHY → MC	High when a read word is valid
dfi_rddata_en_o	DFI_RATE	MC → PHY	Read capture window

Read alignment is done by `pumice_dfi_rd_aligner`: on a RD command it asserts `dfi_rddata_en_o` for the burst's word count starting `t_rddata_en` DFI cycles later, and captures `dfi_rddata_i` on each `dfi_rddata_valid_i`. There is no `dfi_rddata_dnv` port.

21.5 Status / Init Interface

Signal	Width	Direction	Notes
dfi_init_start_o	1	MC → PHY	Controller requests PHY init start
dfi_init_complete_i	1	PHY → MC	High when the PHY has finished init

These are the only status handshake pins on the controller top. Update and training sub-interfaces are not present on this pin bus.

21.6 Timing Parameters Honored

The DFI datapath (`pumice_dfi_layer`) honors these CSR-driven knobs:

- `t_phy_wrlat` — delay from the WR command to `dfi_wrdata_en` (write serializer)
- `t_rddata_en` — delay from the RD command to the `dfi_rddata_en` capture window (read aligner)
- `rd_phase` / `wr_phase` — the DFI sub-phase carrying the READ / WRITE command (DFI_PHASE register)

21.7 Clock Domains and CDC

The controller/AXI side runs on `aclk`; the PHY/DFI side runs on the independent input clock `dfi_clk` (with `dfi_rstn`). The two are **not** related by a fixed frequency ratio and there is no combinational phase replication.

The single clock-domain crossing lives inside `pumice_dfi_layer` (in `pumice_dfi_cdc`) and is built from **asynchronous gaxi FIFOs** — one each for the command, write-data, and read-data streams (plus the init handshake). The internal datapath unit is one DFI word (= one DFI cycle), so one FIFO word maps to one DFI cycle and the crossing is bubble-free.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

22 CSR Register Interface (PeakRDL cpuif)

Configuration and observation registers are exposed through a **PeakRDL-generated register block** (pumice_csr), not a hand-written APB slave. The register map is authored in `rtl/macro/pumice_csr.rdl` and generated with the `passthrough cpuif` (`bin/peakrdl_generate.py`); `pumice_top` instantiates the generated `pumice_csr` and drives the core by name from `hwif_out.*` (see §6.3 for the full map). The DV mirror used for by-name access is `dv/tbclasses/pumice_regmap.py`.

The SoC-facing bus is therefore the PeakRDL **passthrough cpuif**, a simple request/ack register interface. An SoC that natively speaks APB / AXI-Lite places a thin protocol adapter in front of this `cpuif`; the controller itself does not embed one in this generation.

22.1 cpuif Signals

`CSR_ADDR_W` = 12 (4 KB, 32-bit registers). The block runs on `aclk / ~aresetn` (the controller clock/reset — see below).

Signal	Width	Direction	Notes
<code>s_cpuif_req</code>	1	input	Request valid
<code>s_cpuif_req_is_wr</code>	1	input	1 = write, 0 = read
<code>s_cpuif_addr</code>	12	input	Register byte address (4 KB space)
<code>s_cpuif_wr_data</code>	32	input	Write data
<code>s_cpuif_wr_biten</code>	32	input	Per-bit write enable
<code>s_cpuif_req_stall_wr</code>	1	output	Back-pressure a write request
<code>s_cpuif_req_stall_rd</code>	1	output	Back-pressure a read request
<code>s_cpuif_rd_ack</code>	1	output	Read data valid
<code>s_cpuif_rd_err</code>	1	output	Read error (e.g. unmapped address)
<code>s_cpuif_rd_data</code>	32	output	Read data
<code>s_cpuif_wr_ack</code>	1	output	Write accepted
<code>s_cpuif_wr_err</code>	1	output	Write error

22.2 Access Semantics

Standard PeakRDL passthrough behavior:

- Single-cycle accepts for the typical case; the `*_stall_*` outputs allow the block to hold a request when needed.
- `s_cpuif_rd_err` / `s_cpuif_wr_err` are raised for unmapped addresses; the error terminates the access and does not disturb controller state.
- Read-only fields ignore writes; write-only / self-modifying fields (e.g. `CTRL.init_start`, `CTRL.soft_reset`) pulse `swmod`.

22.3 Clock Domain

The register block is clocked by `aclk` (the controller / host-AXI clock) and reset by `~aresetn`. It is **not** on a separate management clock in this generation — there is no independent `apb_pclk`. Config fields (`hwif_out.*`, `hw = r`) are read combinationally by `pumice_top` and fanned out to the core; the single clock-domain crossing to the PHY side (`dfi_clk`) lives inside `pumice_dfi_layer` (async gaxi FIFOs), not in the register block. Observation fields (`hwif_in.*`, `hw = w`) are written from the core clock domain.

22.4 Register Map

The full address map is documented in §6.3, reconciled against `rtl/macro/pumice_csr.rdl` and `dv/tbclasses/pumice_regmap.py`.

22.5 Configuration Sequencing

Recommended bring-up order (see §6.1 for reset details):

1. Hold `aresetn` asserted.
2. Program timing parameters (`TIMINGS_*`), MR overrides, and `PHY_TIMING` (including `memtype` and `t_phy_wrlat` / `t_rddata_en`).
3. Program `ADDR_MAP` (`bank_lsb` / `hash_en` / `hash_seed`) and `DFI_PHASE`.
4. Configure PASR mask (LPDDR2 only).
5. Release `aresetn`.
6. Wait for `STATUS.init_done`.
7. Begin AXI traffic.

Runtime tuning writes (scheduler / refresh / page policy) take effect at the next configuration quiet point; the SoC owns the drain — see §5.4.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

23 Build-Time vs. Runtime Configuration

This controller distinguishes between **build-time parameters** that set the silicon footprint and **runtime CSR registers** that select behavior within that footprint. The two layers serve different purposes and should not be confused. This chapter applies the principle to every configurable knob in the design.

23.1 Principle

Layer	What it controls	Frozen at	Cost of changing
Build-time parameter	Synthesized silicon footprint: signal widths, table depths, queue sizes, instantiation counts, encoder selection, MAX bounds of runtime values	Elaboration time	Full RTL rebuild + place-and-route + tape-out (silicon) or bitstream rebuild (FPGA)
Runtime CSR register	Behavior selection <i>within</i> the synthesized footprint: policy choices, threshold values, on/off toggles, active counts up to the build-time MAX	Programmable any time the configuration quiet point is reached (typically during init or under SoC orchestration)	A single APB write

The architectural rule is straightforward: **if a knob is changing silicon area, it must be build-time. If a knob is only changing behavior of already-synthesized hardware, it must be runtime.**

23.2 Why This Distinction Matters

A controller designed without this distinction tends to collapse one of two ways:

- **Too much build-time** — every algorithmic choice is a parameter that requires a new bitstream / silicon revision to evaluate. Characterization

becomes impossibly slow; post-silicon debug becomes impossible. Most academic prototype controllers fall here.

- **Too much runtime** — every option is always synthesized, including ones that will rarely be used. Area balloons; timing closure suffers; the design becomes harder to verify because every code path is live. This is the failure mode of “ultra-configurable” commercial IP.

This controller targets the middle: **build-time picks what’s physically present; runtime picks what’s active.**

23.3 The Hybrid Pattern: “Build-Time MAX + Runtime Active”

Many of this controller’s knobs follow a hybrid pattern where the build-time parameter sets a synthesized MAX, and a paired runtime CSR field selects the actually-used value within that MAX.

Concrete example for LOOKAHEAD_DEPTH:

Build-time: LOOKAHEAD_DEPTH_MAX = 4
 → synthesizes 4 row-address comparators per bank
machine

Runtime CSR: sched_tuning.lookahead_active [3:0]
 → scheduler peeks min(LOOKAHEAD_DEPTH_MAX,
lookahead_active)
 same-bank requests per cycle

For debug: Write lookahead_active = 0 to force pure page-policy
mode
 Write lookahead_active = 4 to use the full synthesized
window
 No rebuild required.

The same pattern applies to TXN_QUEUE_DEPTH, AGE_MAX, REFRESH_DEFER_MAX, INIT_ZQ_RETRIES, ZQCS_FREQ_HZ, and others — area is paid once at synthesis for the MAX; runtime selects how much of the synthesized hardware is actively gated.

23.4 The “Build-Time + Runtime Mux” Pattern

For algorithmic choices where multiple alternatives have similar area cost, the build-time parameter selects **which** code paths are synthesized, and a runtime CSR field picks **which** of the synthesized paths is active.

Concrete example for PAGE_POLICY:

Build-time: (page policy has no build-time parameter -- the engine is always present; a mode is a CSR value, not a build)

Runtime CSR: refresh_tuning.page_policy_or [1:0]
00 = use build-time default (HAPPY_HYBRID here)
01 = force OPEN
10 = force CLOSE
11 = force HAPPY_HYBRID

For debug: Sweep policies on real workloads with no rebuilds.

The build-time choice PAGE_POLICY = OPEN would have omitted the HAPPY predictor table from the synthesized design entirely, saving area but losing the ability to switch to HAPPY at runtime. The build-time choice PAGE_POLICY = HAPPY_HYBRID synthesizes everything, paying the area cost for the predictor table, in exchange for the ability to switch among any of the three at runtime.

The same pattern applies to REFPB_POLICY, SCHEDULER_MODE, and HAPPY enable/disable.

Note that address mapping is **not** an example of this pattern. Earlier revisions exposed an ADDR_MAP_SCHEME build-time set plus an ADDR_MAP_TUNING.scheme_or runtime mux; both have been retired. Address mapping is now a single runtime placement knob — see “The ‘Runtime Placement’ Pattern” below.

23.5 The “Runtime Placement” Pattern

Some knobs neither gate synthesized alternatives nor cap a synthesized MAX — they simply reposition a field within already-present combinational logic. There is no mux and no area choice; the same decode hardware produces different behavior depending on a runtime value.

Address mapping is the canonical case. addr_mapper.sv decodes a flat AXI address into {rank, bank, row, col} from **one** runtime CSR field, ADDR_MAP.bank_lsb, which selects where the bank field sits in the byte-offset-stripped word address. The column fills below (col_lo) and above (col_hi) the bank; row/rank stack above at invariant positions. The classic “schemes” are just settings of bank_lsb:

Runtime CSR: ADDR_MAP.bank_lsb [4:0] (reset = 0x0A = COL_WIDTH = ROW_MAJOR)
= COL_WIDTH -> bank above whole column = ROW_MAJOR
= log2(DRAM_BL) -> minimal col_lo = BANK_INTERLEAVE
in between -> partial interleave

Runtime CSR: ADDR_MAP.hash_en [8], ADDR_MAP.hash_seed [23:16]

folds row bits + seed into the bank index (= XOR_HASH),
 defeating power-of-two-stride hot-banking.

There is no build-time parameter and no scheme selector: ROW_MAJOR, BANK_INTERLEAVE, and XOR_HASH are all reachable by writing bank_lsb (and optionally hash_en/hash_seed) at runtime. The address-decode logic is combinational and always present.

MEMTYPE is likewise a runtime knob in this generation, not a build-time string. The core decodes memtype_e from the CSR field PHY_TIMING.memtype (0 = DDR2, 1 = LPDDR2) — see pumice_top.sv w_memtype. There is no MEMTYPE build parameter that gates an encoder or step table; the DDR2 vs LPDDR2 behavior is selected at runtime within the synthesized datapath.

23.6 The “Runtime Only” Pattern

A small number of knobs have zero area cost and are runtime-only: simple counter compare values, timeout thresholds, retry counts. Examples include ZQCS_FREQ_HZ, INIT_ZQ_RETRIES, and the various observation-counter clear-on-read controls.

These have no build-time presence at all.

23.7 Configuration Quiet Points

Runtime CSR writes that change scheduler or refresh-manager behavior take effect at the next **configuration quiet point** — defined as: no DRAM commands in flight, no refresh sequence in progress, and the relevant FSM in its idle state. The CSR slave does not enforce quiet-point writes; the SoC firmware is responsible for sequencing. Writing during normal operation is well-defined but the change does not apply until the next quiet point. There is **no** config_apply strobe in this generation — the SoC owns the drain and must quiesce host AXI traffic (and, if needed, wait out any in-flight refresh) before the change is guaranteed to take effect.

Writes to register-file CSRs that have no FSM impact (timing parameter overrides, MR override values, observation counters) take effect immediately and have no quiet-point requirement.

23.8 Summary

The following two chapters document this controller’s complete configuration interface:

- **§5.2 Top-Level Parameter Table** — every build-time parameter, with its type, range, governing section, and (where applicable) the paired runtime CSR field.
- **§5.3 Characterization Knobs** — the subset of build-time + runtime knobs intentionally exposed for performance sweeping during silicon / FPGA characterization.

The full runtime CSR register map is in §6.3.

24 Top-Level Parameter Table

All build-time parameters for the controller, with type, default, valid range, the section that governs their use, and a one-line purpose summary.

24.1 Parameter Table

The parameters below are the actual elaboration parameters of `pumice_top` / `pumice_core` (with the host-width parameter added by `pumice_top_geared`). Several knobs that earlier revisions listed as build-time — `MEMTYPE`, `WRPHASE`, `RDPHASE`, and the address-map scheme set — are runtime CSR fields in this generation and are called out under “Runtime CSR knobs (not build parameters)” below.

Parameter	Type	Default	Range / valid set	Section	Purpose
AXI_ID_WIDTH	integer	8	1-16	§3.1	AXI ID tag width (IW).
AXI_ADDR_WIDTH	integer	32	24-40	§3.1	AXI flat address width (AW).
NUM_RANKS	integer	1	{1, 2, 4}	§3.3	Number of physical ranks (per-rank CS_n, CKE, ODT). 1 = single-rank point-to-point; 2 / 4 = DIMM-class. Rank bits stack at the top of the address map (§3.1) and refresh state is tracked per (rank, bank).
NUM_BANKS	integer	8	{4, 8}	§3.3	Per-device bank count (per rank). Sets the bank-machine / lookup count.
ROW_WIDTH	integer	14	12-16	§3.3	Row-address width.
COL_WIDTH	integer	10	9-12	§3.3	Column-address width. Also the ROW_MAJOR setting of ADDR_MAP.bank_lsb.

Parameter	Type	Default	Range / valid set	Section	Purpose
DFI_RATE	integer	2	{1, 2, 4}	§3.6	DFI frequency-ratio gear (phase count). Drives all DFI bus widths and the internal data-unit width. This is the gear parameter formerly called N_PHASES.
DRAM_BEAT_WIDTH	integer	64	{16, 32, 64}	§3.6	Width of one DRAM data beat (per DFI phase).
BL	integer	8	{4, 8}	§3.6	DRAM burst length in beats. The AXI intake splits host bursts into fixed-BL commands; one AXI burst at the core (DW) side is $(awlen+1)*DFI_RATE == BL$.
NUM_ENTRIES	integer	8	4-32 (power of 2)	§3.2	Depth of the read/write command CAMs (in-flight transaction slots). Pointer width is $\lceil \log_2(\text{NUM_ENTRIES}) \rceil$.
N_SRAM_SLOTS	integer	8	4-32 (power of 2)	§3.2	Write-data SRAM slot count (buffered write payloads awaiting commit).
BYTE_OFFSET_WIDTH	integer	3	$\lceil \log_2(\text{beat byte size}) \rceil$	§3.1	Low byte-offset bits stripped before address decode ($\log_2$ of beat byte size).
AGE_WIDTH	integer	16	8-24	§3.2	Width of the per-transaction age counter used for FR-FCFS anti-starvation tie-breaking.
HOST_AXI_DATA_WIDTH	integer	128	{64, 128, 256} (verified)	§3.1	pumice_top_geared only. Free host-side AXI data width. Formally-verified axi4_dwidth_converter_wr/_rd shims bridge the host width to the fixed core width DW. $\text{HOST_AXI_DATA_WIDTH} == \text{DW}$ is a generate-bypassed, bit-identical direct connection (no converter, no

Parameter	Type	Default	Range / valid set	Section	Purpose
-----------	------	---------	-------------------	---------	---------

added latency).

Derived parameters (computed at elaboration, not overridable independently):

- $DW = DRAM_BEAT_WIDTH * DFI_RATE$ — the fixed core AXI data width **and** the DFI word width (default $64 * 2 = 128$). $DFI_DATA_WIDTH = DW$. One core AXI beat == one DFI word == DFI_RATE DRAM beats.
- $SW = DW / 8$ — core AXI / DFI strobe width.
- $PHW = \lceil \log_2(DFI_RATE) \rceil$ (min 1) — DFI sub-phase index width.
- $DFI_STRB_WIDTH = DW/8$, $DFI_EN_WIDTH = DFI_VALID_WIDTH = DFI_RATE$, $DFI_ADDR_BUS_W = ROW_WIDTH*DFI_RATE$, $DFI_BANK_BUS_W = \lceil \log_2(NUM_BANKS) \rceil * DFI_RATE$, $DFI_CS_BUS_W = NUM_RANKS*DFI_RATE$ — the DFI 2.1 pin-bus widths, all scaled by DFI_RATE .

Runtime CSR knobs (not build parameters):

These select behavior at runtime and have no build-time parameter in this generation. Full field detail is in §6.3.

- `PHY_TIMING.memtype` (1-bit) — 0 = DDR2, 1 = LPDDR2. The core decodes `memtype_e` from this field (`pumice_top.sv w_memtype`); there is no build-time `MEMTYPE` string.
- `DFI_PHASE.wr_phase` / `DFI_PHASE.rd_phase` (3-bit each, sliced to `PHW` downstream) — which DFI sub-phase carries the WRITE / READ command. Defaults 0/0. These replace the former build-time `WRPHASE` / `RDPHASE`.
- `ADDR_MAP.bank_lsb` (5-bit, reset 0x0A = `COL_WIDTH = ROW_MAJOR`), `ADDR_MAP.hash_en` (1-bit), `ADDR_MAP.hash_seed` (8-bit) — the single address-map placement knob (+ optional bank XOR-hash). The classic `ROW_MAJOR` / `BANK_INTERLEAVE` / `XOR_HASH` “schemes” are just settings of these fields; there is no `ADDR_MAP_SCHEMES_SYNTH` / `ADDR_MAP_SCHEME_DEFAULT` build parameter and no scheme mux. See `addr_mapper.sv`.
- `REFRESH_TUNING.page_policy_or` (2-bit) — 00 = build-time default, 01 = `OPEN`, 10 = `CLOSE`, 11 = reserved (was `HYBRID` — retired; maps to build default). Adaptive paging is `PAGE_POLICY_CFG.policy_mode`.

- REFRESH_TUNING.refpb_policy_or (2-bit),
REFRESH_TUNING.refresh_defer_active (4-bit),
REFRESH_TUNING.zqcs_freq_hz (16-bit, reset 1 Hz).
- SCHED_TUNING.lookahead_active (4-bit), force_inorder (1-bit),
age_max_runtime (8-bit), txn_queue_high_water (8-bit); lookahead_max_obs
(4-bit, RO echo of the build MAX).
- PAGE_POLICY_CFG.policy_mode, PAGE_TIMEOUT_CFG.tr_init (the only live
fields after TASK-014; policy_scope, tr_min/tr_max/tr_step and the whole
PAGE_ADAPT_CFG register retired 2026-09-27) — the runtime page-policy
engine (replaced the retired PAGE_PRED_TUNING). **policy_mode RESETS TO
0**. Mode 3 (fixed_open) with tr_init = 2 is the measured best default and
is RECOMMENDED, but the reset change is BLOCKED on BUG-003 (a short
timeout precharges under an in-flight read). Measured 2026-09-26:
measured strictly dominant over open page on the board (+41.2%
col_major_interleaved_bl4, +8.6..11.6% col_major, exactly flat on
incremental/row_major, nothing regressing) (pre-sdpram-fix 2026-10-02;
off-datapath fix, reproduced 2026-10-03 — FINDINGS_litedram_ab_2026-10-
03.md). Beware two traps on these fields — tr_init = 0 DISABLES the
timeout rather than selecting a build default, and check_interval = 0
gates the whole adapt_time TR adjustment off, which is why mode 4
measured inert for every campaign before 2026-09-26. See TASK-013.
- Timing CSRs — TIMINGS_RC_RCD_RP_RAS (tRC/tRCD/tRP/tRAS),
TIMINGS_RFC_REFI (tRFC/tREFI), TIMINGS_RRD_FAW_WTR_CCD
(tRRD/tFAW/tWTR/tCCD), TIMINGS_CL_CWL_WR (CL/CWL/tWR/tRFCpb),
TIMINGS_RTP_RTW (tRTP/tRTW).
- PHY_TIMING.refresh_burst (4-bit, 1..8), PHY_TIMING.t_phy_wrlat (8-bit),
PHY_TIMING.t_rddata_en (8-bit).
- Init timings — INIT_TIMING0 (t_init_wait/t_dll_wait), INIT_TIMING1
(t_mrd_wait/t_rp_wait/t_rfc_wait), INIT_TUNING.zq_retries (4-bit, reset 3),
INIT_TUNING.init_timeout_ms (8-bit, reset 10).

These cost zero silicon area beyond the register flops; making them build-time parameters would have been wrong (and was, in earlier revisions of this document).

24.2 Memtype-Dependent Constraints

MEMTYPE is a runtime CSR field (`PHY_TIMING.memtype`) rather than an elaboration parameter, so the DDR2-vs-LPDDR2 selection does not gate elaboration-time asserts. The memtype-dependent behavior is instead enforced at run time within the synthesized datapath:

- `PHY_TIMING.memtype == 1` (LPDDR2) enables the LPDDR2-only features (PASR bank/segment masks, DPD request, temperature-derate readback, per-bank tRFC via `TIMINGS_CL_CWL_WR.tRFCpb`).
- `PHY_TIMING.memtype == 0` (DDR2) uses the DDR2 command encoding and all-bank refresh.
- The DFI address bus is `ROW_WIDTH * DFI_RATE` wide (`DFI_ADDR_BUS_W`) and carries the LPDDR2 CA packing when `memtype == 1`; it must be wide enough for whichever memtype is programmed, which is guaranteed by `ROW_WIDTH >= 14`.

24.3 Timing-Configuration Sanity Checks

The controller expects the following relationships to hold on the loaded timing CSR values. Because timings are runtime CSRs, these are boot-time programming contracts the SoC firmware must honor (they are documented here so the sweep tool and bring-up scripts can assert them):

- `tREFI_cycles >= 100` — the MC clock must be fast enough relative to tREFI that the refresh timer is well-resolved.
- `tRCD_cycles >= 2`, `tRP_cycles >= 2`, `tRC_cycles >= tRAS + tRP` — basic JEDEC consistency checks.
- `tRFC_cycles > tRP_cycles` — the refresh-cycle time must exceed the precharge time.

These are configuration bugs that should be caught at boot time before traffic starts, not preventable run-time faults.

24.4 Parameter Categories

Category	Parameters
Geometry	NUM_RANKS, NUM_BANKS, ROW_WIDTH, COL_WIDTH
Multi-rank	NUM_RANKS
Bus widths	AXI_ID_WIDTH, AXI_ADDR_WIDTH, DRAM_BEAT_WIDTH, BYTE_OFFSET_WIDTH, HOST_AXI_DATA_WIDTH; derived

Category	Parameters
	DW/DFI_DATA_WIDTH
Gear / DFI	DFI_RATE, BL (with runtime DFI_PHASE.rd_phase / DFI_PHASE.wr_phase)
Capacity	NUM_ENTRIES, N_SRAM_SLOTS, AGE_WIDTH
Characterization	Runtime CSR knobs paired with the geometry above — SCHED_TUNING.lookahead_active, REFRESH_TUNING.page_policy_or / refpb_policy_or / refresh_defer_active, PAGE_POLICY_CFG.* / PAGE_TIMEOUT_CFG.* / PAGE_ADAPT_CFG.*, ADDR_MAP.bank_lsb / hash_en (see §5.3 and §6.3)

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

25 Characterization Knobs

A subset of the build-time parameters is intentionally left configurable for **characterization sweeping**. These are the algorithmic choices that have research-backed alternatives where the best choice depends on workload. Rather than baking in a one-size-fits-all default, we expose them as parameters so the verification plan can sweep them on representative AXI traffic and pick defaults from data.

25.1 Characterization Parameters

Parameter	Choices	Recommended default	Rationale
LOOKAHEAD_DEPTH	0 / 1 / 2 / 4	1	Same-bank lookahead window for exact auto-precharge decisions when the next same-bank request is already in the queue (typical for streaming / DMA). Falls back to PAGE_POLICY when the queue is shallow. 0 disables; 2 or 4 buys diminishing returns at modest comparator cost.
PAGE_POLICY	OPEN / CLOSE / HAPPY_HYBRID	HAPPY_HYBRID (LPDDR2), OPEN (DDR2)	Fallback for when lookahead is inconclusive. Streaming workloads favor OPEN; mobile-mixed favors HAPPY_HYBRID. Ghasempour 2015 reports near-optimal accuracy with small storage.
PAGE_PREDICTOR_TABLE_BITS	8 – 16	12	4 KB predictor is sufficient for typical mobile workloads. Larger tables marginal improvement.
REFPB_POLICY	ROUND_ROBIN / OLDEST_FIRST / DARP	DARP	Chang HPCA 2014: DARP wins over round-robin on uneven bank activity (real workloads).
REFRESH_DEFER_MAX	1 – 8	8 (JEDEC ceiling)	Stuecheli MICRO 2010: deferring up to 8 maximizes Elastic Refresh benefit. Set to 1 for real-time predictability.

Parameter	Choices	Recommended default	Rationale
TXN_QUEUE_DEPTH	4 – 64	16	Larger queues help bandwidth on multi-master AXI; smaller queues reduce area.
AGE_MAX	32 – 1024	256	Caps FR-FCFS starvation; larger values let row-hits dominate longer at the cost of tail latency.
DFI_RATE	1 / 2 / 4	depends on PHY	Build-time DFI gear (phase count). 4 typical for FPGA SoCs; 2 for slower mobile platforms; 1 for simulation-only. Formerly named N_PHASES.
ADDR_MAP.bank_lsb (+ hash_en)	log2(DRAM_BL) .. COL_WIDTH, hash off / on	bank_lsb = COL_WIDTH (ROW_MAJOR), hash off	Runtime address-map placement knob (not a build parameter). bank_lsb = COL_WIDTH = ROW_MAJOR (best for streaming); lowering bank_lsb toward log2(DRAM_BL) interleaves banks (best for random); hash_en folds row bits into the bank index to defeat power-of-two-stride hot-banking (adversarial). Sweep bank_lsb across the legal range with hash off, then repeat the best point with hash on. See addr_mapper.sv. (DRAM_BL, not BL/DFI_RATE — the mapper indexes device words; pumice BUG-022, measured in dv/tests/fub/test_addr_mapper.py ::minimum_bank_lsb_is_measured.)

25.2 Sweep Plan

The verification plan (see §6.4) includes a characterization sweep that runs each parameter through its choices on a small benchmark suite:

25.2.1 Benchmark Suite

A key open question for the sweep is the **interaction between LOOKAHEAD_DEPTH and PAGE_POLICY**. With deep queues and conclusive lookahead, the choice of PAGE_POLICY barely matters because the fallback rarely fires. With shallow queues, PAGE_POLICY dominates. The sweep matrix should cover both extremes.

Workload	Pattern
stream-read	Sequential 4 KB read bursts
stream-write	Sequential 4 KB write bursts
stream-mixed	Interleaved sequential read and write
random-narrow	Random small AXI bursts (1–4 beats)
random-wide	Random large AXI bursts (16–64 beats)
mobile-mixed	Bursty traffic with periodic large writes (modeled on smartphone GPU + display)
multi-master	Two AXI masters issuing concurrent traffic with different patterns

25.2.2 Metrics Collected

For each (workload, parameter) combination, the sweep records:

- **Sustained read bandwidth** (% peak)
- **Sustained write bandwidth** (% peak)
- **Average read latency**
- **99th-percentile read latency**
- **Refresh blocking time** (% of cycles where the bus was blocked by refresh)
- **Row-hit rate** (per-bank, averaged)
- **Refresh deferral utilization** (avg refresh_pending over time)

25.2.3 Default Selection

After the sweep, each parameter's default is set to the value that:

1. Maximizes sustained bandwidth across the benchmark suite
2. Subject to: 99th-percentile latency does not regress by more than 20%
3. Subject to: area cost is within budget

The defaults shown in the parameter table above are the **expected** outcomes based on the cited research. The actual defaults will be confirmed by the sweep.

25.3 Runtime Overrides via CSR

Some characterization parameters are also accessible at runtime via CSR (see §6.3). This allows in-system tuning without rebuild:

- PAGE_POLICY runtime override (REFRESH_TUNING.page_policy_or)
- REFPB_POLICY runtime override (REFRESH_TUNING.refpb_policy_or)
- Address-map placement (ADDR_MAP.bank_lsb, hash_en, hash_seed) — this is a runtime knob outright, not an override of a build default
- Page predictor warm-up cycles and hysteresis (PAGE_PRED_TUNING.warmup_cycles / hysteresis)

Runtime overrides take effect at the next “quiet point” (no commands in flight). The SoC is responsible for ensuring no DRAM traffic during the transition.

25.4 Observability for Sweep

The CSR exposes the following observation counters (described in §6.3):

- OBS_ROW_HIT[8] — per-bank ROW HITS, free-running (subtract two reads).
- PAGE_STATS_HIT / PAGE_STATS_MISS / PAGE_STATS_EMPTY — global page-outcome counters.
- SCHED_STATS_ACT / SCHED_STATS_PRE — activations and closes.
- REF_STATS_REF / REF_STATS_REF_BUSY — refreshes, and those that fired with work pending.
- STALL_* — stall-cause attribution.

AXI latency and queue occupancy are **not** controller counters: they are measured by rtl/amba/shared/axi_bus_meter.sv and axi_perf_latency_hist.sv, which the characterisation harness instantiates directly (char_engine_block.sv:608-712). > 27 OBS_* registers were removed on 2026-09-28 (pumice BUG-020). They were declared, documented and readable, and nothing drove them – every one read zero. OBS_ROW_HIT[8] was kept and is now driven; the rest went. AXI-side latency and occupancy are measured by axi_bus_meter and axi_perf_latency_hist outside the controller.

These feed back into the sweep tool for in-loop default selection.

26 Family-Wide Config Bits

This section is the **family-wide config-bit registry** — a catalog of runtime configuration bits used across the DDR2 / LPDDR2, DDR3 / LPDDR3, and DDR4 / LPDDR4 memory controllers. Some bits apply to all generations; some are introduced in one generation and absent in earlier ones; some are *defined* across the family but ignored in flavors where the underlying mechanism does not exist. **That last case is intentional.** Allocating the bit family-wide — even when unused — keeps the CSR map mechanically compatible across controllers and lets software treat the family as a single product line.

26.1 Why a Family-Wide Registry

Each controller in the family has its own CSR map (per §6.3 of each generation’s HAS). But the field encodings, bit positions within shared registers, and the *names* are aligned across the family. The intent:

- **Software portability:** a single device driver template understands the bit layout and only needs per-flavor capability checks.
- **Bring-up portability:** bring-up software written for DDR3 should mostly work on DDR2 with the same register reads.
- **Verification portability:** the cocotb test suite’s CSR-poke helpers can be reused.
- **Debug consistency:** the same “config dump” script produces a parseable output regardless of which flavor is running.

A bit may be marked N/A in a flavor; reads return 0 and writes are ignored (no `pslverr`, just silently ignored — because software portability requires that writing a fully-portable config word does not error on flavors that don’t implement every bit).

26.2 Applicability Notation

The Apply column uses this shorthand:

- **all** — applies to every flavor in the family
- **DDR-only** — applies to DDR2/3/4 only; LPDDR variants ignore
- **LP-only** — applies to LPDDR2/3/4 only; DDR variants ignore
- **DDR3+** — applies to DDR3, DDR4, and forward; DDR2 ignores
- **DDR4+** — applies to DDR4 only (introduced for bank groups); earlier ignore
- **LPDDR3+** — LPDDR3 and LPDDR4 only
- **LPDDR4-only** — LPDDR4 only

26.3 SCHED_TUNING (Scheduling Family Bits)

Register block at the same offset (0x040) in every controller. These bits govern the FR-FCFS scheduler's runtime behavior.

Bit Field	Width	Apply	Description
lookahead_active	4	all	Runtime lookahead window (0 .. build-time MAX). 0 disables lookahead.
force_inorder	1	all	Forces FR-FCFS into first-ready FIFO mode for debug / real-time.
happy_enable	1	all	Runtime kill-switch for the HAPPY page predictor. Reads as 0 in flavors built without HAPPY.
age_max_runtime	8	all	Anti-starvation age cap; runtime override of AGE_MAX.
txn_queue_high_water	8	all	Backpressure threshold; AXI awready deasserts when queue $\geq$ threshold.
lookahead_max_obs	4	all (R only)	Echo of build-time LOOKAHEAD_DEPTH_MAX. Software discovery.
qos_high_priority	1	all	PLANNED — not in the RDL yet (roadmap Axis 1, QoS step): when 1, scheduler boosts requests with awqos/arqos $\geq$ 8.
bank_group_balance	1	DDR4+	Round-robin across bank groups in scheduler tie-break (DDR4/LPDDR4 only); N/A elsewhere.

26.4 REFRESH_TUNING (Refresh Family Bits)

Bit Field	Width	Apply	Description
refpb_policy_or	2	LP-only	REFpb selection-policy override (LPDDR2/3/4); N/A for DDR-only flavors
page_policy_or	2	all	Runtime override for the LEGACY flat page policy (01=OPEN, 10=CLOSE). Only consulted when PAGE_POLICY_CFG.policy_mode == 0; since policy_mode now RESETS to 3, this field is inert out of reset. 10=CLOSE drives the legacy

Bit Field	Width	Apply	Description
			auto-precharge path, which measures 4.9x the activations of a background precharge – see MAS ch02 “08_page_policy”.
refresh_defer_active	4	all	Active deferral count (1 .. build-time MAX). 1 = no batching.
zqcs_freq_hz	16	all	Periodic ZQCS interval. 0 = init-only.
fgr_mode	2	DDR3+	Fine-Granularity Refresh mode (DDR3 1x/2x/4x; DDR4 fixed-rate, on-the-fly). N/A in DDR2.
refresh_per_rank	1	all	Force per-rank dispatch even when NUM_RANKS=1 (debug). Reset = 1 always when multi-rank.

26.5 PAGE_POLICY_CFG / PAGE_TIMEOUT_CFG (Axis 2)

The runtime page-policy engine (pumice_page_policy). **This axis carries the largest measured runtime win on the controller**, and as of 2026-09-27 its resets SHIP that win: policy_mode = 3 (fixed_open) and tr_init = 2.

Those two must move together. tr_init = 0 disables the timeout outright, so mode 3 with the old tr_init reset of 0 would have been open page wearing another name – which is exactly the trap the field description records.

THE ADAPTIVE MODES WERE REMOVED 2026-09-27 (TASK-014). Paging modes 4 (adapt_time) and 5 (adapt_access) are gone, along with pumice_row_pred_table.sv and the whole PAGE_ADAPT_CFG register. Mode 4 had been MEASURED to be fixed_open(tr_min) – its mistake counter is dominated by the held-too-long case, so TR decayed monotonically to the floor and stayed there, landing exactly on the matching fixed point at three different floors. Mode 5 drove auto-precharge, which costs 4.9x the activations of a background precharge and double the read latency, and commits at the column op before it is known whether more same-row requests are coming – it fights the FR-FCFS reordering that justifies this design. Mode 5 was removed as a DECISION, not a measurement: it was unproven and mis-plumbed rather than disproven.

policy_mode remains a 3-bit field, so software can still WRITE 4..7. Those encodings fall through to the build default and never auto-precharge; that contract is regressed by test_page_predictor and by the scheduler matrix.

Field	Width	Reset	Notes
PAGE_POLICY_CFG. policy_mode	3	3	0=build default, 1=static_open, 2=static_close,

Field	Width	Reset	Notes
			3=fixed_open. 4..7 RETIRED and fall through to the build default (4/5 on 2026-09-27, TASK-014; 6/7 on 2026-09-26, TASK- 011). RESET IS 3 (fixed_open) since 2026-09-27: it is the measured optimum and BUG-003, which blocked the change, is fixed. Measured on the board 2026-09-27 (75 MHz, BL4 x16, 600 MB/s peak, 24 cells over four scenario families, every cell integrity-clean): +9.1% col_major (195.2 -> 212.9 MB/s), +35.1% col_major_interle aved (262.1 -> 354.1), exactly flat on incremental (561.1) and row_major (572.0), nothing regressing (pre- sdpram-fix 2026- 10-02; off-

Field	Width	Reset	Notes
			datapath fix, reproduced 2026-10-03 — FINDINGS_litedram_ab_2026-10-03.md). TR=1 and TR=2 measure identical; the cliff is between 2 and 3.
PAGE_POLICY_CFG. policy_scope	1	0	RESERVED since 2026-09-27. Was documented as “per-bank TR” but could not diverge: r_mc was one global counter driving every r_tr[b]. Retired with mode 4.
PAGE_POLICY_CFG. ctr_open_max / ctr_init	4 each	0	RESERVED since 2026-09-27. Were the mode-5 predictor’s close threshold and counter init. Bit positions held so policy_mode does not move.
PAGE_TIMEOUT_CFG .tr_init	8	2	The ONLY live field in this register. RESET IS 2 since 2026-09-27, moved together with

Field	Width	Reset	Notes
			policy_mode – tr_init=0 DISABLES the timeout, so a reset of 0 would silently neuter mode 3 and the two must change together. Idle MC cycles before the background precharge fires. 0 DISABLES the timeout – it is not a build-default sentinel on this field. TR=1 and TR=2 measure identically; TR=4 already loses the plain col_major wins.
PAGE_TIMEOUT_CFG .tr_min / tr_max / tr_step	8 each	0	RESERVED since 2026-09-27. Were the mode-4 TR clamps and step. Bit positions held so tr_init does not move.

PAGE_ADAPT_CFG (0x078) is **deleted**. The address is left a HOLE rather than reused, exactly as 0x07C was when PAGE_RBL_CFG retired: every register in this map carries an explicit absolute offset, so removing one shifts nothing, and an old host reading 0x078 gets nothing back instead of another register's meaning. Verified by diffing the generated register maps before and after – 81 - > 80 registers, **zero moved**.

STALE ROWS ELSEWHERE IN THIS FILE. `happy_enable` (SCHED_TUNING) and `zqcs_freq_hz / refresh_defer_active / refpb_policy_or` (REFRESH_TUNING) describe retired hardware: the HAPPY predictor and `page_predictor.sv` are gone, and the three REFRESH_TUNING fields were retired 2026-09-09 (refresh mode and the JEDEC postpone/pull-in credits are REF_CTRL now; no ZQCS engine ever consumed the interval). Left in place rather than silently rewritten because they predate this change and are not part of it – they need their own pass against the RDL.

26.6 ADDR_MAP (Address-Mapping Family Bits)

The address-map register at 0x04C is a single **placement knob**, not a scheme mux. The old ADDR_MAP_TUNING register (with a `scheme_or` selector and a `synth_mask_obs` synthesized-scheme bitmask) has been **retired** in this generation: the classic ROW_MAJOR / BANK_INTERLEAVE / XOR_HASH schemes are just settings of `bank_lsb` plus the optional `hash_en` fold, so there is nothing to mux (see §5.4 §3/§4 and `rtl/fub/addr_mapper.sv`).

Bit Field	Width	Apply	Description
<code>bank_lsb</code>	5	all	Bank-field LSB in the word address. = COL_WIDTH -> ROW_MAJOR; lower -> interleave. RTL clamps to [0, COL_WIDTH].
<code>hash_en</code>	1	all	Enable bank XOR-hash fold ($\text{bank} \hat{=} \text{fold}(\text{row}) \hat{=} \text{hash_seed}$) = the old XOR_HASH scheme.
<code>hash_seed</code>	8	all	XOR-hash seed. Runtime seed change without rebuild.

Family note: for DDR4+ the bank-group field is placed by extending the same `bank_lsb`-relative stack rather than by a separate `bg_field_position` selector; that generation's RDL documents the exact field split.

26.7 INIT_TUNING (Init Family Bits)

Bit Field	Width	Apply	Description
<code>zq_retries</code>	4	all	ZQ calibration retry count (DDR2 uses OCD; LPDDR2 uses MR10 ZQ).
<code>init_timeout_ms</code>	8	all	Per-step init timeout (ms; scaled by SIM_INIT_SCALE in sim).

Bit Field	Width	Apply	Description
wl_retries	4	DDR3+	Write-leveling retry count. N/A in DDR2 (no write-leveling).
mpr_enable	1	DDR3+	Multi-Purpose Register read-back during init. N/A pre-DDR3.
ca_train_enable	1	LPDDR3+	LPDDR3/4 CA-bus training. N/A in LPDDR2 (no CA training).
cbt_enable	1	LPDDR4-only	Command-Bus-Training. N/A pre-LPDDR4.

26.8 POWER_TUNING (Power-State Family Bits)

Bit Field	Width	Apply	Description
apd_idle_threshold	16	all	Cycles of bank-idleness before APD entry
srf_idle_threshold	24	all	Cycles before Self-Refresh entry
dpd_enable	1	LP-only	Deep-Power-Down enable. N/A for DDR-only flavors.
pasr_active	1	LP-only (R)	Reflects whether any rank has a non-zero PASR mask. R-only.
ckedis_after_sr	1	DDR3+	Drop CKE after SR-entry for sub-mW power floor. N/A in DDR2.
low_freq_mode	1	DDR3+	Reduced-frequency operation hint to PHY. N/A in DDR2.

26.9 ECC_TUNING (Future — Inline ECC)

This block is reserved for future inline-ECC controllers. The bits are **N/A in all current controllers** (DDR2/3/4 and LPDDR2/3/4 v1) because inline ECC is out of scope for the current generation. Future revisions of any flavor that adopt inline ECC will populate this block.

Bit Field	Width	Apply	Description
ecc_enable	1	(reserved)	Enable inline ECC. Reads as 0 in all v1 controllers.

Bit Field	Width	Apply	Description
<code>ecc_correct_mode</code>	2	(reserved)	SECDED / Chipkill / off

26.10 RANK_TUNING (Multi-Rank Family Bits)

These are present in any flavor with `NUM_RANKS > 1`. For `NUM_RANKS = 1` builds, the bits read as 0 and writes are silently ignored.

Bit Field	Width	Apply	Description
<code>rank_enable_mask</code>	4	all	Per-rank enable; clearing a bit suppresses all commands to that rank. Up to 4 ranks.
<code>odt_rule_or</code>	2	all	Runtime override for <code>ODT_RULE_MULTIRANK</code> (00 = build-time default; 01 = JEDEC_DDR2; 10 = JEDEC_LPDDR2; 11 = OFF).
<code>num_ranks_obs</code>	3	all (Read only)	Echo of build-time <code>NUM_RANKS</code> . Software discovery.
<code>cs_assert_cycles</code>	4	all	Number of cycles to hold <code>CS_n</code> asserted per command; tunable for PHY timing margin.

26.11 Quiet-Point Behavior

Runtime config bits take effect at the **next configuration quiet point** — no DRAM commands in flight, no refresh sequence in progress. The register block does not enforce quiet points automatically; SoC firmware is responsible for sequencing writes at a drain point (typically during init or an orchestrated idle window). This generation's RDL does not implement a `config_apply` strobe or a `config_settled` status bit — a future family revision may add one; today the SoC owns the drain.

The quiet-point requirement is family-wide because every flavor has hand-off state between scheduler / refresh / power-state that would be corrupted by mid-flight reconfiguration.

26.12 Bit Discovery via the ID Register

This generation does not implement a dedicated capability vector; software discovers the build via the ID register at `0xFF0` (see §6.3): `memtype`, `n_phases` (gear ratio), and `version` are readable there, and `module_id` is the fixed `0xD2` family tag. Echo fields embedded in the tuning registers (`SCHED_TUNING.lookahead_max_obs`, etc.) expose the remaining synthesized ceilings. A future family revision may add a packed capability vector along the lines below:

Bits	Field	Description
0	cap_happy	1 = HAPPY predictor synthesized
1	cap_bg_balance	1 = bank-group balance scheduler logic present (DDR4+)
2	cap_fgr	1 = Fine-Granularity Refresh supported (DDR3+)
3	cap_pasr	1 = PASR mask is implemented (LP-only)
4	cap_dpd	1 = Deep-Power-Down supported (LP-only)
5	cap_wl	1 = Write-leveling implemented (DDR3+)
6	cap_cbt	1 = CBT implemented (LPDDR4-only)
7	cap_xor_hash	1 = bank XOR-hash implemented
11:8	cap_max_ranks	Build-time NUM_RANKS (echo of geometry)
15:1	cap_n_phases	Build-time gear ratio (DFI_RATE)
2		
19:1	cap_memtype	0 = DDR2, 1 = DDR3, 2 = DDR4, 4 = LPDDR2, 5 = LPDDR3, 6 = LPDDR4
6		

26.13 Per-Flavor Applicability Matrix

Bit	DDR2	DDR3	DDR4	LPDDR2	LPDDR3	LPDDR4
lookahead_active	Y	Y	Y	Y	Y	Y
force_inorder	Y	Y	Y	Y	Y	Y
happy_enable	Y	Y	Y	Y	Y	Y
bank_group_balance	—	—	Y	—	—	Y
refpb_policy_or	—	—	—	Y	Y	Y
fgr_mode	—	Y	Y	—	—	—
bg_field_position	—	—	Y	—	—	Y
wl_retries	—	Y	Y	—	—	—
ca_train_enable	—	—	—	—	Y	Y
cbt_enable	—	—	—	—	—	Y
dpd_enable	—	—	—	Y	Y	Y
ckedis_after_sr	—	Y	Y	—	—	—
rank_enable_mask	Y	Y	Y	Y	Y	Y

Bit	DDR2	DDR3	DDR4	LPDDR2	LPDDR3	LPDDR4
odt_rule_or	Y	Y	Y	Y	Y	Y
hash_en / hash_seed	Y	Y	Y	Y	Y	Y

The “—” entries are present in the CSR map at the documented offsets but read 0 and ignore writes in that flavor. Software treats these as “soft-N/A” — not an error condition, just an absent feature.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

27 Clocks and Reset

27.1 Clock Domains

The controller uses two independent clocks. The register block is a PeakRDL passthrough `cpui f` clocked by `aclk` — there is no separate management clock.

27.1.1 `aclk`

Controller clock. All host-AXI4 logic, the scheduler, the bank timers, the write/read CAMs, and the register block (`pumice_csr`) run on `aclk`. The controller-side of the command / write-data / read-data streams into the DFI layer is on `aclk`.

Typical frequency: 100 – 200 MHz on FPGA SoCs; higher on silicon.

27.1.2 `dfi_clk`

PHY / DFI-side clock. The DFI 2.1 command bus, write serializer, and read aligner run on `dfi_clk`; the PHY consumes and drives the DFI pin bus on this clock. The gear ratio (`DFI_RATE`) sets how many DRAM phases each `dfi_clk` cycle carries.

`aclk` and `dfi_clk` are asynchronous to each other. The single crossing between them lives inside `pumice_dfi_layer` (see the CDC summary below).

27.2 Reset Signals

Both resets are active-low.

27.2.1 `aresetn` (active low, controller / AXI domain)

Controller reset, tied to the `aclk` domain. The register block resets on `~aresetn` (`pumice_csr.rst(~aresetn)`). Resets:

- Host AXI4 interface state (`pumice_axi4_layer`)
- Write-data and read-command CAMs
- Command scheduler and bank timers (`pumice_scheduler_layer`)
- Refresh manager and init sequencer
- Register block (`pumice_csr`) and observation state

27.2.2 `dfi_rstn` (active low, DFI / PHY domain)

DFI-domain reset, tied to the `dfi_clk` domain. Resets the DFI command path, the write serializer, the read aligner, and the DFI-side of the CDC FIFOs inside `pumice_dfi_layer`.

Note: `init_sequencer` names its ports `mc_clk` / `mc_rst_n` internally, but the top level (`pumice_top` → `pumice_core`) ties these to `aclk` / `aresetn`.

27.3 Reset Sequencing

27.3.1 Cold Boot

1. Both resets asserted (`aresetn`, `dfi_rstn`)
2. Clocks stable
3. `dfi_rstn` deasserted; `aresetn` deasserted (software programs the CSR timings/phases/policy via the register `cpuif`, or defaults are used)
4. The init sequencer asserts `dfi_init_start_o` and waits on `dfi_init_complete_i` (the PHY runs its own DLL-lock / IO training)
5. The init sequencer walks the JEDEC MRS / precharge / refresh sequence (see the Init Sequences chapter)
6. On `init_done_o`, the controller begins servicing AXI traffic

27.3.2 Warm Reset (Clock-Gated)

The SoC can warm-reset the controller without losing DRAM content:

1. SoC requests self-refresh entry (via `powerdown_ctrl`, optional)
2. Controller acknowledges; DRAM is in self-refresh
3. SoC asserts `aresetn`; clocks may be gated
4. SoC deasserts `aresetn`; clocks resume
5. Controller re-runs the init sequence
6. Controller resumes normal operation

27.4 Clock Domain Crossing Summary

There is exactly one clock-domain crossing in the design: the `aclk` ↔ `dfi_clk` crossing inside `pumice_dfi_layer` (`pumice_dfi_cdc.sv`). It is built from asynchronous gaxi FIFOs only — command, write-data, and read-data each get their own async FIFO, plus the `init_start` / `init_complete` handshake. The register `cpuif` shares the `aclk` domain, so there is no CSR-clock CDC.

Crossing	Mechanism	Latency
<code>aclk</code> → <code>dfi_clk</code>	Async gaxi FIFOs (<code>cmd</code> / <code>wrdata</code>) in <code>pumice_dfi_cdc</code>	FIFO + N_FL0P_CROSS-flop sync

Crossing	Mechanism	Latency
dfi_clk → aclk	Async gaxi FIFO (rddata) in pumice_dfi_cdc	FIFO + N_FLOP_CROSS-flop sync

27.5 Reset and Power Considerations

DFI v2.1 requires dfi_init_start to be asserted by the controller as part of init; the init sequencer drives dfi_init_start_o out of reset (once it leaves S_RESET) and waits on the PHY's dfi_init_complete_i before proceeding to the MRS / precharge / refresh walk. The init_start / init_complete handshake crosses the aclk ↔ dfi_clk boundary through the same CDC as the command and data streams.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

28 Init Sequences

The cold-boot init sequences for DDR2 and LPDDR2 are implemented as a single FSM in `rtl/fub/init_sequencer.sv` (not separate step-package files). The FSM issues real MRS / precharge / refresh commands to the DRAM through the command scheduler while `init_busy_o` is high, and updates the mode-register shadow in lockstep so the live CL / CWL / BL decode tracks what was programmed. The `memtype_i` input selects the DDR2 or the LPDDR2 branch.

The DDR2 branch mirrors LiteDRAM's `get_ddr2_phy_init_sequence` and is proven on the Nexys A7 board.

28.1 DDR2 Cold Init

Per JESD79-2F. FSM states from `init_sequencer.sv`; MR data rides the ROW request field (`init_cmd_row_o`), and the MR / EMR number is carried in the bank index (`init_cmd_bank_o`).

Step	State	Command	MR data	Post-wait
1	S_DFI_INIT	assert <code>dfi_init_start</code> , wait <code>dfi_init_complete</code> , then <code>tINIT</code>	—	<code>t_init_wait</code> <code>t</code>
2	S_PREA1	PRECHARGE all	—	<code>t_rp_wait</code> (<code>tRP</code>)
3	S_EMR2	MRS EMR(2)	0x0000	<code>t_mrd_wait</code> (<code>tMRD</code>)
4	S_EMR3	MRS EMR(3)	0x0000	<code>t_mrd_wait</code>
5	S_EMR1	MRS EMR(1)	0x0000	<code>t_mrd_wait</code>
6	S_MR0_DLL	MRS MR0 + DLL reset	MR0.VAL \ 0x100 (reset default 0x0533 = BL8 / CL3 / <code>tWR3</code> / DLL_RESE T)	<code>t_dll_wait</code> (<code>tDLLK</code>)
7	S_PREA2	PRECHARGE all	—	<code>t_rp_wait</code>
8	S_REF1	AUTO REFRESH	—	<code>t_rfc_wait</code> (<code>tRFC</code>)
9	S_REF2	AUTO REFRESH	—	<code>t_rfc_wait</code>

Step	State	Command	MR data	Post-wait
10	S_MR0	MRS MR0 (DLL reset cleared)	MR0.VAL (reset default 0x0433)	t_mrd_wait
11	S_OCD_DEF	MRS EMR(1) + OCD default	0x0380	t_mrd_wait
12	S_OCD_EXIT	MRS EMR(1) + OCD exit	0x0000	t_mrd_wait
13	S_DONE	assert init_done	—	—

Note the JEDEC extended-mode-register order: EMR(2), then EMR(3), then EMR(1), before MR0. The OCD-default state (S_OCD_DEF) leaves the MR1 shadow unchanged; S_OCD_EXIT restores it to the final 0x0000 value so the live decode is stable.

28.2 LPDDR2 Cold Init

Per JESD209-2F, LPDDR2 uses the Mode Register Write (MRW) chain. Because the MR address (MA) can reach MR63 / MR10 — beyond a 3-bit bank port — the sequencer packs {MA[5:0], OP[7:0]} into the ROW request field, and `dfi_cmd_formatter.sv` unpacks it (row[13:8] = MA, row[7:0] = OP) to build the bit-exact LPDDR2 CA-bus MRW word. Only MR1 / MR2 / MR3 update the CL / CWL / BL decode shadow; MR63 and MR10 are issued to the DRAM but not shadowed.

Step	State	Command	OP data	Post-wait
1	S_DFI_INIT	assert dfi_init_start, wait dfi_init_complete, then tINIT	—	t_init_wait
2	S_L_RESET	MRW(MR63) Reset	0x00	t_init_wait (tINIT4)
3	S_L_ZQ	MRW(MR10) ZQ Init Calibration	0xFF	t_dll_wait (tZQINIT)
4	S_L_MR1	MRW(MR1) BL8 / nWR3	0x23	t_mrd_wait (tMRW)
5	S_L_MR2	MRW(MR2) RL3 / WL1	0x01	t_mrd_wait
6	S_L_MR3	MRW(MR3) DS 40 ohm	0x02	t_mrd_wait
7	S_DONE	assert init_done	—	—

The LPDDR2 branch reuses the same CSR waits as DDR2: `t_init_wait` covers both `tINIT` and `tINIT4`, `t_dll_wait` covers `tZQINIT`, and `t_mrd_wait` covers the post-MRW `tMRW` delay.

LPDDR2 is now fully functional in sim: reads and writes, bit-exact JESD209-2F CA encoding, and the full MR init chain above.

28.3 Inter-Command Waits (CSR-Backed)

The JEDEC inter-command delays were previously hardcoded; they are now driven by CSR fields (zero-extended to the 16-bit internal countdown). All counts are in MC (aclk) cycles.

CSR field	Purpose	Reset value
INIT_TIMING0.t_ini_wait	CKE / tINIT settle	512
INIT_TIMING0.t_dll_wait	DLL lock (tDLLK)	256
INIT_TIMING1.t_mrd_wait	post mode-register (tMRD)	8
INIT_TIMING1.t_rp_wait	post precharge (tRP)	8
INIT_TIMING1.t_rfc_wait	post auto-refresh (tRFC)	16

Init is a one-time event, so generous margins are fine — the reset defaults comfortably cover the JEDEC minimums at the on-board clock rate. Simulation programs shorten INIT_TIMING* values to keep test runtimes practical; there is no separate scaling parameter — the shorter waits are just smaller CSR values.

28.4 Power-State Transitions

Self-refresh and deep-power-down transitions are handled by the optional `powerdown_ctrl.sv` power-state FSM, not by the init sequencer. On an idle threshold it requests a low-power state and drops CKE (`dfi_cke_o`); it wakes on any new controller activity. The two supported entry modes are:

- Precharge Power Down (PDE) — CKE low, banks may stay active.
- Self Refresh (SR) — CKE low plus the SRE command; the DRAM self-refreshes internally and the controller stops issuing REFs.

Self-refresh entry requires all banks precharged first (the scheduler enforces this precondition before granting). Self-refresh exit timing (tXSDLL for DDR2 / tXSR for LPDDR2) is enforced by the scheduler; the power-state FSM itself only drops and restores CKE.

LPDDR2 Deep Power Down (DPD) is supported as a planned power-state entry (CKE low after precharge-all). DPD exit loses DRAM content, so the full LPDDR2 cold init sequence above must run again.

29 CSR Register Map

The controller's configuration and observation registers are defined in `rtl/macro/pumice_csr.rdl` and generated with PeakRDL into a passthrough-cpuif register block (`bin/peakrdl_generate.py`). The DV mirror is `dv/tbclasses/pumice_regmap.py`; software addresses every field **by name** through that generated map — hardcoded offsets are forbidden. This chapter is the authoritative narrative for that RDL; every register, offset, and reset value below is reconciled against it.

The map occupies a 4 KB space (12-bit address, `CSR_ADDR_W = 12`). All registers are 32 bits wide, little-endian, 4-byte aligned. The register bus is the PeakRDL **passthrough cpuif** (`s_cpuif_*`), not a hand-written APB slave — see §4.3. Config fields are consumed `hw = r(hwif_out.*)` and drive the core by name; status/observation fields are `hw = w(hwif_in.*)`.

29.1 Control and Status (0x000 – 0x00F)

29.1.1 CTRL (0x000, R/W)

Bit	Field	Reset	Description
0	<code>init_start</code>	0	Write 1 to start init (self-modifying strobe)
1	<code>init_force_restart</code>	0	Write 1 to force re-init even mid-sequence
3:2	<code>RSVD_3_2</code>	0	Reserved
4	<code>pwr_req_low_power</code>	0	Request power-down state
5	<code>pwr_req_dpd</code>	0	Request DPD (LPDDR2 only)
6	<code>pwr_req_active</code>	0	Request return to ACTIVE
7	<code>pwr_req_self_refresh</code>	0	Request self-refresh
30:8	<code>RSVD_30_8</code>	0	Reserved
31	<code>soft_reset</code>	0	Write 1 to assert internal soft reset (self-clearing)

29.1.2 STATUS (0x004, R only)

Bit	Field	Description
0	<code>init_done</code>	1 when init complete
1	<code>init_error</code>	1 on init failure
7:4	<code>power_state</code>	Current power-state FSM state (encoded)

Bit	Field	Description
8	pasr_active	LPDDR2: PASR mask is non-zero
23:16	init_step_dbg	Current init step number (for bring-up)
6		
31	version_match	1 when build matches expected version

29.1.3 STATUS_HISTORY (0x008, R only)

Last 8 power-state transitions, 4 bits each. Most recent in [3:0]; new transitions push the oldest off. Useful for bring-up debugging of power-state oscillation.

29.2 Timing Parameters (0x010 – 0x01F)

Packed timing parameters in MC (aclk) cycles. All fields are sw = rw, hw = r and drive the core scheduler / DFI layer.

29.2.1 TIMINGS_RC_RCD_RP_RAS (0x010, R/W)

Bits	Field	Reset	Description
7:0	tRC	60	tRC in cycles
15:8	tRCD	15	tRCD in cycles
23:16	tRP	15	tRP in cycles
31:24	tRAS	40	tRAS in cycles

29.2.2 TIMINGS_RFC_REFI (0x014, R/W)

Bits	Field	Reset	Description
15:0	tRFC	16	tRFC (or tRFCab) in MC cycles — mission-mode REF recovery, consumed by the arbiter
31:16	tREFI	1950	tREFI in cycles

29.2.3 TIMINGS_RRD_FAW_WTR_CCD (0x018, R/W)

Bits	Field	Reset	Description
7:0	tRRD	6	tRRD, in MC cycles to block (spacing enforced is N+1)
15:8	tFAW	35	tFAW, in MC cycles
23:16	tWTR	4	tWTR, in MC cycles to block (spacing

Bits	Field	Reset	Description
31:24	tCCD	4	enforced is N+1) tCCD, in MC cycles to block (spacing enforced is N+1)

Every JEDEC timing field in this map is a count of MC cycles to BLOCK, so the command spacing the DRAM sees is N+1. Proved and bounded tight in formal/pumice/{bank_timer,global_timers}; see pumice_mas/ch04_apb_config/03_runtime_overrides.md for what that means when programming a part from its datasheet, and pumice TASK-034 for the open question of recovering the cycle.

29.2.4 TIMINGS_CL_CWL_WR (0x01C, R/W)

Bits	Field	Reset	Description
7:0	CL	6	CAS latency
15:8	CWL	4	CAS write latency
23:16	tWR	15	Write recovery
31:24	tRFCpb	70	LPDDR2 per-bank tRFC

29.3 Mode Register Values (0x020 – 0x02F)

29.3.1 MR0, MR1, MR2, MR3 (0x020, 0x024, 0x028, 0x02C, R/W)

Low 16 bits (VAL) contain the MR value; upper 16 bits reserved. These hold override MR values; the default init walk uses the JEDEC-standard values built into `init_sequencer.sv` (§6.2). Reset = 0.

29.4 LPDDR2-Specific (0x030 – 0x03F)

29.4.1 PASR_BANK_MASK_RANK0 (0x030, R/W)

Bits	Field	Reset	Description
7:0	pasr_bank	0	LPDDR2 PASR per-bank mask for rank 0 (MR16); bit N = 1 masks bank N

29.4.2 PASR_SEG_MASK_RANK0 (0x034, R/W)

Bits	Field	Reset	Description
7:0	pasr_segs	0	LPDDR2 PASR segment mask for rank 0

29.4.3 TEMP_DERATE_RANK0 (0x038, R only)

Bits	Field	Description
1:0	temp_class	From rank 0's MR4: 00 = nominal, 01 = 2x refresh, 10 = 4x refresh

The current RDL defines the single-rank instances only (NUM_BANKS = 8, rank 0). Multi-rank builds would extend this block; the map as generated today does not allocate per-rank PASR/temperature registers beyond rank 0.

29.5 Scheduler / Refresh / Page / Addr / Init Tuning (0x040 – 0x05F)

29.5.1 SCHED_TUNING (0x040, R/W)

Bits	Field	Reset	Description
3:0	lookahead_active	0	Active lookahead window (0..LOOKAHEAD_DEPTH_MAX). 0 disables lookahead.
4	force_inorder	0	1 = force first-ready FIFO (disable row-hit reordering).
5	happy_enable	1	1 = HAPPY predictor active (only meaningful if synthesized).
15:8	age_max_runtime	0	Runtime AGE_MAX override (0 = use build-time default).
23:16	txn_queue_high_water	0	Backpressure-assertion threshold for the txn queue.
27:24	lookahead_max_obs	—	(R only, hw = w) Echo of build-time LOOKAHEAD_DEPTH_MAX.

29.5.2 PAGE_PRED_TUNING (0x044, R/W)

Bits	Field	Reset	Description
15:0	warmup_cycles	1024	Warmup cycles before predictor is trusted
23:16	hysteresis	2	Saturating-counter hysteresis

29.5.3 REFRESH_TUNING (0x048, R/W)

Bits	Field	Reset	Description
1:0	refpb_policy_or	0	00 = build-time; 01 = RR; 10 = OLDEST_FIRST; 11 = DARP.
3:2	page_policy_or	0	00 = build-time; 01 = OPEN; 10 = CLOSE; 11 = HAPPY_HYBRID.
7:4	refresh_defer_active	1	Active refresh deferral count (1..REFRESH_DEFER_MAX). 1 = no batching.
31:16	zqcs_freq_hz	1	Periodic ZQCS interval in Hz. 0 disables periodic ZQCS.

29.5.4 ADDR_MAP (0x04C, R/W)

Replaces the retired ADDR_MAP_TUNING register. The AXI-to-DRAM mapping is driven by a single knob rather than a scheme selector — see §5.4 and §3.1, and `rtl/fub/addr_mapper.sv`.

Bits	Field	Reset	Description
4:0	bank_lsb	0x0A	Bank-field LSB in the word address. Default 10 = COL_WIDTH = ROW_MAJOR placement. RTL clamps to [0, COL_WIDTH].
8	hash_en	0	Enable bank XOR-hash: $\text{bank} \wedge = \text{fold}(\text{row}) \wedge \text{hash_seed}$.
23:16	hash_seed	0	XOR-hash seed (<code>seed[BW-1:0]</code>).

The register reset value is 0x0000000A (`bank_lsb = 10`, hash off). Setting `bank_lsb = COL_WIDTH` gives ROW_MAJOR; `bank_lsb = log2(DRAM_BL)` gives maximum bank interleave with burst locality preserved; `hash_en` folds an XOR-hash on top of any placement (the old XOR_HASH scheme). There is no separate scheme-selector field. (`DRAM_BL`, not `cols/burst` — the mapper indexes device words; pumice BUG-022.)

29.5.5 INIT_TUNING (0x050, R/W)

Bits	Field	Reset	Description
3:0	zq_retries	3	ZQ calibration retry count before raising <code>init_error</code> . Range 1–8.
15:8	init_timeout_ms	10	Per-step init timeout in ms. Range 1–255.

29.5.6 INIT_TIMING0 (0x058, R/W)

JEDEC init-sequence waits (MC cycles), consumed by `init_sequencer.sv`. Previously hardcoded.

Bits	Field	Reset	Description
15:0	t_init_wait	512	CKE / tINIT settle
31:16	t_dll_wait	256	DLL lock (tDLLK)

29.5.7 INIT_TIMING1 (0x05C, R/W)

Bits	Field	Reset	Description
7:0	t_mrd_wait	8	tMRD (post mode-reg)
15:8	t_rp_wait	8	tRP (post precharge)
23:16	t_rfc_wait	16	tRFC (post refresh)

29.5.8 TIMINGS_RTP_RTW (0x054, R/W)

Read-to-precharge and read-to-write turn-around. Previously tRTP was hardcoded (8'd4) and tRTW was tied to it; both are now independent configs.

Bits	Field	Reset	Description
7:0	tRTP	4	Read to precharge
15:8	tRTW	6	Read to write

29.6 DFI / PHY Timing (0x060 – 0x067)

29.6.1 DFI_PHASE (0x060, R/W)

Which DFI sub-phase carries the READ vs WRITE command, to match the PHY's rdphase / wrphase contract (see §4.2). Fields are sliced to $\text{clog}_2(\text{DFI_RATE})$ bits downstream; upper bits are ignored when DFI_RATE is narrower than the field. Defaults 0/0 preserve the legacy all-on-phase-0 behavior.

Bits	Field	Reset	Description
2:0	rd_phase	0	READ command DFI sub-phase
6:4	wr_phase	0	WRITE command DFI sub-phase

29.6.2 PHY_TIMING (0x064, R/W)

PHY / DFI data timing plus memory-type selection. All fields hw = r so they drive the controller core.

Bits	Field	Reset	Description
7:0	t_phy_wrlat	0	WRITE command -> dfi_wrdata_en (Nexys A7 bring-up tuple programs 1)
15:8	t_rddata_en	6	RD command -> dfi_rddata_en window
16	memtype	0	0 = DDR2, 1 = LPDDR2 (drives the whole core memtype_e path)
23:20	refresh_burst	1	REFs drained per request (1..8)

PHY_TIMING.memtype is the runtime memory-type select. It is decoded into the memtype_e enum at the top and fanned out to the addr/scheduler/DFI/init logic; geometry (DFI_RATE, DRAM_BEAT_WIDTH, NUM_BANKS, ROW/COL width, BL) remains build-time — see §5.1.

29.7 Per-Bank Observation (0x080 – 0x09F)

Single-rank (rank 0), NUM_BANKS = 8. Each is a regfile of 8 word-wide registers striding by 4 bytes.

29.7.1 OBS_ROW_HIT[0..7] (0x080 + N×4, R only)

Per-bank ROW HITS: column ops issued to a bank whose row was **already open**. VAL[31:0], hw = w, driven by pumice_page_policy from the issued command stream. Free-running — they clear only on aresetn, so read two and subtract for a window, as with PAGE_STATS_*.

These are the sound row-hit count. Do **not** re-derive hits as PAGE_STATS_HIT - SCHED_STATS_ACT: that goes negative under a background-close mode (ISSUE-014).

Until 2026-09-28 these were sw = rw with onread = rclr, which could never have worked — the field is hw = w and driven every cycle, so the hardware rewrites whatever a read clears.

29.8 Calibration / Training (0x0A0 – 0x0FF)

LPDDR2 ZQ calibration and MRR DQ calibration, driven and harvested by pumice_training_layer (Ch 3.9). Both FUBs are inert for DDR2 builds, so every register in this block reads its reset (or holds zero) when memtype == DDR2.

29.8.1 CAL_CTRL (0x0A0, R/W)

Bits	Field	Reset	Description
0	zq_en	0	Enable periodic ZQCS/ZQCL
1	zq_defer_en	0	Mode-C deferral: hold ZQ under demand (see Ch 3.9 — the ZQ FUB's demand_i is tied off in the current integration, so this is dormant until the sideband lands)
3:2	RSVD_3_2	0	Reserved
4	cal_start	0	Write 1 to start a DQ-cal sequence (self-clearing strobe)
5	cal_abort	0	Write 1 to abort the current DQ-cal sequence
15:6	RSVD_15_6	0	Reserved
28:16	zq_overdue_max	0	Max ZQ deferral cycles before overdue (0 = no cap)
31:29	RSVD_31_29	0	Reserved

29.8.2 CAL_ZQ_INTERVAL (0x0A4, R/W)

VAL[31:0], reset 0. ZQCS interval in MC cycles; 0 disables periodic ZQ.

29.8.3 CAL_ZQ_TIMING (0x0A8, R/W)

Bits	Field	Reset	Description
15:0	t_zqcs	0	Post-grant bus-quiet hold for ZQCS (MC cycles)
31:16	t_zqcl	0	Post-grant bus-quiet hold for ZQCL (MC cycles)

29.8.4 CAL_TRAIN_TIMING (0x0AC, R/W)

Bits	Field	Reset	Description
15:0	t_mrr	0x2	MRR-to-MRR spacing, MC cycles (JEDEC tMRR = 2)
31:16	t_readout	0	MRR issue → captured-data timeout

29.8.5 CAL_MRR32_DATA[0..3] (0x0B0 + N×4, R only)

First captured DFI read-data beat for MRR MR32 (DQ-cal pattern A). Four 32-bit slices of the full DFI read width (128 bits at the default geometry); hw = w, harvested from pumice_lp_cal. Held until the next cal_start run captures a new beat.

29.8.6 CAL_MRR40_DATA[0..3] (0x0E0 + N×4, R only)

First captured DFI read-data beat for MRR MR40 (DQ-cal pattern B). Same layout as CAL_MRR32_DATA. The array starts at 0x0E0 because the retired 0x0C0–0x0DC hole below is never reused.

29.8.7 CAL_STATUS (0x0F0, R only)

Bits	Field	Description
0	zq_busy	ZQ calibration busy (post-grant hold window)
1	cal_busy	DQ calibration busy (sequence in progress)
2	cal_done	DQ calibration done — sticky, cleared only by soft reset
3	cal_err	DQ calibration readout timeout — sticky, cleared by soft reset
4	zq_overdue	ZQ interval expired without grant (see the demand_i note above)
15:5	RSVD_15_5	Reserved
31:1	zqcs_total	ZQCS/ZQCL commands issued since reset

6

29.9 Retired observation registers (0x0C0 – 0x1E0)

Twenty-seven OBS_* registers were removed on 2026-09-28. They were in the register map, in this book and in the generated docs, and **nothing drove them**: pumice_top carried no hwif_in.OBS_* assignment, so every one read zero forever. A 99.4%-row-hit board workload reporting zero hits is what finally surfaced it (pumice BUG-020). Their addresses are left as holes so the rest of the map does not shift; the calibration registers above were allocated around them (0x0B0–0x0BC before the hole, 0x0E0 / 0x0F0 after it), never through it.

Range	Was	Why it went
0x0C0..0x0DC	OBS_REF_LATE NCY[8]	never driven, no consumer. Refresh remains observable via REF_STATS_REF / REF_STATS_REF_BUSY, which are driven
0x100..0x104	OBS_TXN_QUEUE_DEPTH_{MAX,AVG}	AXI-side occupancy; belongs to an external meter
0x108..0x118	OBS_REFRESH_PENDING_MAX, OBS_REFRESH_DEFER_HIST_0 ..3	never driven, no consumer
0x120	OBS_PAGE_PRED_ACCURACY	retired earlier with the HAPPY predictor
0x130..0x138	OBS_AXI_{R,W}_LATENCY_*	AXI latency is a property of the interface and is measured by axi_bus_meter + axi_perf_latency_hist, which the characterisation harness already instantiates. Measuring it from inside the controller is the wrong side of the interface
0x1C0..0x1E0	OBS_WORDS[9]	the aggregator that packed them was deleted in e8908eebf (2026-07-21, “retire three dead code trees”); the window outlived its source by two months and nobody noticed, because nothing read it

OBS_ROW_HIT[8] at 0x080 was **kept and wired**: it is the only sound row-hit count, and ISSUE-014 exists because the arithmetic used in its absence is not.

29.10 Module Identification (0xFF0 – 0xFFC)

29.10.1 ID (0xFF0, R only, reset = 0xD2020001)

Bits	Field	Reset	Description
7:0	version	0x01	Build version
15:8	memtype	0x00	0 = DDR2, 1 = LPDDR2
23:16	n_phases	0x02	Gear ratio (1, 2, or 4)
31:24	module_id	0xD2	Fixed 0xD2

29.10.2 BUILD (0xFF4, R only)

Build hash word. VAL[31:0], reset = 0.

29.11 Notes on This Revision

- ADDR_MAP_TUNING (with scheme_or / synth_mask_obs) has been **retired** and replaced by ADDR_MAP at 0x04C. No scheme-selector field exists in RTL.
- TIMINGS_RTP_RTW (0x054), INIT_TIMING0 (0x058), INIT_TIMING1 (0x05C), DFI_PHASE (0x060), and PHY_TIMING (0x064) expose knobs that were formerly hardcoded in the FUBs; they are now runtime CSRs.
- PHY_TIMING.memtype and PHY_TIMING.refresh_burst are new runtime fields.
- The observation block is single-rank as generated; the multi-rank per-rank layout described in earlier revisions is not present in the current RDL.

RTL Design Sherpa · Learning Hardware Design Through Practice [GitHub](#) · [Documentation Index](#) · [MIT License](#)

30 Verification Strategy

A layered verification approach with the existing DFI BFM as the primary reference.

30.1 Layered Approach

30.1.1 Layer 1: Module-Level Unit Tests

Each live FUB gets its own cocotb unit test (see `dv/tests/fub/`):

Module	Test focus
<code>pumice_wr_intake</code>	AW-meta + wr-data FIFO handshakes; backpressure; burst splitting into fixed-BL commands
<code>pumice_rd_intake</code>	AR channel handshakes; address mapping; snarf (read-your-write) probe
<code>pumice_wr_data_cam</code>	WR CAM fill / commit-drain / snarf movers; <code>r_fdone</code> fill-complete gating
<code>pumice_rd_cmd_cam</code>	RD CAM return-fill / drain movers; oldest-first, data-ready gating
<code>addr_mapper</code>	Address translation across <code>ADDR_MAP.bank_lsb</code> settings (<code>ROW_MAJOR</code> .. <code>interleave</code>) and hash on/off
<code>pumice_cmd_arbiter</code>	Single-pick priority order; inline open-page decision; refresh prioritization
<code>pumice_bank_timers</code> <code>/bank_timer</code>	Per-(rank,bank) preset/decrement timers; combinational <code>safe_act</code> / <code>rd</code> / <code>wr</code> / <code>pre</code> readiness
<code>global_timers</code>	<code>tFAW</code> / <code>tRRD</code> / <code>tWTR</code> / <code>tRTW</code> / <code>tCCD</code> against JESD79-2 / JESD209-2 truth tables
<code>refresh_ctrl</code>	<code>tREFI</code> counter; refresh-deferral; refresh request/ack handshake
<code>init_sequencer</code>	Step-by-step DDR2 and LPDDR2 JEDEC MR init sequences
<code>mode_register</code>	CL / CWL / BL / AL decode (DDR2 and LPDDR2 MR2 RL/WL enum)
<code>dfi_cmd_formatter</code>	Round-trip table tests against JEDEC encoding; LPDDR2 CA-bus branch
<code>dfi_signal_pack</code>	DW-to-phase split; idle-phase handling

Module	Test focus
pumice_dfi_cdc	The single async-FIFO clock-domain crossing; bubble-free cmd/wr/rd flow
pumice_dfi_wr_serializer	CWL alignment; write-data serialization onto DFI phases
pumice_dfi_rd_aligner	CL alignment; read-data assembly; back-pressure

The four macro wrappers and the integration levels are exercised at Layer 2/3: pumice_axi4_layer, pumice_scheduler_layer, pumice_dfi_layer, pumice_core, and pumice_top (the PeakRDL pumice_csr register block is verified with the top).

30.1.2 Layer 2: Subsystem Tests

Multi-module integration tests, framed around the three macro layers (see dv/tests/macro/):

- **AXI interface** (pumice_axi4_layer = wr/rd intakes + the two CAMs): full AXI transaction flow with diverse traffic, including read-your-write snarf forwarding
- **Command scheduler** (pumice_scheduler_layer = cmd arbiter + bank/global timers + refresh + init + mode register): scheduler decisions on representative queue contents, inline open-page reordering, refresh interleave
- **DFI layer** (pumice_dfi_layer = single CDC + cmd path + wr serializer + rd aligner): command/data flow across the clock-domain crossing with the DFI BFM slave

30.1.3 Layer 3: End-to-End with DFI BFM Slave

The full controller drives the DFI BFM slave through our existing co-sim harness (tests/sim/dfi/test_litedram/... patterns in the DV repo).

- AXI4 stimulus generators issue representative traffic
- DFI BFM slave captures the controller's command stream
- Assertions check: command sequencing matches AXI, timing constraints are honored, data integrity round-trips through the slave's MemoryModel

30.1.4 Layer 4: External-Reference Cross-Validation

For DDR2: cross-validate against LiteDRAM's DDR2 model.

- Generate LiteDRAM controller alongside ours

- Both drive the same DFI BFM slave with the same AXI stimulus
- Compare command streams; differences flag potential design issues

This is a real, proven path. A LiteDRAM DDR2 memtest passes on the Nexys A7 board (128 MiB), which confirmed the board, PHY, pin-out, and DRAM are good. pumice itself is now board-proven on the same Nexys A7: DDR2 reads and writes are clean (0 mismatches across the characterization traffic). Both memtypes pass the full sim suite.

LPDDR2 has no LiteDRAM analog, so cross-validation for LPDDR2 is limited to the BFM master to BFM slave path. LPDDR2 is nonetheless fully functional in sim (reads and writes, bit-exact JESD209-2F CA encoding, full MR init) and passes the sim suite.

30.2 Characterization Sweep

Post-functional verification, the sweep runs each characterization parameter through its choices on the benchmark suite from §5.2:

30.2.1 Sweep Matrix

Parameter	Sweep values
LOOKAHEAD_DEPTH	0, 1, 2, 4
PAGE_POLICY	OPEN, CLOSE, HAPPY_HYBRID
REFPB_POLICY	ROUND_ROBIN, OLDEST_FIRST, DARP
REFRESH_DEFER_MAX	1, 2, 4, 8
TXN_QUEUE_DEPTH	8, 16, 32
DFI_RATE	(build-determined; usually 4)
ADDR_MAP.bank_lsb	ROW_MAJOR (== COL_WIDTH), interleave, hash on/off

The full cross-product would be $4 \times 3 \times 3 \times 4 \times 3 = 432$ builds; in practice we sweep one parameter at a time holding others at recommended defaults, which reduces to ~17 builds per memtype. The exception is the LOOKAHEAD_DEPTH $\times$ PAGE_POLICY pair — these interact strongly (the fallback policy only matters when lookahead is inconclusive), so this pair is swept as a full $4 \times 3 = 12$ -point grid.

30.2.2 Sweep Outputs

Each (parameter, value) run produces a JSON report:

```
{
  "memtype": "LPDDR2",
  "page_policy": "HAPPY_HYBRID",
  "refpb_policy": "DARP",
```

```

"workload": "mobile-mixed",
"metrics": {
  "read_bw_pct": 73.2,
  "write_bw_pct": 68.1,
  "read_latency_avg_ns": 142,
  "read_latency_p99_ns": 198,
  "refresh_block_pct": 4.3,
  "row_hit_rate_avg": 0.81,
  "page_predictor_accuracy": 0.91
}
}

```

These feed into a sweep visualization tool that selects defaults from data.

30.2.3 Out-of-Order Scheduler Trade-Offs

The centralized FR-FCFS scheduler proposed in §1 (Differentiating Features) is the most architecturally aggressive choice in this controller and warrants explicit characterization beyond just the sweep matrix above. Three concrete concerns:

- **Timing-critical path scales with TXN_QUEUE_DEPTH.** Scanning the full queue every cycle for the best candidate is a comparator pyramid: priority resolves over (ready-flag, row-hit-flag, age) for every entry simultaneously. At TXN_QUEUE_DEPTH = 16 this is well within an embedded-SoC clock budget; at 32 it stays comfortable; at 64 it becomes the critical path on the controller side. The characterization sweep should report scheduler combinational depth at synthesis time alongside the performance numbers — the right TXN_QUEUE_DEPTH is the largest value that doesn't push the scheduler off the clock budget.
- **Area cost is super-linear in queue depth.** Each queue entry needs its full metadata replicated (bank, row, col, ID, age, row-hit cache). At depth 16, ~2K gates; at depth 64, ~10K gates — comparable to the rest of the scheduler combined. The HAPPY predictor's gate cost is a separate line item.
- **Verification cost is significantly higher than FIFO.** OoO issue means assertions like "request N landed at cycle X" can't be used. The DV plan relies on protocol-level scoreboarding: the BFM master records arrival order per ID, the BFM slave records issue order, and the scoreboard verifies the AXI4 per-ID ordering invariant is preserved. Within-ID ordering violations are a higher-severity bug than performance regressions and should be caught at Layer 1 unit tests, not deferred to integration.

- **Fairness tail matters under adversarial workloads.** With AGE_MAX as the only anti-starvation cap, pathological patterns can drive 99th-percentile latency well above mean. The random-narrow and multi-master workloads in the benchmark suite intentionally exercise this; the mobile-mixed workload simulates the common case. The recommended default for AGE_MAX should be picked based on tail-latency targets, not throughput targets.

These concerns are unique to the OoO scheduler design choice — a simpler per-bank FIFO controller (the typical open-source pattern) would not face them, at the cost of leaving row-hit reordering performance on the table.

30.3 Verification Hooks

Internal observation outputs are exposed via CSR (see §6.3) for in-system characterization. Additionally, the controller emits internal “event” signals that can be tied to waveform-dump triggers:

- ev_init_done
- ev_refresh_issued
- ev_page_conflict_detected
- ev_queue_full
- ev_refresh_pending_critical

Specific events can be wired to a test-bench-controlled waveform-capture trigger during bring-up.

30.4 Coverage Targets

Functional coverage (collected during regression):

- All FSM states reached in each module that has one (the bank timers and CAMs are FSM-free; their timer/counter states are covered instead)
- All page policies exercised
- All refresh policies exercised
- All scheduler priority levels exercised
- All AXI burst types exercised (if enabled)
- All ADDR_MAP.bank_lsb settings (ROW_MAJOR .. interleave) exercised, with hash on and off

Code coverage targets: 100% line / branch coverage for all modules.

Assertion coverage: all design assertions (registered protocol checks) hit during regression.

31 Open Issues and TODOs

Items where design choices were made provisionally and should be reviewed before the HAS is promoted to formal status.

31.1 Open Issues

31.1.1 1. AXI4 Narrow-Burst Handling Confirmation

AXI4 allows narrow transfers (data width less than bus width). The current plan is to handle them via byte-mask through `wrdata_mask`. **Confirm** the `wstrb-to-wrdata-mask` wiring honors the AXI partial-write semantics correctly, particularly for unaligned bursts.

31.1.22. AXI4 Read-Modify-Write Strobe Handling

Partial writes in DDR2 / LPDDR2 use `wrdata_mask` to mask byte lanes; this is straightforward. **Confirm** that per-byte masking is wired all the way through the write path (`pumice_wr_intake -> pumice_wr_data_cam -> pumice_dfi_wr_serializer`) and that `pumice_wr_intake` doesn't widen narrow writes implicitly.

31.1.33. CSR Address Space Allocation

The config bus is a PeakRDL passthrough `cpuif` on `aclk` (no APB slave, no separate `apb_pclk`). `CSR_ADDR_W = 12` provides a 4 KB register region. **Confirm** the SoC's address-map allocation has at least 4 KB available for the `cpuif` window, and that there's no conflict with adjacent peripherals.

31.1.44. Per-Bank Refresh Book-Keeping Cost

Keeping `last_ref_age` per bank costs one counter per bank. For 8 banks this is 8 small counters. **Confirm** the synthesis area impact is acceptable; we expect ~200 LUTs for the per-bank refresh tracking on a typical FPGA.

31.1.55. HAPPY Predictor Hash Function

RESOLVED (2026-08-25): the HAPPY address-hash predictor is retired — it was never wired into the rearchitected core. Its Ghasempour-2015 successors landed as runtime modes of `pumice_page_policy` (`adapt_time` and `adapt_access`) and have since been retired themselves (2026-09-27, TASK-014): `adapt_time` measured as `fixed_open(tr_min)`, and `adapt_access` drove auto-precharge, which costs 4.9x the activations of a background precharge. The surviving mechanism is `fixed_open`'s background close, which needs no predictor and no address hash.

31.1.66. Self-Refresh Exit Latency

JESD79-2 requires `tXSNR` (~200 ns) before any command after SR exit. **Confirm** the optional `powerdown_ctrl` enforces this correctly and that the scheduler is gated for the full duration.

31.1.77. Init Sequencer Retry on Error

If ZQ fails, do we retry up to 3 times then halt, or just halt? Current plan: 3 retries (parameter `INIT_ZQ_RETRIES`) then raise `init_error`. **Confirm** with the system architect — some systems prefer fail-fast. (Note: DDR2 ZQ init differs from LPDDR2 — `init_sequencer` handles both JEDEC sequences.)

31.1.88. CSR Write Atomicity

Multi-register parameter changes (e.g., updating both `MR0` and a related timing) need a “config quiet period” to take effect atomically. Today the SoC owns the drain: software is expected to quiesce AXI traffic before reprogramming timing/mode CSRs. A dedicated `config_apply` strobe (hardware-enforced quiet period) is **not implemented** and is a possible future addition.

Document the software drain protocol regardless.

31.1.99. Burst Splitting at Row Boundaries

AXI4 allows up to 256 beats per burst; a DRAM row contains $2^{\text{COL_WIDTH}}$ columns. Burst splitting into fixed-BL commands happens in `pumice_wr_intake / pumice_rd_intake`. **Confirm** the split logic correctly tracks partial-burst completion for ID-based ordering.

31.1.10 10. Multi-Rank

Multi-rank scaffolding now exists: `NUM_RANKS` is a real build parameter (default 1), the DFI `dfi_cs_n_o / dfi_odt_o` buses are per-rank (width `NUM_RANKS * DFI_RATE`), and `addr_mapper` carries a rank field (`rank_o`) at the top of the address stack. The command path and bank timers are stamped per (rank, bank). **Remaining gap:** per-rank refresh coordination, PASR (partial-array self-refresh), and per-rank observation registers are not yet implemented. **Confirm** the multi-rank data path with a `NUM_RANKS > 1` build before relying on it.

31.1.11 11. ECC

Out of scope for this controller. ECC is expected to be handled at the SoC level or in a sideband wrapper, not the controller itself.

31.1.12 12. DFI Training Sub-Interface

Not driven in v1; the assumption is that the PHY handles its own training or training is not required for this DDR2 / LPDDR2 target. **Confirm** with the PHY vendor before silicon tape-out.

31.1.13 13. AXI Quality of Service (QoS)

Currently the `awqos / arqos` fields are forwarded to the scheduler as priority hints, but the scheduler does not implement explicit QoS classes. **Decide** whether to add proper QoS support (multi-class scheduler) or leave QoS as a future enhancement.

31.1.14 14. Multi-Master AXI

Currently single AXI port. **Decide** whether to add a multi-port AXI crossbar internally (simplifies SoC integration but adds area) or leave as a single-port with the SoC responsible for AXI arbitration upstream.

31.1.15 15. Observation Counter Overflow

Observation counters (row-hit, queue depth max, etc.) are 32-bit. At high traffic rates they could wrap. **Decide** on overflow handling: saturate, clear-on-read, or rely on SoC reading frequently.

31.1.16 16. Retired-Architecture Modules Still on Disk (OLD/)

The pre-rearchitecture modules still live under `rtl/fub/OLD/`, `rtl/macro/OLD/`, and `rtl/top/OLD/` (e.g. `scheduler.sv`, `wr_cmd_cam.sv`, `rd_cmd_cam.sv`, `xbank_timers.sv`, `axi_intake.sv`, the `*_macro.sv` blocks, `pumice_csr_slave.sv`). They are referenced only by retired sentinel tests in `dv/tests/` (`test_scheduler.py`, `test_wr_cmd_cam.py`, `test_xbank_timers.py`, `test_axi_intake.py`, and the `*_macro` tests). **Remove** the OLD/ trees and their sentinel tests once the new architecture is fully signed off, so the live module set is the only thing that builds.

Status (2026-07-22): done — the OLD/ trees and their sentinel tests have been removed; only the rearchitected module set remains under `rtl/`.

31.1.17 17. Open-Page Read-Fetches-Zero Under Gapped Reads

A known scheduler/CAM interaction: under open-page policy with gapped read traffic, a read can be fetched before its data is valid (returns zero) in a narrow timing window (reproduces at PUMICE_SEED=2, the burst-pause / hit-miss-oscillation pattern). **Root-cause and fix** the read-fetch gating in the open-page path before promoting the HAS to formal status. The `r_fdone` fill-complete gating in the CAMs is the relevant mechanism.

31.2 TODOs Before v0.2

- Add bit-level pinout tables for the AXI, DFI, and `cpuif` (register) interfaces
- Add timing diagrams for the WR / RD command issue, including CWL / CL alignment
- Add sequence diagrams for `init_sequencer` (DDR2 and LPDDR2 MR init) and, where present, the optional `powerdown_ctrl`; document the FSM-free `bank_timer` countdown timers rather than an FSM diagram
- Add a draft of the Verilog package skeletons for `ddr2_init_steps_pkg.sv` and `lpddr2_init_steps_pkg.sv`
- Cross-reference each section to the corresponding pre-aspec.md bullet

- Add quantitative area / power estimates per module (synthesis pass needed)
- Add waveform examples for the canonical init sequences

31.3 Feature Roadmap — planned advanced modes (ordered)

The full catalog of planned, config-bit-selectable advanced modes lives in docs/design-requirements.md (“Advanced modes — selectable scheduling / paging / refresh”); the family-level split (commodity-legal here vs model-only parked for DDR3/DDR4) is in vault/Tasks/memory-controllers/ADVANCED_MODES_ROADMAP.md. Entry gate (TASKS.md TASK-FEATURES): board reads validated at the bring-up tuple, refresh-collision fix re-soaked on silicon.

Serial pre-silicon implementation order (each step OFF-by-default with its own red-to-green model test):

1. **Foundation** — SCHED_POLICY / PAGE_POLICY_CFG / REFRESH_MODE mode-select CSRs + *_STATS telemetry + PHY capability straps (no behavior change; defaults bit-identical).
2. **Scheduling (Axis 1)** — in_order -> fr_fcfs (confirm current) -> age_threshold -> most/fewest_pending -> ACCESS_PREF -> write-batching -> **QoS** (AxQoS-aware pick, QOS_EN).
3. **Paging (Axis 2)** — static_open/close (confirm) -> fixed_open. (adapt_time/adapt_access retired 2026-09-27; rbl_static/rbl_dyn retired 2026-09-26. Modes 4..7 fall through to the build default.)
4. **Refresh (Axis 3, commodity)** — JEDEC pull-in/postpone sweep -> refbp_rr.

None of the mode-select CSRs above exist in the RDL yet; open issue 13 (QoS) is subsumed by step 2 of this order.